

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

Навчально-науковий інститут комп'ютерних систем

Кафедра інженерії програмного забезпечення

Катерина ГРАТІЛОВА

(група АС-221)

КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА

Програмна система підтримки бізнес-процесів ресторану

Спеціальність:

121 Інженерія програмного забезпечення

Освітньо-професійна програма:

Інженерія програмного забезпечення

Керівник:

Олексій КУНГУРЦЕВ, к.т.н., професор

АНОТАЦІЯ

Гратілова К. О. Програмна система підтримки бізнес-процесів ресторану : кваліфікаційна робота бакалавра за спеціальністю «121 Інженерія програмного забезпечення» / Катерина Олександрівна Гратілова ; керівник Олексій Борисович Кунгурцев. – Одеса : Нац. ун-т «Одес. політехніка», 2026. – 105 с.

Кваліфікаційна робота містить основну текстову частину на 78 сторінках, список використаних джерел з 16 найменувань на 2 сторінках, додатки на 20 сторінках.

Метою кваліфікаційної роботи є підвищення ефективності обслуговування клієнтів ресторану шляхом автоматизації процесів ресторанного бізнесу на основі вебзастосунку, що скорочує час оформлення бронювання клієнтом до 3 хвилин.

У роботі проведено аналіз предметної області ресторанного бізнесу та наявних аналогів, визначено їх недоліки та обґрунтовано необхідність створення комплексного рішення. Сформульовано функціональні та нефункціональні вимоги до системи, розроблено архітектуру, модель даних і реалізовано клієнтську та серверну частини програмної системи з використанням сучасних технологій.

У результаті створено програмну систему, яка забезпечує зручне онлайн-бронювання столиків, можливість попереднього замовлення страв із відображенням харчової цінності, підтримку роботи персоналу через окремий інтерфейс, а також систему відгуків. Проведене тестування підтвердило відповідність системи встановленим вимогам та її готовність до практичного використання.

Ключові слова: програмна система, ресторанний бізнес, онлайн-бронювання, попереднє замовлення, харчова цінність, Spring Boot, PostgreSQL, React, REST API, автоматизація.

ABSTRACT

Hratilova K. O. Software system for supporting business processes for a restaurant : bachelor's qualification work in the specialty «121 Software engineering» / Kateryna Oleksandrivna Hratilova ; supervisor Oleksii Borysovykh Kungurtsev. – Odesa : Odesa Polytech. Nat. Univ., 2026. – 105 p.

The qualification work contains the main text part on 78 pages, a list of used sources with 16 titles on 2 pages, appendices on 20 pages.

The purpose of the qualification work is to improve the efficiency of restaurant customer service through automation of restaurant business processes using a web application, which reduces the time required for a customer to make a reservation to 3 minutes.

The work analyzes the subject area of the restaurant business and existing solutions, identifies their shortcomings, and justifies the need to create a comprehensive solution. Functional and non-functional requirements for the system were formulated, the architecture and data model were developed, and the client and server components of the software system were implemented using modern technologies.

As a result, a software system was created that provides convenient online table reservations, the ability to pre-order dishes with nutritional information displayed, support for staff operations via a separate interface, as well as a feedback system. Testing confirmed the system's compliance with established requirements and its readiness for practical use.

Keywords: software system, restaurant business, online booking, pre-order, nutritional value, Spring Boot, PostgreSQL, React, REST API, automation.

ЗМІСТ

Вступ.....	6
1 Специфікація вимог до програмної системи.....	8
1.1 Аналіз предметної області.....	8
1.2 Аналіз наявних програмних рішень.....	9
1.3 Функціональні вимоги до програмної системи	11
1.4 Нефункціональні вимоги.....	17
Висновки до розділу 1	19
2 Планування програмного проєкту.....	20
2.1 Оцінювання тривалості розробки.....	20
2.2 Розробка плану виконання проєкту.....	26
2.3 Планування реакцій на ризики	30
Висновки до розділу 2	36
3 Проєктування програмної системи	37
3.1 Архітектура програмної системи	37
3.2 Проєктування процесів у системі.....	38
3.3 Проєктування структури бази даних.....	47
3.4 Проєктування класів	45
Висновки до розділу 3	53
4 Програмна реалізація системи	54
4.1 Вибір інструментів розробки	54
4.2 Ключові елементи програмної реалізації	55
Висновки до розділу 4	57
5 Тестування програмної системи	58
5.1 Принципи вибору тестових прикладів.....	58
5.2 Функціональне тестування.....	58
5.3 Нефункціональне тестування.....	65
Висновки до розділу 5	69
6 Експлуатація програмної системи.....	70

6.1 Контрольний приклад застосування програмної системи	70
6.2 Експлуатаційні випробування програмної системи	80
Висновки до розділу 6	82
Загальні висновки.....	83
Список використаних джерел	84
Додаток А Лістинг програми	86

ВСТУП

Актуальність теми роботи. Сучасний ресторанний бізнес перебуває в стані стрімкої цифрової трансформації, де традиційні методи взаємодії з клієнтами поступово втрачають свою ефективність. Традиційні методи бронювання через телефонні дзвінки характеризуються високою ймовірністю помилок, втратою інформації та неефективним використанням робочого часу персоналу.

Проблемою також є організація попереднього замовлення страв і надання даних про їхню харчову цінність: гості хочуть заздалегідь обирати страви та отримувати деталі про них, а ресторан – ефективніше планувати роботу кухні.

Розробка програмної системи, яка об'єднає функціональність бронювання столиків, попереднього замовлення страв, спеціалізованого інтерфейсу для персоналу та системи відгуків, є актуальним завданням для підвищення ефективності роботи ресторанного бізнесу та покращення досвіду клієнтів.

Мета і задачі роботи. Метою кваліфікаційної роботи є підвищення ефективності обслуговування клієнтів шляхом автоматизації процесів ресторанного бізнесу на основі вебзастосунку, що скорочує час оформлення бронювання клієнтом до 3 хвилин.

Для досягнення поставленої мети необхідно вирішити наступні задачі:

- проаналізувати предметну область ресторанного бізнесу та існуючі системи автоматизації, виявити їх переваги, недоліки та обмеження;
- обґрунтувати функціональні та нефункціональні вимоги до системи;
- виконати планування програмного проєкту, оцінити тривалість розробки та визначити основні етапи роботи та потенційні ризики;
- розробити архітектуру системи та спроектувати модель даних відповідно до виявлених вимог;
- реалізувати серверну частину програмної системи з підтримкою рольової моделі доступу, бізнес-логіки бронювання та управління замовленнями;
- розробити клієнтську частину зі зручним інтерфейсом для клієнтів ресторану та спеціалізованим інтерфейсом для персоналу;

– виконати інтеграцію всіх компонентів системи та перевірити відповідність реалізованого функціоналу визначеним вимогам.

Об’єкт роботи. Процес розробки програмної системи для автоматизації бізнес-процесів ресторанного закладу.

Предмет роботи. Програмна система для автоматизації процесів бронювання столиків, управління попередніми замовленнями та організації зворотного зв’язку в ресторанному бізнесі з підтримкою спеціалізованого інтерфейсу для персоналу.

Практична значущість полягає в тому, що розроблена програмна система може бути впроваджена в реальному ресторанному закладі для автоматизації процесів бронювання столиків, управління попередніми замовленнями та координації роботи персоналу, що дозволить скоротити час обслуговування клієнтів, зменшити кількість помилок персоналу та підвищити якість сервісу завдяки заздалегідь відомим деталям замовлення. Розроблена програмна система є готовою основою для подальшого розширення функціональності – інтеграції з платіжними системами, додавання модуля аналітики або розробки мобільного застосунку.

Апробація матеріалів роботи. Основні результати були апробовані під час виступу на конференції «Modern Information Technology 2026». Доповідь опублікована в збірнику матеріалів конференції [1].

1 СПЕЦИФІКАЦІЯ ВИМОГ ДО ПРОГРАМНОЇ СИСТЕМИ

1.1 Аналіз предметної області

Ресторанний бізнес у сучасних умовах стикається з низкою викликів, пов'язаних з необхідністю забезпечення високої якості обслуговування клієнтів та ефективної організації роботи персоналу [2]. Традиційні методи бронювання столиків через телефонні дзвінки часто призводять до непорозумінь та втрати інформації. Клієнти змушені витратити час на очікування відповіді, не завжди можуть отримати актуальну інформацію про наявність вільних місць, а адміністратори ресторанів стикаються з необхідністю ручного ведення журналів бронювань, що збільшує ймовірність помилок.

Сучасні відвідувачі ресторанів очікують не лише якісне обслуговування під час візиту, але й зручність на етапі планування відвідування. Вони прагнуть мати можливість швидко ознайомитися з локаціями закладу, переглянути меню та заздалегідь забронювати столик у зручний для них час. Водночас відсутність прозорої системи зворотного зв'язку ускладнює для потенційних відвідувачів прийняття обґрунтованого рішення про вибір ресторану, оскільки відгуки попередніх гостей часто розпорошені по різних платформах або взагалі недоступні.

Окремою проблемою є координація роботи персоналу ресторану. Офіціанти потребують чіткої інформації про свої бронювання, щоб забезпечити належний рівень обслуговування. Відсутність централізованої системи призводить до того, що персонал дізнається про деталі бронювань безпосередньо перед приходом гостей, що не дозволяє належним чином підготуватися до обслуговування.

Важливим аспектом є також можливість попереднього замовлення страв. Багато гостей, особливо ті, хто планує відвідування ресторану у рамках ділових зустрічей або святкувань, хотіли б заздалегідь визначитися з меню, щоб не витратити час на вибір страв під час візиту. Для ресторану це створює можливість оптимізувати роботу кухні, заздалегідь підготувати необхідні інгредієнти та

розподілити навантаження на кухарів. Однак без відповідної технологічної підтримки реалізація такої можливості стає складною.

Інформація про харчову цінність страв набуває великої важливості для споживачів, які дбають про здоров'я або мають харчові обмеження [3]. Надання інформації про калорійність, вміст білків, жирів, вуглеводів та вітамінів у стравах дозволяє клієнтам робити усвідомлений вибір та підвищує довіру до закладу.

Таким чином, предметна область охоплює комплекс взаємопов'язаних процесів, що включають автоматизацію бронювання столиків, управління попередніми замовленнями, координацію роботи персоналу та організацію зворотного зв'язку з гостями. Ефективна автоматизація цих процесів вимагає створення інтегрованої програмної системи, яка об'єднає всі зацікавлені сторони та забезпечить зручну взаємодію між клієнтами та персоналом ресторану.

1.2 Аналіз наявних програмних рішень

Для визначення функціональних можливостей, які будуть реалізовані в програмній системі TasteTales, було проведено аналіз наявних систем бронювання столиків у ресторанах. Розглянуто три популярні платформи, що представляють різні підходи до вирішення завдань автоматизації ресторанного бізнесу.

Платформа OpenTable дозволяє шукати ресторани, переглядати столики та створювати бронювання [4]. Вона підтримує відгуки та інтеграцію з ресторанными системами. Проте платформа не має функціоналу попереднього замовлення страв та інформації про харчову цінність. Інтерфейс для офіціантів є обмеженим.

Система Resy орієнтована на преміум-сегмент і пропонує сучасний інтерфейс [5]. Проте вона не підтримує попереднє замовлення страв, харчову цінність у меню, має обмежений інтерфейс для офіціантів та недеталізовану систему відгуків.

Платформа TableAgent надає комплексне рішення для ресторанів, пропонуючи інструменти управління столиками та графіками [6]. Водночас вона не містить функції попереднього замовлення страв, не показує харчову цінність, не має окремого інтерфейсу для офіціантів, а її система відгуків є базовою.

Аналіз розглянутих систем показав, що жодна з них не надає повного набору функціональних можливостей, які планується реалізувати у програмній системі TasteTales. Зокрема, відсутній функціонал попереднього замовлення страв із можливістю перегляду харчової цінності позицій меню. Також наявні рішення не передбачають повноцінних окремих інтерфейсів для офіціантів. Існуючі модулі для персоналу в цих системах зведені до функцій хостес, тому є нефункціональними для офіціантів, оскільки повністю позбавлені інструментів для внесення змін в бронювання та управління попередніми замовленнями. Система відгуків у більшості аналогів не диференціює оцінки сервісу та якості кухні.

Порівняння функціональних можливостей програмної системи TasteTales та її аналогів представлено в табл. 1.1.

Таблиця 1.1 – Порівняння функціональних можливостей систем бронювання

Функціональна можливість	TasteTales	OpenTable	Resy	TableAgent
Онлайн-бронювання столиків	+	+	+	+
Перегляд локацій з описами	+	+	+	—
Попереднє замовлення страв	+	—	—	—
Перегляд меню з харчовою цінністю	+	—	—	—
Окремий інтерфейс для офіціантів	+	—	—	—
Перегляд бронювань офіціантами	+	+	+	—
Підтвердження/відхилення попередніх замовлень	+	—	—	—
Система відгуків	+	+	+	+
Окремі оцінки сервісу та кухні	+	+	—	—
Можливість залишати відгуки лише після відвідування	+	+	—	—

Результати порівняльного аналізу свідчать про те, що програмна система TasteTales матиме суттєві конкурентні переваги завдяки реалізації унікального набору функцій, які не представлені в комплексі в жодному з розглянутих аналогів.

Зокрема, поєднання таких можливостей, як попереднє замовлення страв, детальне інформування про харчову цінність, спеціалізований інтерфейс для персоналу та деталізована система відгуків, підвищує цінність як для клієнтів ресторану, так і для його співробітників.

1.3 Функціональні вимоги до програмної системи

Для забезпечення ефективної взаємодії з програмною системою TasteTales передбачено розподіл користувачів на три основні ролі, кожна з яких має визначений рівень доступу:

- гість (неавторизований користувач) – може переглядати меню, інформацію про локації, читати відгуки та реєструватися/авторизуватися в системі;
- клієнт (авторизований користувач) – має можливість бронювати столики, робити попередні замовлення страв до свого бронювання та залишати відгуки після фактичного візиту;
- офіціант (авторизований користувач): має доступ до спеціалізованого інтерфейсу для управління призначеними йому бронюваннями та обробки (підтвердження/відхилення) попередніх замовлень.

На основі визначених ролей та їхніх потреб було виділено 8 базових варіантів використання (сценаріїв) системи. Вони охоплюють повний цикл взаємодії з системою: процеси забезпечення доступу (реєстрація та авторизація), інформаційні функції (перегляд меню і локацій), основні операції клієнта (бронювання столика та попереднє замовлення страв), робочі задачі персоналу (обробка попередніх замовлень офіціантом), а також механізм оцінювання (залишення відгуку).

1.3.1 Діаграма варіантів використання. Проаналізувавши предметну область ресторанного бізнесу та потреби користувачів майбутньої системи, було складено діаграму варіантів використання, яка зображена на рис. 1.1.

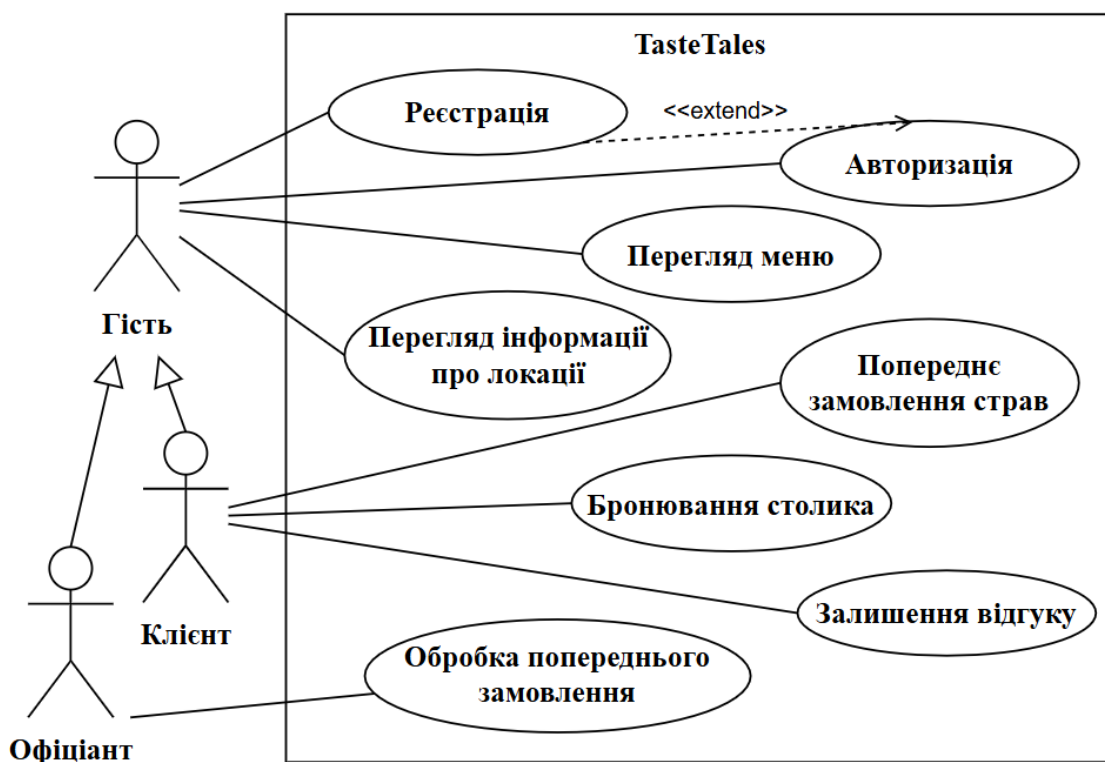


Рисунок 1.1 – Діаграма варіантів використання

1.3.2 Опис варіантів використання. Виходячи з вищенаведених варіантів використання, складемо сценарії [7] для кожного з них (сценарії 1.1 – 1.8).

Сценарій 1.1 – Реєстрація

Основна дійова особа: гість.

Результат: створено обліковий запис користувача.

Тригер: користувач відкриває сторінку реєстрації.

Основний успішний сценарій:

1. Гість переходить до сторінки реєстрації.
2. Система відображає форму для введення даних (повне ім'я, email та пароль).
3. Гість заповнює форму та натискає кнопку «Зареєструватися».
4. Система перевіряє коректність введених даних (формату email, паролю тощо).
5. Система перевіряє, чи існує вже обліковий запис з такою електронною поштою.
6. Система надсилає на вказаний email повідомлення з підтвердженням реєстрації.
7. Гість переходить за посиланням у листі та підтверджує електронну пошту.
8. Система підтверджує реєстрацію та активує обліковий запис користувача.

Альтернативні сценарії:

4а. Введені дані некоректні.

4а.1. Система виводить помилку та пропонує виправити дані. Повернення до п. 3.

5а. Обліковий запис з такою поштою вже існує.

5а.1. Система пропонує ввести іншу електронну пошту. Повернення до п. 3.

Сценарій 1.2 – Авторизація

Основна дійова особа: зареєстрований користувач (клієнт або офіціант).

Результат: користувач успішно увійшов до системи та отримав доступ до функціоналу відповідно до своєї ролі.

Тригер: користувач відкриває сторінку входу в систему.

Основний успішний сценарій:

1. Гість переходить до сторінки авторизації.
2. Система відображає форму для введення логіна (електронна пошта) та пароля.
3. Гість заповнює поля й натискає кнопку «Увійти».
4. Система перевіряє наявність облікового запису з вказаним email.
5. Система перевіряє, чи правильний пароль для вказаного облікового запису.
6. Гість успішно авторизується та отримує роль клієнта або офіціанта.

Альтернативні сценарії:

4а. Обліковий запис не знайдено.

4а.1. Система повідомляє, що обліковий запис не знайдено і пропонує ввести логін ще раз (перехід до п. 3) або створити новий обліковий запис.

4а.2. Гість погоджується створити новий обліковий запис.

4а.3. Варіант використання «Реєстрація».

5а. Введено неправильний пароль.

5а.1. Система повідомляє про неправильний пароль і пропонує повторити спробу.

5а.2. Гість повертається до заповнення полів (перехід до п. 3) або натискає на посилання «Забули пароль?».

5а.3. Система надсилає на email користувача інструкції для відновлення пароля.

5а.4. Гість слідує інструкціям у листі для скидання пароля. Перехід до п. 3.

Сценарій 1.3 – Перегляд меню

Основна дійова особа: гість або авторизований користувач.

Результат: користувач ознайомився зі стравами, їх описами та харчовою цінністю.

Тригер: перехід користувача до розділу «Меню».

Основний успішний сценарій:

1. Користувач переходить до розділу «Меню».
 2. Система відображає список страв з їх назвами, фотографіями, вагою та цінами.
 3. Користувач переглядає страви та натискає на окрему позицію меню.
 4. Система відображає вікно з описом та харчовою цінністю обраної страви.
 5. Користувач завершує перегляд меню.
-

Сценарій 1.4 – Перегляд інформації про локації

Основна дійова особа: гість або авторизований користувач.

Результат: користувач отримав повну інформацію про обрану локацію.

Тригер: перехід користувача до розділу «Локації» на головній сторінці.

Основний успішний сценарій:

1. Користувач переходить до розділу «Локації» на головній сторінці.
 2. Система відображає список локацій з їх фотографіями та короткою інформацією.
 3. Користувач обирає локацію для перегляду детальної інформації.
 4. Система відображає всі деталі локації, включаючи адресу та опис атмосфери.
 5. Система відображає спеціальні страви та відгуки обраної локації.
 6. Користувач завершує перегляд інформації про локацію.
-

Сценарій 1.5 – Бронювання столика

Основна дійова особа: авторизований клієнт.

Результат: створено бронювання столика, призначено офіціанта.

Тригер: перехід користувача до розділу «Бронювання».

Основний успішний сценарій:

1. Авторизований клієнт переходить до розділу «Бронювання».
2. Система відображає список доступних локацій ресторану з назвами.

3. Клієнт обирає бажану локацію.
4. Система відображає календар для вибору дати і часу та поле для кількості гостей.
5. Клієнт обирає дату, час та вводить кількість гостей.
6. Система перевіряє наявність вільних столиків на вибрану дату та час.
7. Система відображає доступні столики.
8. Клієнт обирає столик.
9. Клієнт підтверджує бронювання.
10. Система призначає офіціанта для обслуговування та створює бронювання.
11. Система відображає клієнту повідомлення про успішне бронювання з деталями.

Альтернативні сценарії:

- 6а. Вільні столики відсутні.
 - 6а.1. Система відображає повідомлення про відсутність вільних місць та пропонує обрати інший час або дату. Повернення до п. 5.
-

Сценарій 1.6 – Попереднє замовлення страв

Основна дійова особа: авторизований клієнт з активним бронюванням.

Результат: попереднє замовлення страв збережено та прив'язано до бронювання.

Тригер: перехід користувача до розділу «Мої бронювання» та намір замовити страви до бронювання.

Основний успішний сценарій:

1. Клієнт з активним бронюванням переходить до розділу «Мої бронювання».
2. Система відображає список активних бронювань клієнта.
3. Клієнт обирає бронювання та натискає кнопку «Попереднє замовлення».
4. Система відображає меню доступних страв з назвами, фотографіями та цінами.
5. Клієнт переглядає страви та натискає «Додати до кошика» на обраних позиціях.
6. Система додає обрані страви до кошика.
7. Клієнт переходить до кошика та переглядає обрані страви.
8. Клієнт налаштовує кількість порцій для страв або видаляє непотрібні позиції.
9. Клієнт натискає кнопку «Підтвердити замовлення».
10. Система зберігає попереднє замовлення та прив'язує його до бронювання.

11. Система відображає повідомлення про успішне оформлення попереднього замовлення та надсилає повідомлення офіціанту про необхідність його обробки.
-

Сценарій 1.7 – Обробка попереднього замовлення

Основна дійова особа: авторизований офіціант.

Результат: всі позиції замовлення отримали статус підтвердження або відхилення.

Тригер: наявність неперевіреного попереднього замовлення у офіціанта.

Основний успішний сценарій:

1. Офіціант переходить до розділу «Мої бронювання».
2. Система відображає список бронювань, призначених офіціанту, з позначками про наявність попередніх замовлень.
3. Офіціант натискає на бронювання з попереднім замовленням.
4. Система відображає список попередньо замовлених страв з кількістю порцій.
5. Офіціант переглядає список страв та консультується з кухнею щодо можливості приготування кожної з них до зазначеного часу.
6. Для кожної страви офіціант обирає: «Підтвердити» або «Відхилити».
7. Після завершення перевірки система оновлює загальний статус замовлення.
8. Система надсилає клієнту повідомлення на email, де міститься: список підтверджених страв; список відхилених страв із зазначенням причин.

Альтернативні сценарії:

- 6a. Офіціант відхиляє певну страву.
 - 6a.1. Система відкриває форму введення причини відхилення цієї страви.
 - 6a.2. Офіціант вводить причину і підтверджує відхилення.
 - 6a.3. Система фіксує статус страви як «Відхилено» та зберігає причину.
 - 6b. Офіціант підтверджує певну страву.
 - 6b.1. Система фіксує статус страви як «Підтверджено».
-

Сценарій 1.8 – Залишення відгуку

Основна дійова особа: авторизований клієнт, який має завершене бронювання.

Результат: відгук опубліковано, середні оцінки локації оновлено.

Тригер: перехід користувача до розділу «Мої бронювання» та намір поділитися враженнями після відвідування ресторану.

Основний успішний сценарій:

1. Клієнт, який відвідав ресторан, переходить до розділу зі своїми бронюваннями.
2. Система повертає історію бронювань клієнта.
3. Клієнт обирає бронювання, на котре хоче залишити відгук.
4. Система відображає форму для залишення відгуку.
5. Клієнт виставляє окремі оцінки для сервісу та кухні за шкалою від 1 до 5.
6. Клієнт вводить тексти відгуку окремо для оцінки сервісу та кухні.
7. Клієнт натискає кнопку «Опублікувати відгук».
8. Система перевіряє, чи клієнт дійсно відвідував обрану локацію.
9. Система зберігає відгук та оновлює середні оцінки сервісу та кухні для локації.
10. Система відображає повідомлення про успішне публікування відгуку.
11. Відгук стає видимим для всіх користувачів системи на сторінці даної локації.

Альтернативні сценарії:

8а. Відвідування не підтверджено.

8а.1. Система повідомляє, що неможливо залишити відгук. Завершення сценарію.

Таким чином, розроблені сценарії варіантів використання охоплюють повний цикл взаємодії користувачів із програмною системою. Кожен сценарій чітко визначає дійову особу, умову запуску та результат, що забезпечує однозначне розуміння вимог. Опис альтернативних сценаріїв дозволяє врахувати нестандартні ситуації та помилкові стани системи, що є критично важливим для забезпечення надійності та зручності системи в реальних умовах використання.

1.4 Нефункціональні вимоги

Окрім функціональних вимог, важливою складовою специфікації є нефункціональні вимоги, які визначають якісні характеристики системи та умови її коректної роботи [8]. Вони охоплюють такі ключові аспекти, як надійність,

ефективність, безпека, зручність використання та масштабованість, і є обов'язковими для забезпечення високої якості програмної системи TasteTales в реальних умовах експлуатації.

Вимоги до надійності системи:

- система повинна відпрацьовувати не менше 97% запитів користувача;
- при виникненні критичної помилки система повинна зберігати цілісність даних користувача та не допускати створення дублікатів бронювань;
- система повинна вести логування всіх критичних операцій для можливості діагностики проблем;
- кількість помилок, що виникають під час виконання запитів та непомітні користувачу, повинна становити не більше 1% від загальної кількості запитів.

Вимоги до ефективності роботи:

- система повинна обробляти кожний запит до сервера протягом не більше ніж 3 секунд для користувачів із швидкістю з'єднання 4G;
- система повинна підтримувати одночасну роботу не менше 50 одночасних користувачів без зниження продуктивності;
- система повинна зберігати дані в базі даних не більше ніж за 1 секунду після того, як сервер передав їх на зберігання.

Вимоги до безпеки даних:

- система повинна надавати всім авторизованим користувачам доступ до функцій відповідно до їх ролі (клієнт або офіціант);
- гості мають доступ до обмеженої множини функцій, а саме: перегляд опису локацій ресторану, перегляд меню, перегляд відгуків, реєстрація, авторизація;
- паролі користувачів мають зберігатися в базі даних у хешованому вигляді.

Вимоги до зручності використання:

- середній час навчання користувача основним функціям (перегляд локацій, бронювання столика, замовлення страв) не повинен перевищувати 7 хвилин;
- всі повідомлення про помилки введення користувача повинні бути чіткими і зрозумілими, щоб він розумів, чому саме виникла помилка та як її виправити;

- середній час виконання бронювання столика користувачем не має перевищувати 3 хвилини;
- після авторизації клієнт повинен мати змогу почати процес бронювання не більше ніж за два кліки;
- кольорова схема інтерфейсу повинна забезпечувати достатній контраст для читабельності тексту.

Вимоги до масштабованості системи:

- архітектура системи повинна дозволяти незалежне масштабування окремих компонентів (наприклад, модуля бронювання або обробки замовлень);
- база даних повинна підтримувати зростання обсягу даних без суттєвого погіршення часу відповіді.

Сформульовані нефункціональні вимоги створюють чітку основу для прийняття архітектурних рішень на етапі проєктування системи. Дотримання цих вимог гарантує, що програмна система буде надійною, захищеною та зручною.

Висновки до розділу 1

В першому розділі було виконано детальний аналіз аналогів та вимог до програмної системи TasteTales. Визначено ключові функціональні вимоги до системи, що включають процеси реєстрації та авторизації користувачів, бронювання столиків у ресторані, попереднє замовлення страв, обробку замовлень офіціантами, залишення відгуку, а також функціонал перегляду меню та локацій.

Розроблено детальні сценарії варіантів використання для основних функцій системи, що дозволяє чітко визначити послідовність дій користувачів та реакції системи, включаючи альтернативні сценарії при виникненні помилок або нестандартних ситуацій.

Сформульовано комплекс нефункціональних вимог, які охоплюють такі ключові аспекти, як: надійність, ефективність, безпеку, зручність використання та масштабованість. Ці вимоги забезпечують високу якість роботи системи, захист даних, швидкодію та простоту подальшого розвитку і підтримки системи.

2 ПЛАНУВАННЯ ПРОГРАМНОГО ПРОЄКТУ

2.1 Оцінювання тривалості розробки

Для оцінювання тривалості розробки програмного проєкту системи TasteTales було обрано метод Use Case Points (UCP) [9]. Вибір цього методу обґрунтовується специфікою проєкту та етапом його розробки.

По-перше, на момент оцінювання вже виконано детальний аналіз функціональних вимог та розроблено повну специфікацію варіантів використання, що є необхідною вихідною інформацією для застосування методу UCP.

По-друге, проєкт характеризується чітко визначеними акторами (гість, клієнт, офіціант) та структурованими варіантами використання, що дозволяє точно класифікувати їх за складністю.

По-третє, метод UCP враховує не лише функціональні аспекти системи, але й технічні фактори та організаційні ризики, що критично важливо для індивідуального проєкту.

Розрахунок за методом UCP виконується в п'ять послідовних етапів відповідно до методології.

2.1.1 Етап 1 – оцінка акторів системи. Для визначення нескоректованої оцінки акторів (UAW) необхідно класифікувати їх за складністю взаємодії.

Згідно з методологією UCP, актори поділяються на три типи:

- прості (вага 1) – інші системи з визначеним API;
- середні (вага 2) – системи з протокольною взаємодією;
- складні (вага 3) – користувачі, що взаємодіють через GUI або вебсторінки.

У програмній системі TasteTales визначено три актори:

- гість – неавторизований користувач, який взаємодіє з системою через вебінтерфейс (GUI) та класифікується як складний з вагою 3;
- клієнт – авторизований користувач із розширеним доступом через вебінтерфейс, який також визначається як складний з вагою 3;

– офіціант – авторизований користувач зі спеціалізованим інтерфейсом для роботи з бронюваннями, що також класифікується як складний актор з вагою 3.

Результати класифікації акторів представлені в табл. 2.1.

Таблиця 2.1 – Визначення типів акторів

Актор	Тип актора	Вага
Гість	Складний	3
Клієнт	Складний	3
Офіціант	Складний	3

Для знаходження значення UAW буде використано формулу:

$$UAW = (\text{Кількість простих акторів} \times 1) + (\text{Кількість середніх акторів} \times 2) + (\text{Кількість складних акторів} \times 3).$$

Обчислення нескоректованої оцінки акторів:

$$UAW = (0 \times 1) + (0 \times 2) + (3 \times 3) = 9.$$

2.1.2 Етап 2 – оцінка варіантів використання. Для визначення нескоректованої оцінки варіантів використання (UUCW) необхідно класифікувати кожен варіант за складністю. Згідно з методологією UCR, складність варіанту використання визначається за кількістю транзакцій (неподільних операцій) у його сценарії: прості (до 4 транзакцій, вага 5), середні (від 4 до 7 транзакцій, вага 10) та складні (більше 7 транзакцій, вага 15).

Результати класифікації варіантів використання представлені в табл. 2.2.

Для знаходження значення UUCW буде використано формулу:

$$UUCW = (\text{Кількість простих варіантів використання} \times 5) + (\text{Кількість середніх варіантів використання} \times 10) + (\text{Кількість складних варіантів використання} \times 15)$$

Таблиця 2.2 – Визначення типів варіантів використання

Варіант використання	Кількість транзакцій	Тип варіанту	Вага
Реєстрація	8	Складний	15
Авторизація	6	Середній	10
Перегляд меню	4	Середній	10
Перегляд інформації про локації	5	Середній	10
Бронювання столика	11	Складний	15
Попереднє замовлення страв	11	Складний	15
Обробка попереднього замовлення	8	Складний	15
Залишення відгуку	11	Складний	15

Обчислення нескоректованої оцінки варіантів використання:

$$UUCW = (0 \times 5) + (3 \times 10) + (5 \times 15) = 105.$$

Наступним етапом є обчислення показнику UUCP за формулою:

$$UUCP = UAW + UUCW.$$

У випадках, коли проєкт не є першою ітерацією розробки або вимагає врахування розподілу завдань в команді, до загальної оцінки застосовується коефіцієнт повторного використання коду (%REUSE). За таких умов розрахунок виконується за модифікованою формулою:

$$UUCP = (UAW + UUCW) \times \left(\frac{100 - \%REUSE}{100} \right).$$

Оскільки це перша ітерація розробки проєкту без попередньо створених компонентів для повторного використання, показник %REUSE встановлюється на рівні 0%. Таким чином:

$$UUCP = (9 + 105) \times ((100 - 0) / 100) = 114.$$

2.1.3 Етап 3 - оцінка технічних факторів. Технічні фактори відображають складність архітектури та нефункціональні вимоги до системи. Кожен фактор оцінюється за шкалою від 0 (відсутність впливу) до 5 (сильний вплив) та має вагу від 0,5 до 2.

Результати оцінки технічних факторів представлені в табл. 2.3.

Таблиця 2.3 – Оцінка технічних факторів

Фактор	Опис	Вага	Обґрунтування оцінки	Оцінка
T1	Розподіленість системи	2	Клієнт-серверна архітектура, інтеграція з БД та email	3
T2	Час відгуку	1	Вимога обробки запитів до 2 секунд	4
T3	Ефективність кінцевого користувача	1	Бронювання за 3 хв та 2 кліки після авторизації	5
T4	Складність обробки	1	Перевірка доступності, обчислення рейтингів	3
T5	Фокус на повторному використанні коду	1	Модульна архітектура без універсальної бібліотеки	2
T6	Простота інсталяції	0,5	Розгортання на хостингу без клієнтської інсталяції	2
T7	Простота використання	0,5	Інтерфейс має бути простим і зрозумілим, чіткі повідомлення	5
T8	Портативність	2	Робота на різних пристроях та браузерах	4
T9	Простота змін	1	Структурований код, швидке додавання модулів	4
T10	Паралельні обчислення	1	Підтримка 50 одночасних користувачів	2
T11	Засоби захисту	1	HTTPS, хешування паролів, рольова модель	5
T12	Доступ до третьої сторони	1	Email-сервіс, можлива інтеграція платежів в майбутньому	3
T13	Потреби в спеціальному навчанні	1	Навчання до 15 хвилин, без спеціальних тренінгів	1

Для визначення загального показника TFactor необхідно обчислити суму добутків ваги та встановленої оцінки для всіх технічних факторів:

$$\begin{aligned} \text{TFactor} = & (2 \times 3) + (1 \times 4) + (1 \times 5) + (1 \times 3) + (1 \times 2) + (0,5 \times 2) + (0,5 \times 5) + (2 \times 4) + (1 \times 4) + \\ & + (1 \times 2) + (1 \times 5) + (1 \times 3) + (1 \times 1) = 46,5. \end{aligned}$$

Для визначення значення технічного фактору застосовується така формула:

$$\text{TCF} = 0,6 + (0,01 \times \text{TFactor}).$$

Обчислення оцінки технічного фактору:

$$\text{TCF} = 0,6 + (0,01 \times 46,5) = 0,6 + 0,465 = 1,065.$$

2.1.4 Етап 4 - оцінка зовнішніх факторів. Зовнішні фактори відображають організаційні ризики та характеристики команди розробки. Кожен фактор оцінюється за шкалою від 0 до 5 та має вагу від -1 до 2.

Результати оцінки зовнішніх факторів представлені в табл. 2.4.

Значення показника EFactor розраховується шляхом підсумовування добутків вагових коефіцієнтів та оцінок, присвоєних кожному із факторів:

$$\text{EFactor} = (1,5 \times 3) + (0,5 \times 2) + (1 \times 4) + (0,5 \times 3) + (1 \times 5) + (2 \times 4) + (-1 \times 3) + (-1 \times 1) = 20.$$

Розрахунок оцінки зовнішнього фактору виконується за такою формулою:

$$\text{EF} = 1,4 + (-0,03 \times \text{EFactor}).$$

Таблиця 2.4 – Оцінка зовнішніх факторів

Фактор	Опис	Вага	Обґрунтування оцінки	Оцінка
F1	Знайомство з процесом розробки	1,5	Середній досвід веброзробки, обмежений досвід у ресторанному бізнесі	3
F2	Досвід подібних проєктів	0,5	Обмежений досвід систем бронювання та складної бізнес-логіки	2
F3	Досвід об'єктно-орієнтованої розробки	1	Хороші навички ООП та проєктування архітектури	4
F4	Навички системного аналізу	0,5	Середній рівень навичок самостійного збору та формалізації вимог	3
F5	Мотивація	1	Висока мотивація до якісного виконання роботи	5
F6	Стабільність вимог	2	Стабільні вимоги з можливими незначними коригуваннями	4
F7	Часткова зайнятість розробника	-1	Розробка паралельно з навчанням та іншими активностями	3
F8	Складність мови програмування	-1	Знайомі технології з потребою поглиблення окремих аспектів	1

Обчислення зовнішнього фактору:

$$EF = 1,4 + (-0,03 \times 20) = 1,4 - 0,6 = 0,8.$$

2.1.5 Етап 5 - результуюча оцінка. Скоректовані Use Case Points обчислюються за наступною формулою:

$$AUCP = UUCP \times TCF \times EF.$$

Обчислення скоректованих UCP:

$$AUCP = 114 \times 1,065 \times 0,8 = 97,128.$$

Оцінка часу розробки вимагає підрахунку тих факторів із групи F1–F8, абсолютне значення оцінки яких становить більше ніж 3. У нашому випадку це фактори F3 (оцінка 4), F5 (оцінка 5) та F6 (оцінка 4) – всього 3 фактори. Згідно з методологією, при результаті 3 або 4 одному UCP відповідає 28 робочих годин.

Загальна тривалість розробки:

$$\text{Трудовіткість} = 97,128 \times 28 = 2719,6 \approx 2720 \text{ людино-годин.}$$

При 8-годинному робочому дні це становить:

$$\text{Тривалість} = 2720 / 8 = 340 \text{ робочих днів.}$$

Або при 5-денному робочому тижні:

$$\text{Тривалість} = 340 / 5 \approx 68 \text{ робочих тижнів.}$$

Однак враховуючи часові рамки виконання кваліфікаційної роботи, необхідно оптимізувати процеси проєкту, зосередившись на реалізації основного функціоналу та інтенсифікувати процес розробки до 6-7 годин на день, що дозволить завершити проєкт за реалістичний термін близько 4-5 місяців.

2.2 Розробка плану виконання проєкту

Планування виконання проєкту є критично важливим етапом, який визначає послідовність робіт, їх взаємозалежності та часові рамки реалізації. Для структурування проєкту розробки програмної системи TasteTales було використано метод декомпозиції робіт (Work Breakdown Structure, WBS), який дозволяє розбити загальний обсяг робіт на керовані компоненти та встановити логічну послідовність їх виконання.

2.2.1 Структура декомпозиції робіт (WBS). Проєкт розробки програмної системи TasteTales було поділено на сім основних робіт [10].

Робота «Аналіз предметної області та вимог» передбачає дослідження ресторанного бізнесу та роботи офіціантів, аналіз систем-аналогів, уточнення функціональних і нефункціональних вимог. Результат – специфікація вимог.

Робота «Проектування архітектури та моделі даних» включає визначення архітектури системи, розробку діаграм послідовності, побудову діаграми класів, проектування структури бази даних та вибір технологічного стеку. Результат – документ архітектури з діаграмами компонентів та ER-діаграмою.

Робота «Реалізація бази даних» передбачає створення бази даних, виконання SQL-скриптів для створення таблиць з відповідними зв'язками, налаштування індексів та створення тестових даних. Результатом є налаштована база даних.

Робота «Розробка серверної частини» передбачає розробку API-сервера, реалізацію модулю аутентифікації, реалізацію бізнес-логіки бронювання столиків, розробку функціоналу попередніх замовлень, створення модуля для офіціантів, реалізацію системи відгуків. Результатом є повнофункціональний API-сервер.

Робота «Розробка клієнтської частини» зосереджена на створенні вебінтерфейсу, сторінок реєстрації та авторизації, перегляду локацій, меню з фільтрами, бронювання, кошика, інтерфейсу офіціантів та відгуків. Результат – готовий користувацький інтерфейс.

Робота «Тестування системи» спрямована на розробку тест-кейсів для всіх функціональних вимог, проведення тестування вимог, проведення модульного тестування, виправлення помилок. Результатом є стабільна система з успішними результатами тестування.

Робота «Підготовка документації» включає створення технічної документації (архітектура, API, БД), інструкцій з розгортання, користувацького керівництва та підготовку презентації. Результат – повний комплект документації.

2.2.2 Оцінка тривалості та визначення залежностей робіт. Для кожної роботи з WBS було оцінено тривалість виконання з урахуванням складності завдань та необхідності вивчення нових технологій. Також було визначено логічні

залежності між роботами – які роботи повинні завершитись перед початком інших. Результати оцінки тривалості та визначення залежностей зображені в табл. 2.5.

Таблиця 2.5 – Тривалість та залежності робіт проекту

№	Назва роботи	Тривалість (дні)	Попередники	Обґрунтування тривалості
1	Аналіз предметної області та вимог	3	-	Дослідження процесів ресторанного бізнесу; аналіз наявних систем бронювання; формулювання вимог
2	Проектування архітектури та моделі даних	7	1	Вибір архітектури; розробка діаграм послідовності; діаграми класів; визначення структури БД; вибір технологічного стеку
3	Реалізація бази даних	2	2	Створення структури таблиць; налаштування зв'язків; налаштування індексів; заповнення тестовими даними
4	Розробка серверної частини	22	2, 3	Реалізація REST API; модуль автентифікації; бізнес-логіка бронювання; функціонал замовлення; система відгуків
5	Розробка клієнтської частини	18	4	Реалізація сторінок реєстрації та авторизації; сторінки меню; сторінки локацій; інтерфейс бронювання; функціонал замовлення; панель офіціанта
6	Тестування системи	5	5	Функціональне тестування всіх модулів; нефункціональне тестування; модульні тести; виправлення виявлених помилок
7	Підготовка документації	3	6	Технічна документація; інструкції з розгортання; керівництво користувача; підготовка презентації

Обґрунтування залежностей:

– робота 1 (Аналіз предметної області та вимог) є початковою та не має попередників, оскільки формує базу для всіх наступних робіт;

- роботи 2 (Проектування архітектури та моделі даних) залежить від роботи 1, оскільки потребує повної специфікації вимог;
- робота 3 (Реалізація бази даних) залежить від роботи 2, оскільки потребує завершені моделі даних для створення таблиць та зв'язків;
- робота 4 (Розробка серверної частини) залежить від робіт 2 та 3, оскільки потребує визначеної архітектури та готової бази даних для реалізації бізнес-логіки;
- робота 5 (Розробка клієнтської частини) залежить від роботи 4, оскільки фронтенд повинен взаємодіяти з готовими API-ендпоінтами бекенду;
- робота 6 (Тестування системи) залежить від роботи 5, оскільки комплексне тестування можливе лише після інтеграції всіх компонентів;
- робота 7 (Підготовка документації) залежить від роботи 6, оскільки документація має відображати фінальний стан системи після усунення дефектів.

2.2.3 Діаграма Ганта. На основі визначених робіт, їх тривалості та залежностей було побудовано діаграму Ганта, яка наочно відображає графік виконання проєкту з початком 1 грудня 2025 року, що зображена на рис. 2.1.

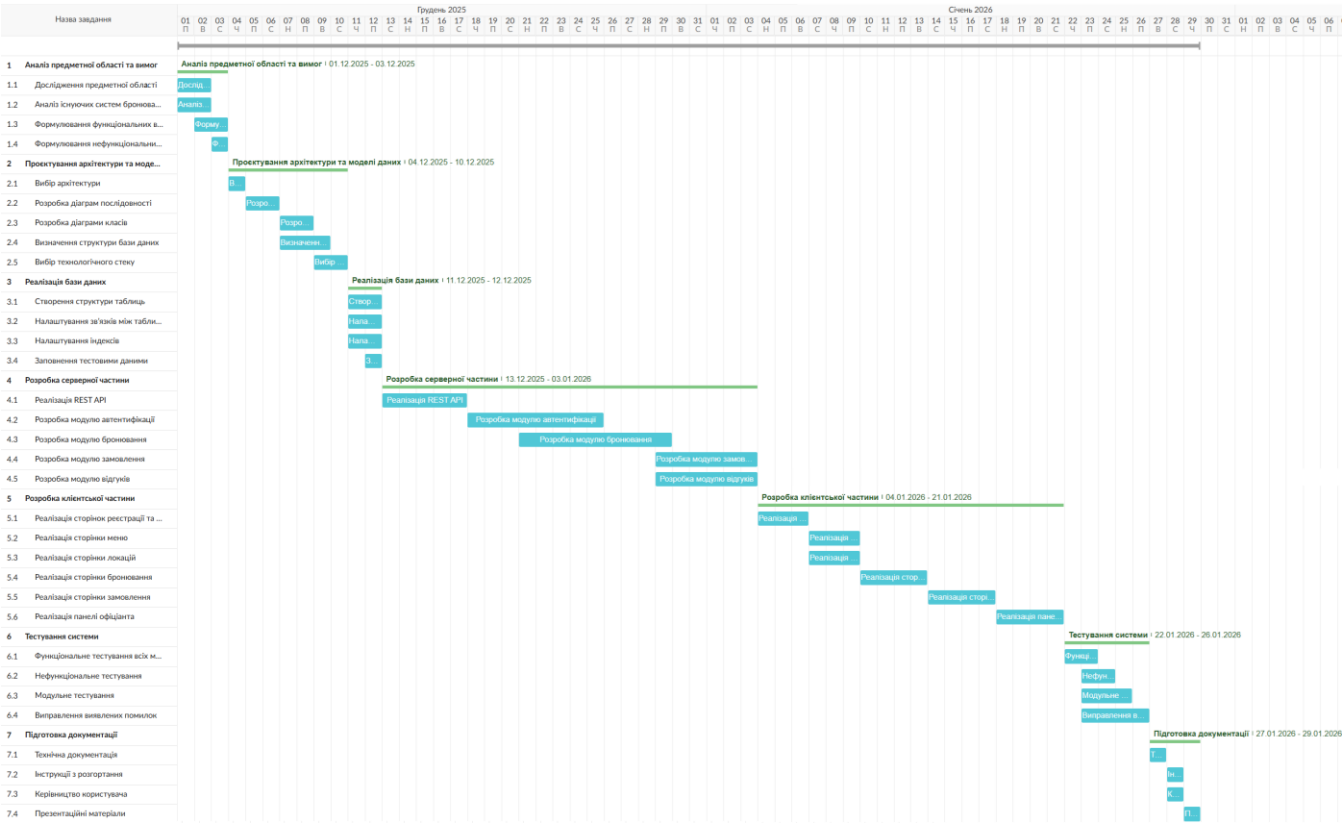


Рисунок 2.1 – Діаграма Ганта проєкту розробки програмної системи

Аналіз діаграми Ганта дозволяє зробити важливі висновки щодо організації робіт над проєктом. Чітко простежується послідовний характер виконання більшості робіт, що обумовлено їх логічними залежностями. Проєктування архітектури неможливо розпочати без завершення аналізу вимог, реалізація бази даних потребує готової моделі даних, а розробка серверної частини вимагає як завершенної архітектури, так і створеної бази даних.

Діаграма виявляє найбільш трудомісткі етапи розробки. Проєкт має обмежені можливості для паралелізації робіт через індивідуальний характер розробки. Будь-яка затримка на критичному шляху автоматично призведе до зсуву термінів завершення всього проєкту.

Загальна тривалість проєкту становить 60 календарних днів, що при роботі 6-8 годин на день відповідає приблизно 2 місяцям активної розробки.

2.3 Планування реакцій на ризики

Процес управління ризиками є критично важливим для успішного виконання будь-якого програмного проєкту, особливо в контексті індивідуальної розробки, де виконавець має обмежені ресурси. Специфіка розробки програмної системи TasteTales полягає в тому, що проєкт охоплює широкий спектр функціональності, від бронювання столиків до управління попередніми замовленнями страв, що створює численні потенційні точки відмови та вразливості.

2.3.1 Ідентифікація ризиків проєкту. Для ідентифікації ризиків проєкту було застосовано підхід структурованого аналізу можливих загроз. У процесі аналізу було виявлено наступні ризики:

- неефективний тайм-менеджмент (ризик R-01) – ризик полягає в неправильному розподілі часу між різними етапами розробки, що може призвести до затримок у реалізації критичних компонентів системи;

- неточне визначення функціональних вимог (ризик R-02) – ризик неповного або неправильного розуміння потреб користувачів може призвести до розробки функціоналу, який не відповідає вимогам клієнтів ресторану або персоналу;

- вибір невідповідних технологій (ризик R-03) – використання технологічного стеку, який не підходить для реалізації специфічних вимог проєкту, може призвести до технічного боргу та ускладнень у розробці;

- недостатня продуктивність системи (ризик R-04) – неоптимізовані запити до бази даних або неефективна архітектура можуть призвести до повільної роботи програмної системи, особливо при одночасній роботі багатьох користувачів або завантаженні великих обсягів даних про меню та локації;

- проблеми з інтеграцією компонентів (ризик R-05) – труднощі при з'єднанні різних модулів системи, таких як модуль бронювання, система замовлень та інтерфейс для користувачів, можуть призвести до нестабільної роботи системи та необхідності додаткового часу на налагодження;

- втрата вихідного коду проєкту (ризик R-06) – відсутність належної системи резервного копіювання та контролю версій може призвести до втрати значних обсягів розробленого коду внаслідок технічних проблем;

- технічні несправності обладнання (ризик R-07) – існує ймовірність виходу з ладу комп'ютера розробника, що може призвести до втрати робочого часу та необхідності відновлення робочого середовища на новому обладнанні.

2.3.2 Аналіз найбільш критичних ризиків. Проаналізувавши виявлені ризики, було визначено три найбільш впливових, які потребують детального дослідження та розробки стратегій реагування.

Наступні три ризики мають найвищий вплив на проєкт: R-02 (Неточне визначення функціональних вимог), R-03 (Вибір невідповідних технологій) та R-05 (Проблеми з інтеграцією компонентів). Ці ризики було обрано для поглибленого аналізу за допомогою діаграм Ішикави [11].

2.3.2.1 Неточне визначення функціональних вимог може мати катастрофічні наслідки для проєкту, оскільки призводить до розробки функціоналу, який не відповідає реальним потребам користувачів системи TasteTales.

Для аналізу кореневих причин цього ризику було побудовано діаграму Ішикави, що зображена на рис. 2.2.



Рисунок 2.2 – Діаграма Ішикави для ризику неточного визначення функціональних вимог

Аналіз діаграми виявив чотири основні категорії причин, які можуть призвести до неточного визначення функціональних вимог:

- недоліки методології – відсутність систематичного підходу до збору вимог, зокрема проведення інтерв'ю з потенційними користувачами системи, а також аналіз аналогів можуть призвести до пропуску важливих функціональних аспектів;
- недостатня компетентність – недостатній досвід розробника в створенні систем бронювання та управління замовленнями, а також відсутність глибоких знань про внутрішні бізнес-процеси ресторанного бізнесу можуть призвести до розробки функціоналу, який є технічно коректним, але незручним у використанні;
- нестача інформації та даних – відсутність доступу до реальних даних функціонуючих ресторанів та неповне розуміння специфічних потреб офіціантів ускладнюють формування точних вимог до системи;
- обмеженість інструментарію – відсутність спеціалізованих засобів прототипування для ранньої валідації вимог з потенційними користувачами може завадити своєчасному виявленню помилок у вимогах.

За результатами аналізу сформуємо стратегії реагування на цей ризик. Стратегія реагування – зниження ризику.

Заходи з реагування:

- провести детальне дослідження наявних систем бронювання ресторанів з акцентом на аналізі відгуків користувачів про зручність інтерфейсів та функціональні можливості;

- організувати неформальне інтерв'ю з 2-3 офіціантами знайомих ресторанів для валідації передбачуваних робочих процесів обробки попередніх замовлень.

2.3.2.2 Вибір технологічного стеку є важливим рішенням, яке визначає можливості реалізації функціональних та нефункціональних вимог. Невідповідний вибір технологій може призвести до технічних обмежень при реалізації складних функцій.

Для аналізу причин цього ризику було побудовано діаграму Ішикави, що зображена на рис. 2.3.

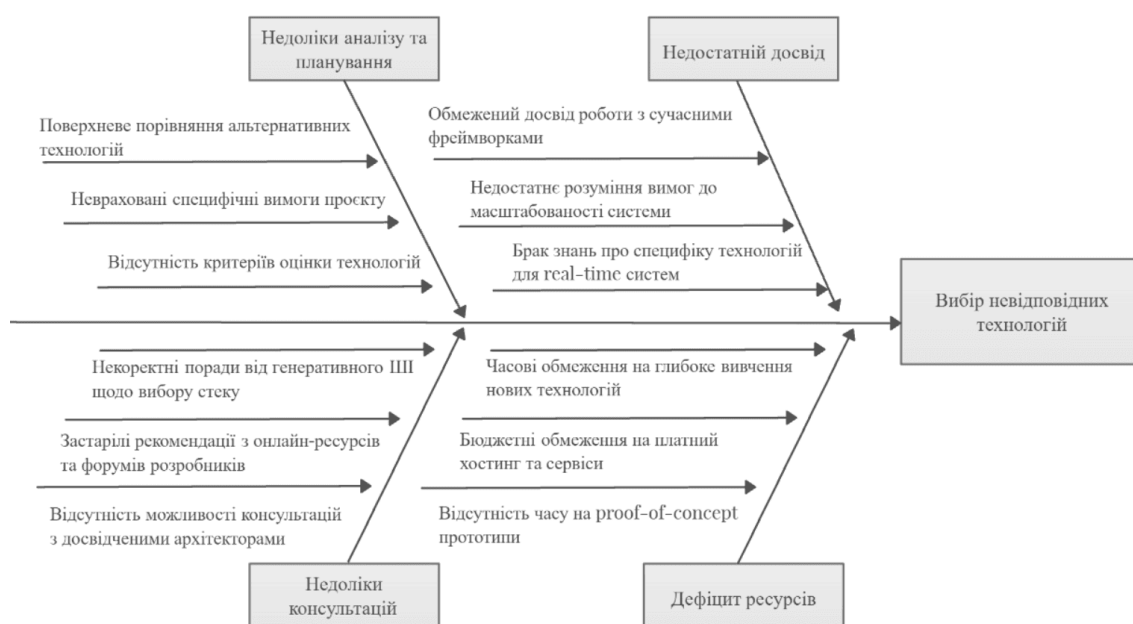


Рисунок 2.3 – Діаграма Ішикави для ризику вибору невідповідних технологій

Діаграма виявляє наступні категорії причин, які можуть призвести до невдалого вибору технологічного стеку:

- недостатній досвід — обмежений практичний досвід роботи з фреймворками та бібліотеками ускладнює оцінку їх придатності для конкретних завдань проєкту;

- недоліки аналізу та планування – поверхнєве порівняння альтернативних технологічних рішень у контексті специфічних вимог проєкту може призвести до вибору популярних, але не оптимальних інструментів;

- дефіцит ресурсів – обмежені терміни знижують можливості глибокого вивчення технологій, що може призвести до вибору не оптимальних інструментів, а бюджетні обмеження змушують використовувати безкоштовні рішення, які не завжди забезпечують потрібну функціональність і продуктивність;

- недоліки консультацій – застарілі рекомендації з онлайн-ресурсів, форумів розробників та навчальних матеріалів можуть не відображати сучасний стан технологій та їх реальну придатність для конкретних завдань.

За результатами аналізу сформуємо стратегії реагування на цей ризик. Стратегія реагування – зниження ризику.

Заходи з реагування:

- використати генеративний ШІ для отримання декількох альтернативних рекомендацій щодо технологічного стеку та перевірити актуальність порад через офіційну документацію та нещодавні публікації в технічних блогах;

- отримати експертну оцінку обраного технологічного стеку від керівника проєкту перед початком активної розробки;

- передбачити модульну архітектуру системи, яка дозволить замінити окремі компоненти без повної переробки програмної системи в разі виявлення невідповідності технологій.

2.3.2.3 Проблеми з інтеграцією компонентів становлять значну загрозу для проєкту, оскільки програмна система складається з кількох взаємопов'язаних модулів. Неправильна інтеграція цих компонентів може призвести до нестабільної роботи програмної системи, втрати даних або повної неспроможності окремих функцій системи.

Для аналізу кореневих причин цього ризику було побудовано діаграму Ішикави, що зображена на рис. 2.4.

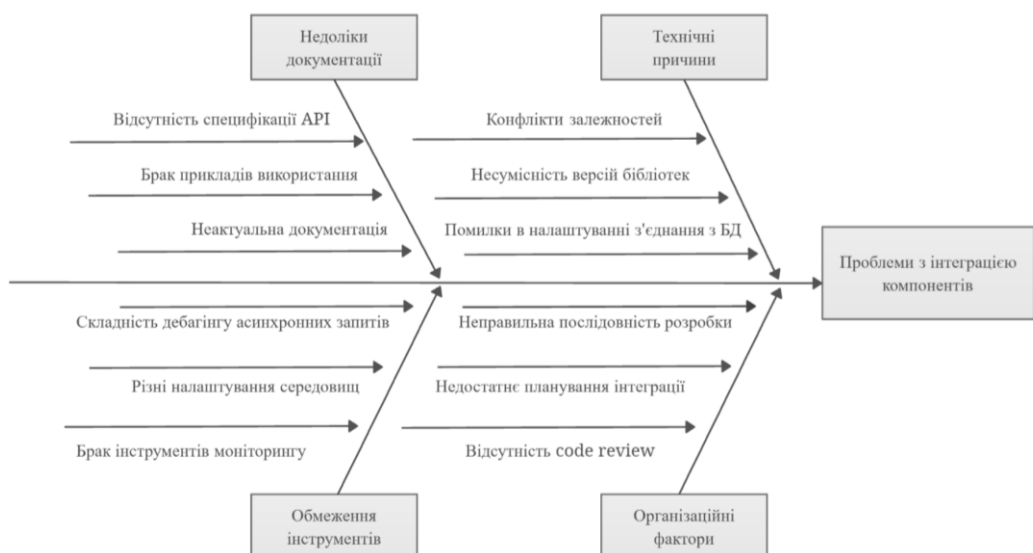


Рисунок 2.4 – Діаграма Ішікави для ризику проблем з інтеграцією компонентів

Аналіз діаграми виявив чотири основні категорії причин, які можуть призвести до проблем з інтеграцією компонентів:

— технічні причини – конфлікти залежностей між різними версіями бібліотек можуть призвести до несумісності модулів системи, а помилки в налаштуванні з'єднання з базою даних можуть блокувати доступ до даних;

— недоліки документації – відсутність специфікації API-ендпоінтів ускладнює розробку клієнтської частини, тоді як брак прикладів використання API збільшує ймовірність помилок при реалізації взаємодії компонентів;

— організаційні фактори – неправильна послідовність розробки модулів ускладнює інтеграцію, а відсутність code review не дозволяє своєчасно виявити помилки у взаємодії між компонентами;

— обмеження інструментів та середовища – відсутність ефективних інструментів для моніторингу та дебагінгу міжкомпонентної взаємодії ускладнює виявлення та усунення проблем інтеграції.

За результатами аналізу сформуємо стратегії реагування на цей ризик. Стратегія реагування – зниження ризику.

Заходи з реагування:

— реалізувати contract-first підхід: спочатку визначити контракти взаємодії між компонентами, а потім розробляти реалізацію;

- використовувати інструменти для тестування API (Postman) для верифікації роботи бекенду до початку розробки клієнтської частини;
- налаштувати детальне логування всіх запитів між компонентами для швидкої діагностики проблем інтеграції.

Висновки до розділу 2

У другому розділі було виконано комплексне планування програмного проєкту системи TasteTales, що включає оцінювання тривалості проєкту, розробку плану виконання робіт та аналіз ризиків.

Для оцінювання трудомісткості проєкту було застосовано метод Use Case Points (UCP), який дозволив врахувати як функціональні аспекти системи через аналіз трьох акторів та восьми варіантів використання, так і нефункціональні вимоги через оцінку технічних та зовнішніх факторів. За результатами розрахунків було визначено нескориговану оцінку акторів $UAW = 9$, нескориговану оцінку варіантів використання $UUCW = 105$, технічний фактор $TCF = 1,065$ та зовнішній фактор $EF = 0,8$. Скоригована оцінка проєкту становить 97,1 UCP, що відповідає трудомісткості приблизно 2720 людино-годин або 340 робочих днів.

Розроблено детальну структуру декомпозиції робіт, яка включає сім основних етапів: аналіз вимог, проєктування архітектури, реалізація бази даних, розробка серверної та клієнтської частин, тестування та підготовка документації. Для кожного етапу визначено тривалість виконання та встановлено логічні залежності. Побудовано діаграму Ганта, яка демонструє, що загальна тривалість проєкту становить 60 календарних днів. Найбільш трудомісткими є етапи реалізації серверної частини (22 дні) та клієнтської частини (18 днів).

Виконано ідентифікацію семи потенційних ризиків проєкту, серед яких три є критичними: неточне визначення функціональних вимог, вибір невідповідних технологій та проблеми з інтеграцією компонентів. Для кожного було проведено аналіз причин за допомогою діаграм Ішикави та розроблено стратегії реагування.

3 ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ

3.1 Архітектура програмної системи

Вибір архітектури програмного забезпечення є одним із найважливіших рішень на етапі проєктування системи, оскільки саме архітектура визначає здатність програмної системи відповідати функціональним та нефункціональним вимогам протягом усього життєвого циклу [12].

Для реалізації програмної системи TasteTales було обрано трирівневу архітектуру (Three-Tier Architecture). Цей архітектурний стиль передбачає логічне розділення системи на три окремі рівні, кожен з яких виконує специфічні функції та має чітко визначені межі відповідальності.

Рівень представлення є найвищим рівнем архітектури й відповідає за взаємодію з кінцевими користувачами системи. Цей рівень включає весь користувацький інтерфейс програмної системи. Взаємодія з рівнем бізнес-логіки здійснюватиметься через REST API з обміном даними у форматі JSON.

Рівень бізнес-логіки є серцевиною системи та містить всю функціональну логіку обробки бізнес-процесів ресторанного закладу. На цьому рівні буде реалізовано REST API-сервер, який надаватиме структурований набір ендпоінтів для виконання операцій системи. Будуть створені модулі для головних функцій: аутентифікації, управління бронюваннями, попередніх замовлень, відгуків тощо.

Рівень даних відповідає за зберігання та управління інформацією системи в реляційній базі даних. Структура бази даних буде спроектована відповідно до третьої нормальної форми з оптимізованою системою індексів.

Взаємодія між рівнями відбуватиметься виключно через визначені інтерфейси, що забезпечить слабке зв'язування компонентів. Рівень представлення ніколи не звертатиметься безпосередньо до рівня даних, всі операції виконуватимуться через рівень бізнес-логіки.

Діаграма компонентів трирівневої архітектури програмної системи TasteTales представлена на рис. 3.1.

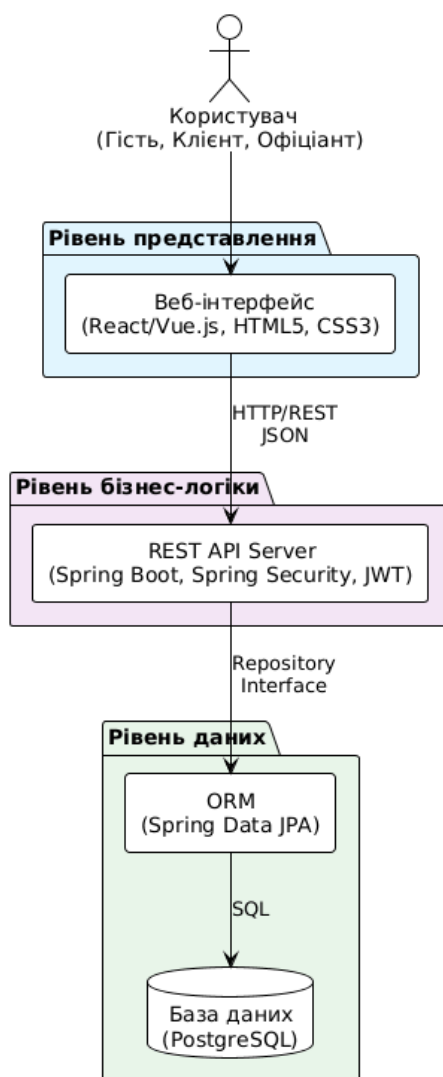


Рисунок 3.1 – Діаграма компонентів трирівневої архітектури

3.2 Проєктування процесів у системі

Для деталізації взаємодії компонентів системи під час виконання основних бізнес-процесів було розроблено діаграми послідовності [13]. Ці діаграми описують порядок обміну повідомленнями між акторами, контролерами, сервісами та репозиторіями для восьми сценаріїв використання програмної системи.

Діаграма послідовності для сценарію «Реєстрація», що зображена на рис. 3.2, показує процес створення облікового запису. GUI надсилає запит до AuthController, який делегує обробку AuthService. Сервіс перевіряє унікальність email через UserRepository, визначає роль через WaiterRepository, кодує пароль та зберігає користувача. Діаграма включає альтернативний сценарій для існуючого email.

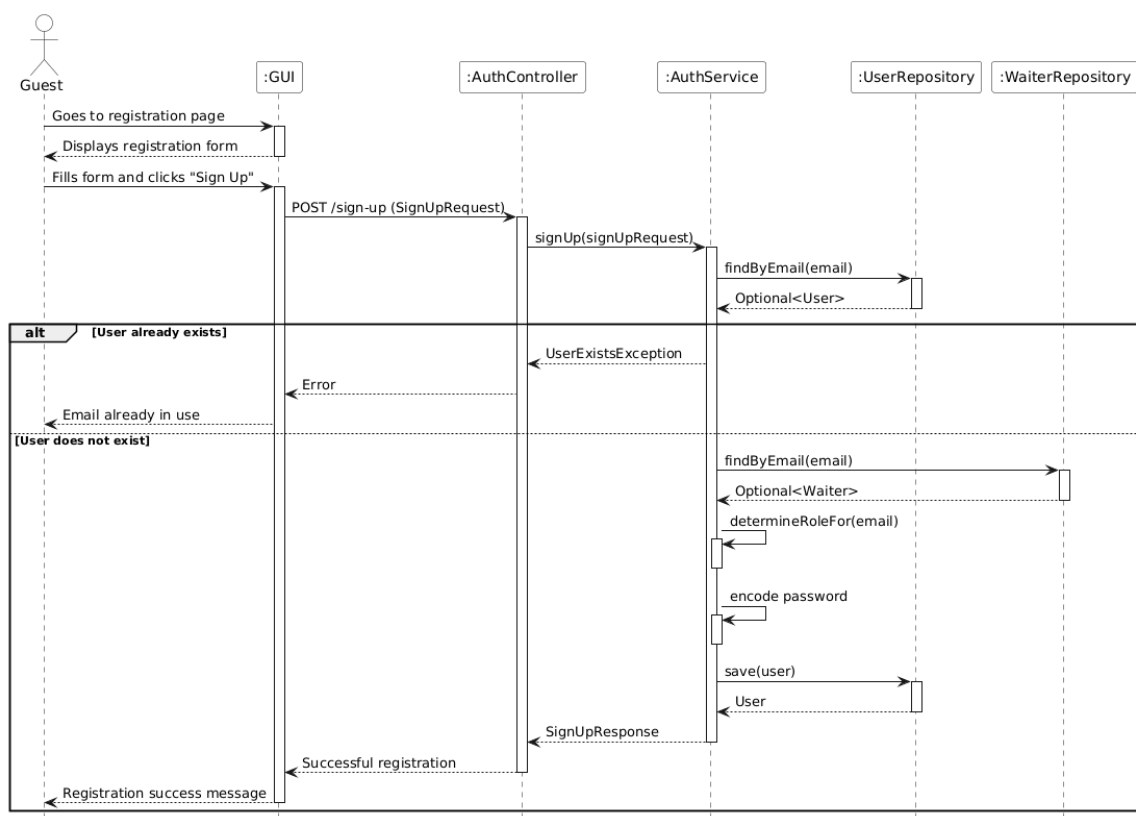


Рисунок 3.2 – Діаграма послідовності для сценарію «Реєстрація»

Діаграма послідовності для сценарію «Авторизація», що зображена на рис. 3.3, демонструє процес входу в систему.

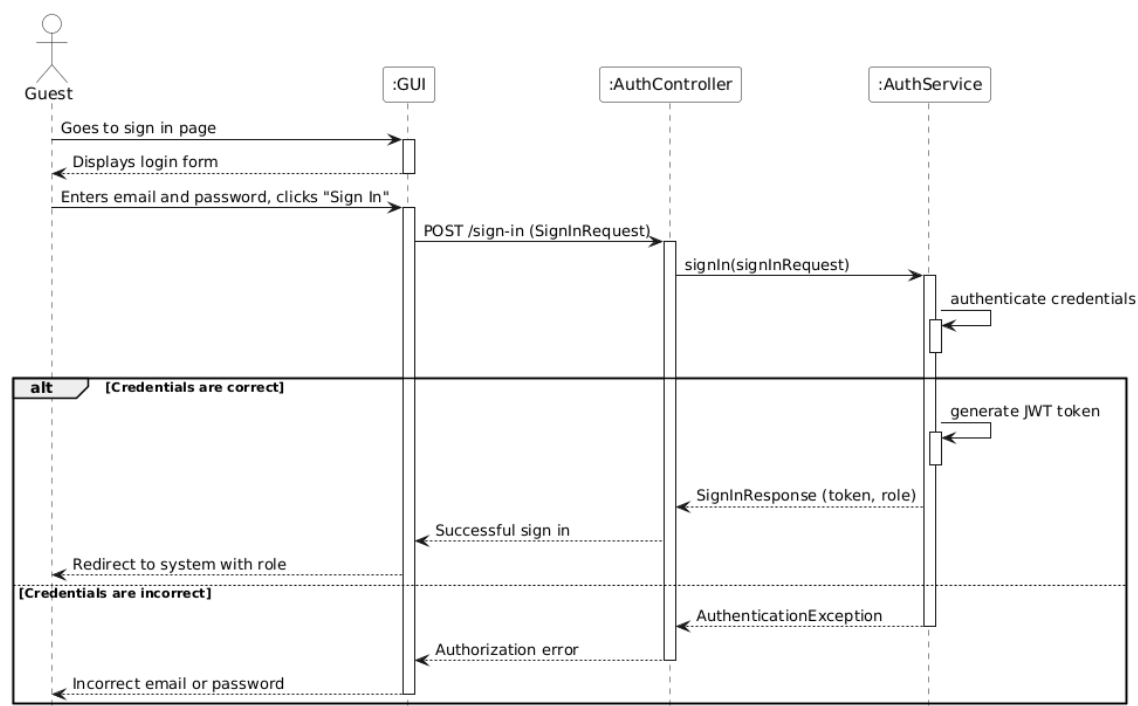


Рисунок 3.3 – Діаграма послідовності для сценарію «Авторизація»

GUI надсилає запит з обліковими даними до AuthController. AuthService перевіряє облікові дані та генерує JWT токен з інформацією про роль користувача.

Діаграма послідовності для сценарію «Залишення відгуку», що зображена на рис. 3.4, ілюструє публікацію відгуків. Користувач обирає завершене бронювання, FeedbackService перевіряє можливість залишити відгук. Після відправки форми обробку виконує FeedbackService, який використовує окремі блоки логіки для збереження оцінок сервісу та кухні. Після валідації обидва записи фіксуються у FeedbackRepository, а клієнт отримує повідомлення про успішну публікацію.

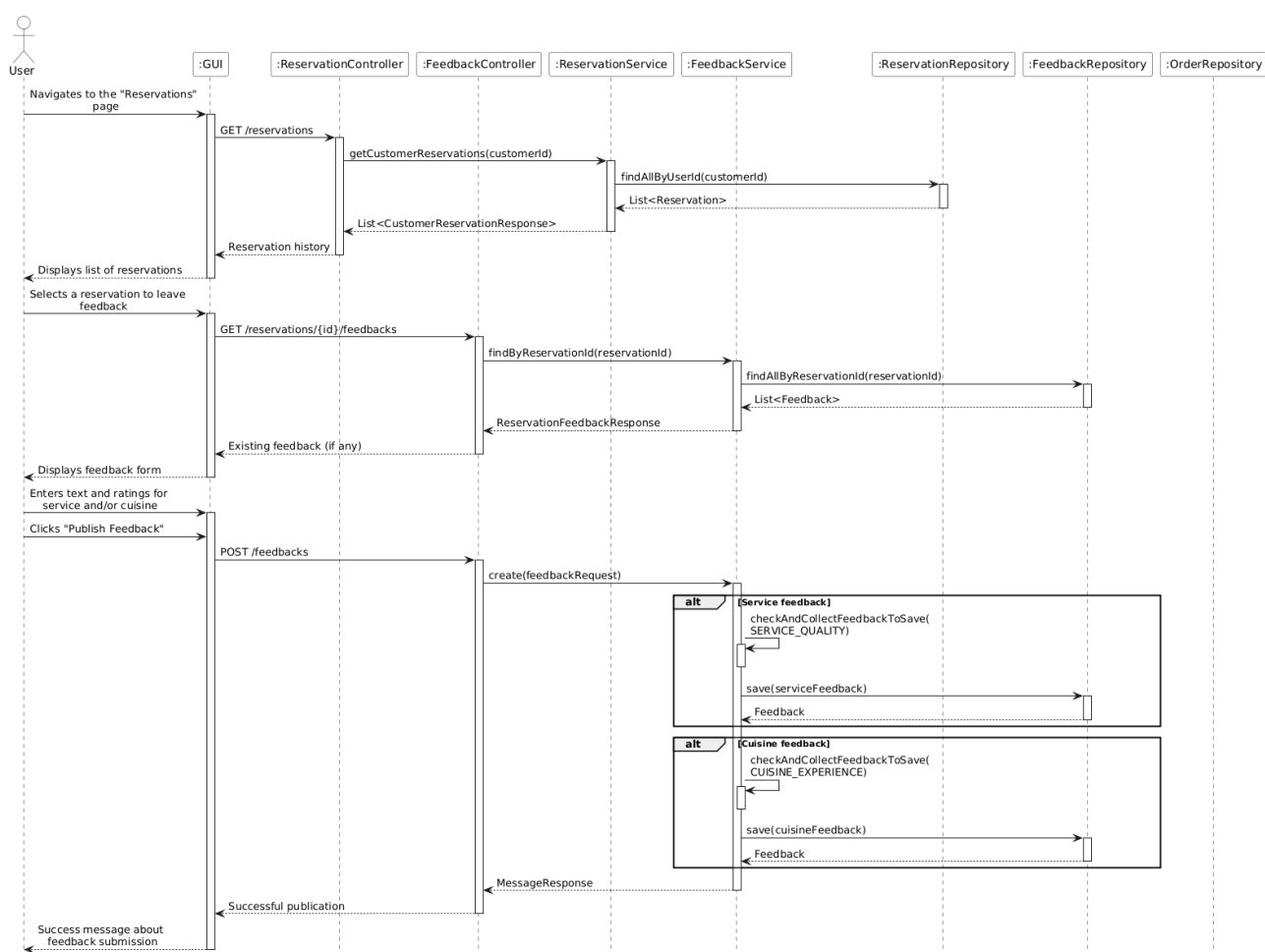


Рисунок 3.4 – Діаграма послідовності для сценарію «Залишення відгуку»

Діаграма послідовності для сценарію «Перегляд меню», що зображена на рис. 3.5, ілюструє отримання меню страв. MenuController отримує GET-запит з параметрами фільтрації та делегує обробку до MenuService, який звертається до DishesRepository. Система підтримує фільтрацію за типом страви та сортування.

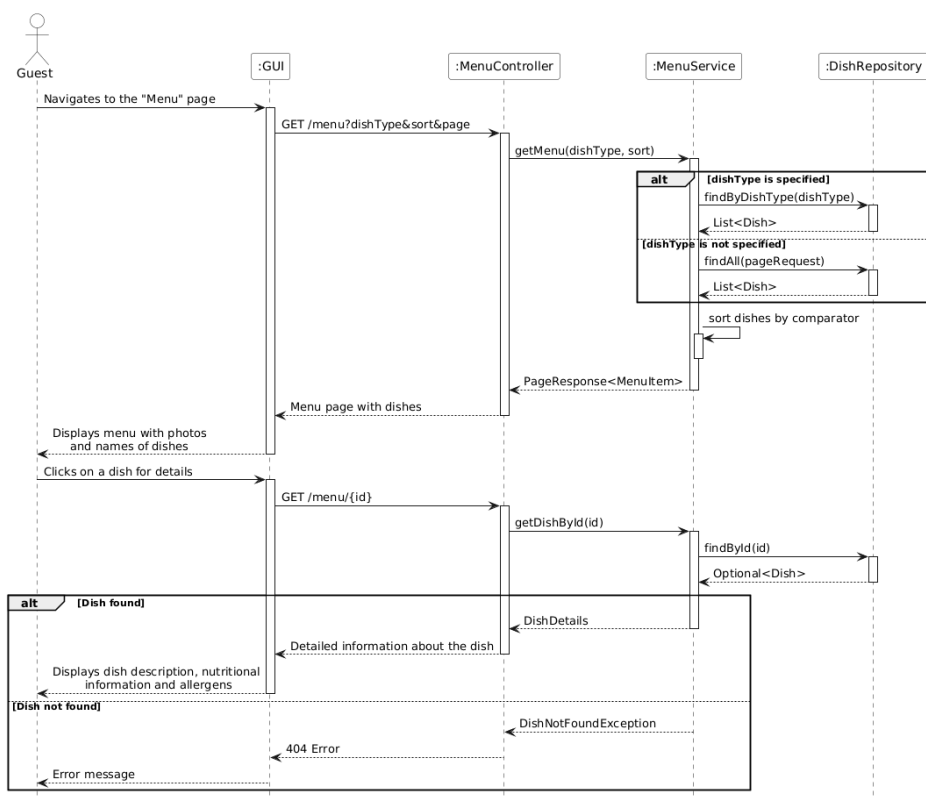


Рисунок 3.5 – Діаграма послідовності для сценарію «Перегляд меню»

Діаграма послідовності для сценарію «Перегляд інформації про локації», що зображена на рис. 3.6, описує отримання інформації про локації ресторану.

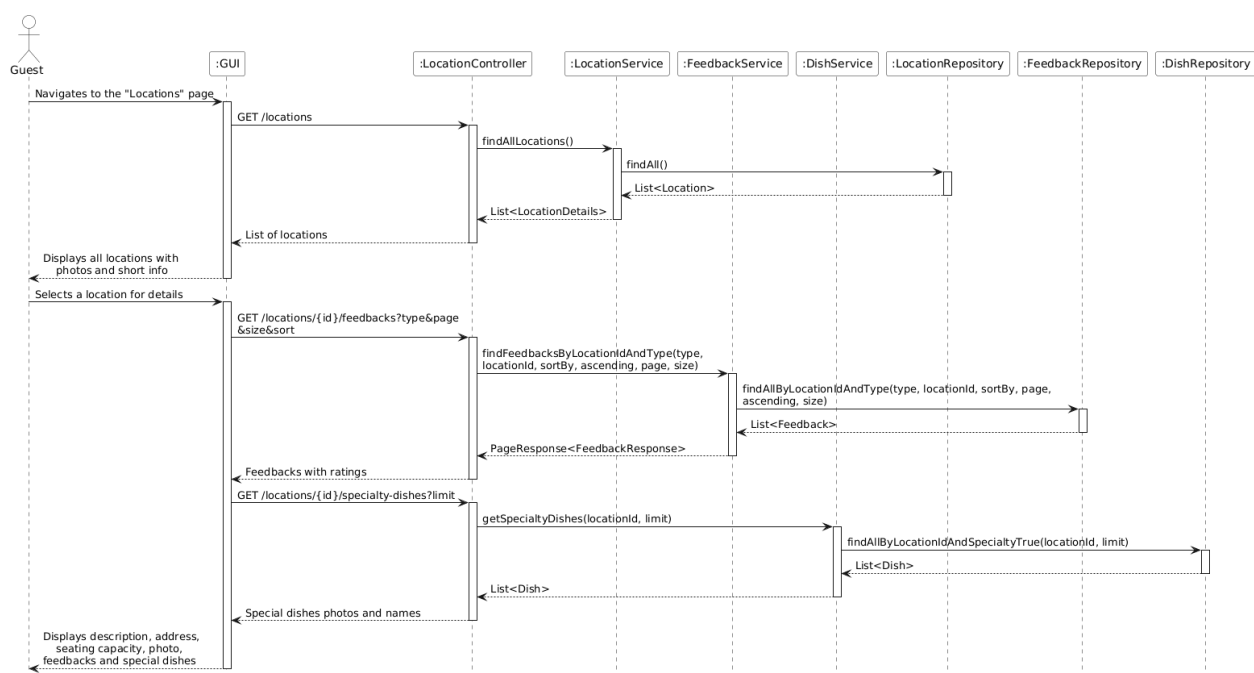


Рисунок 3.6 – Діаграма послідовності для сценарію «Перегляд інформації про локації»

LocationController через LocationService отримує список локацій з LocationRepository. Для перегляду відгуків виконується запит до FeedbackService, який отримує відгуки з FeedbackRepository з підтримкою фільтрації та пагінації.

Діаграма послідовності для сценарію «Попереднє замовлення страв», що зображена на рис. 3.7, описує процес додавання страв до кошика. Клієнт переглядає свої бронювання через ReservationController, обирає меню через MenuController та додає страви до замовлення через OrderService. Система перевіряє бронювання та додає позицію через OrderItemRepository. Після підтвердження OrderService остаточно зберігає замовлення, а його статус оновлюється на SUBMITTED.

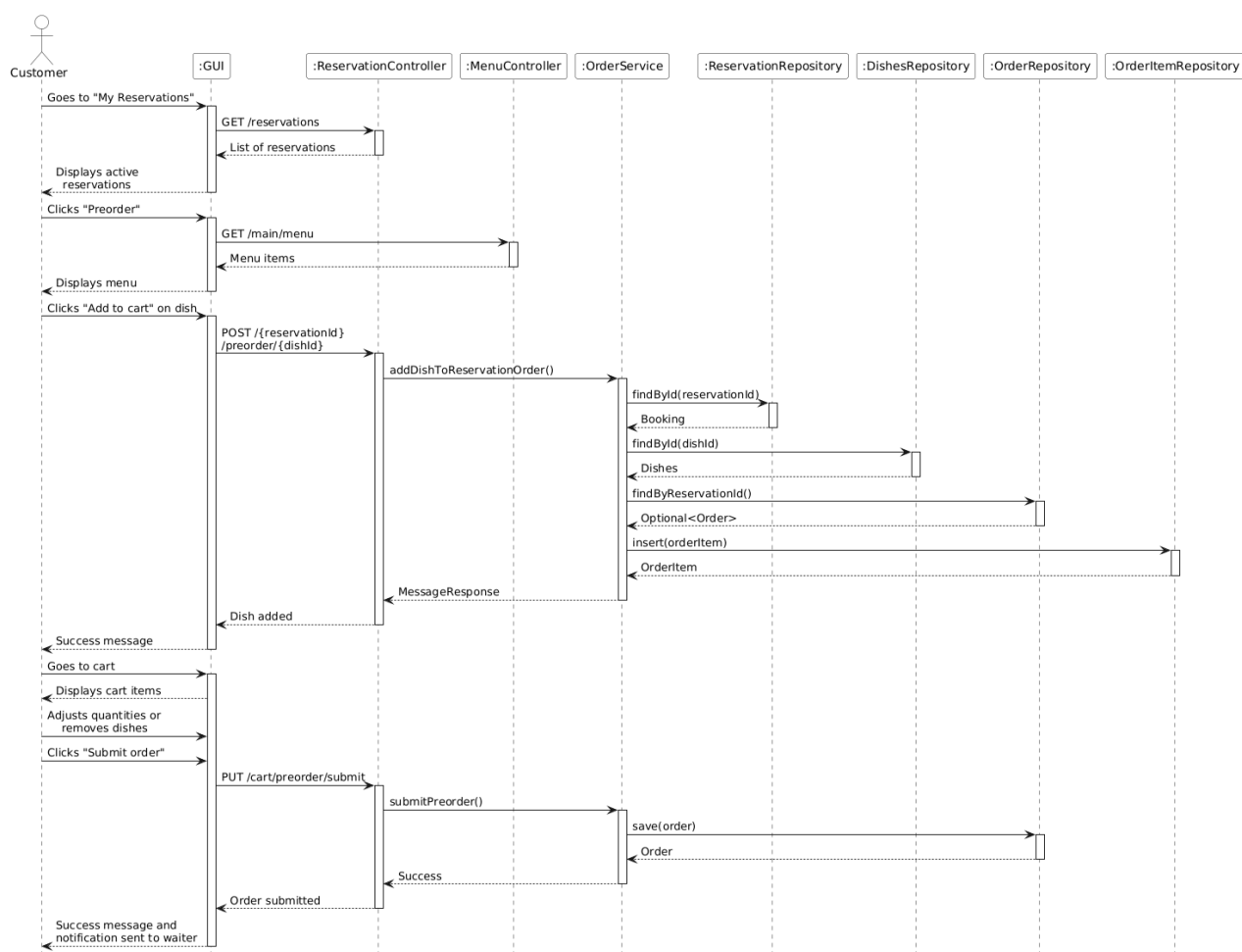


Рисунок 3.7 – Діаграма послідовності для сценарію «Попереднє замовлення страв»

Діаграма послідовності для сценарію «Обробка попереднього замовлення», що зображена на рис. 3.8, показує роботу офіціанта з попередніми замовленнями.

Офіціант отримує список своїх бронювань через ReservationService. Для кожної страви офіціант приймає рішення про підтвердження або відхилення. OrderService оновлює статуси позицій через OrderItemRepository. Після обробки всіх страв система оновлює загальний статус замовлення.

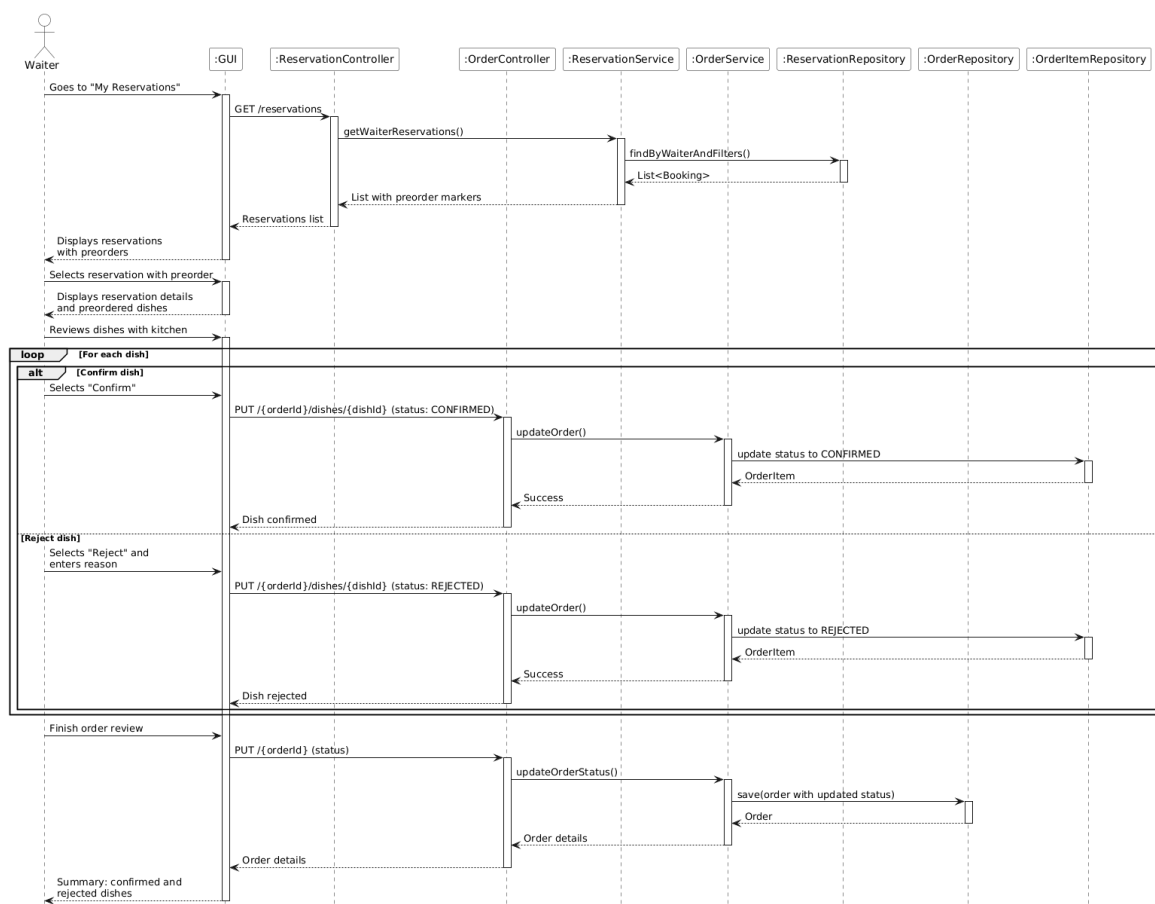


Рисунок 3.8 – Діаграма послідовності для сценарію «Обробка попереднього замовлення»

Діаграма послідовності для сценарію «Бронювання столика», що зображена на рис. 3.9, показує найскладніший процес системи. Користувач отримує список локацій через LocationService та обирає бажану. Після цього TableService перевіряє доступність столиків через ReservationRepository та TableRepository. Система фільтрує столики за місткістю та часовими слотами. Діаграма включає альтернативний сценарій відсутності вільних місць, за якого система пропонує змінити час або дату. Після вибору столика ReservationService створює об'єкт бронювання, автоматично знаходить і призначає офіціанта та зберігає дані в базу.

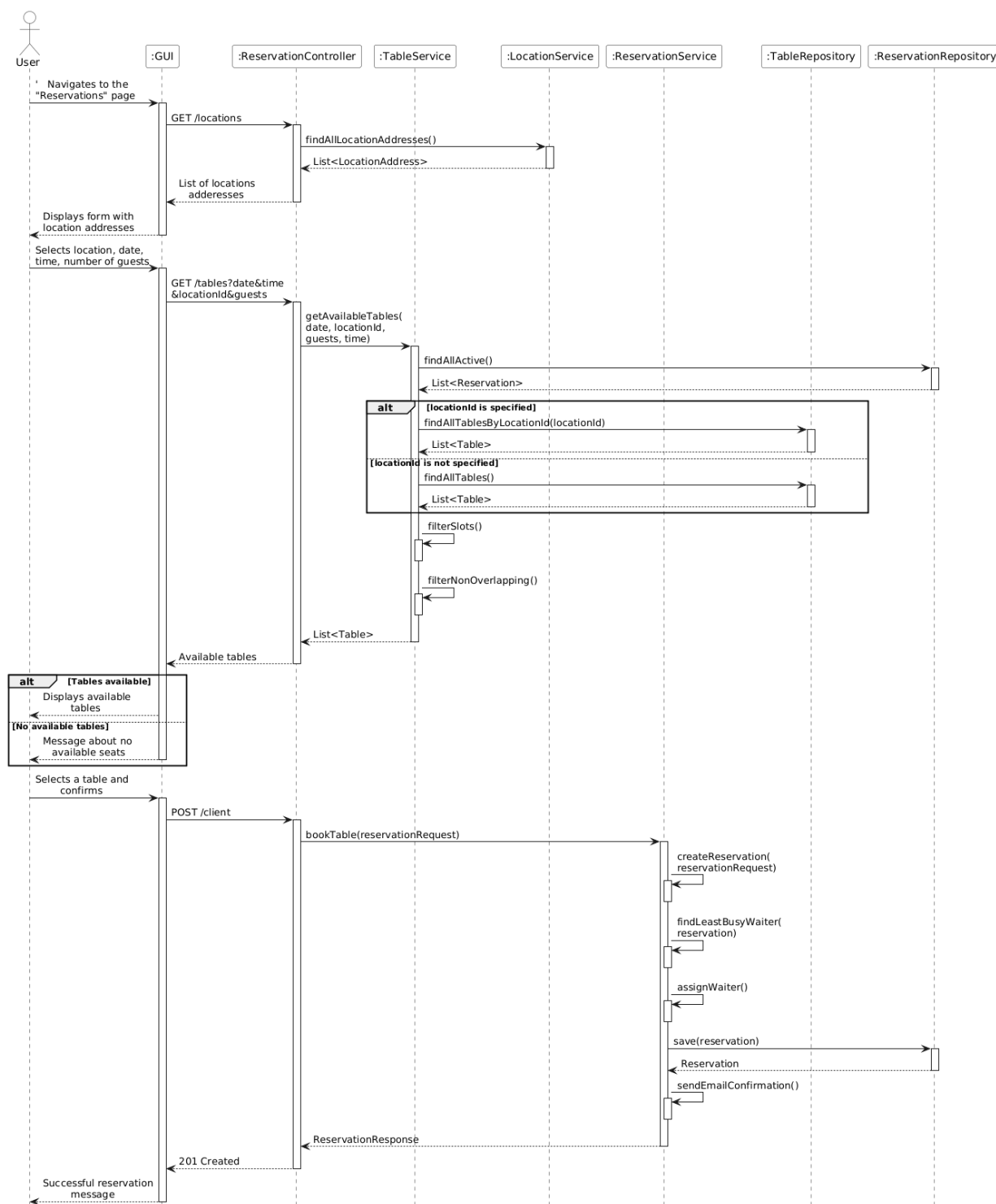


Рисунок 3.9 – Діаграма послідовності для сценарію «Бронювання столика»

Розроблені діаграми послідовності служать основою для подальшої реалізації функціоналу програмної системи. Кожна діаграма ілюструє застосування трирівневої архітектури: рівень представлення (GUI) відповідає за взаємодію з користувачем, рівень бізнес-логіки (контролери та сервіси) виконує обробку запитів, а рівень даних (репозиторії) забезпечує збереження та отримання

інформації. Діаграми відображають обробку як основних, так і альтернативних сценаріїв, що є дуже важливим для забезпечення надійності та безпеки системи.

3.3 Проєктування класів

На основі визначених у попередніх розділах функціональних вимог та діаграм послідовності можна побудувати діаграму класів, яка відображає структуру коду та взаємодію між компонентами системи. Оскільки програмна система TasteTales реалізує широкий спектр функціональності, зосередимось на сценарії бронювання столика, який є ключовою бізнес-цінністю продукту та має бути реалізований у першу чергу.

Діаграма класів для цього сценарію представлена на рис. 3.10.

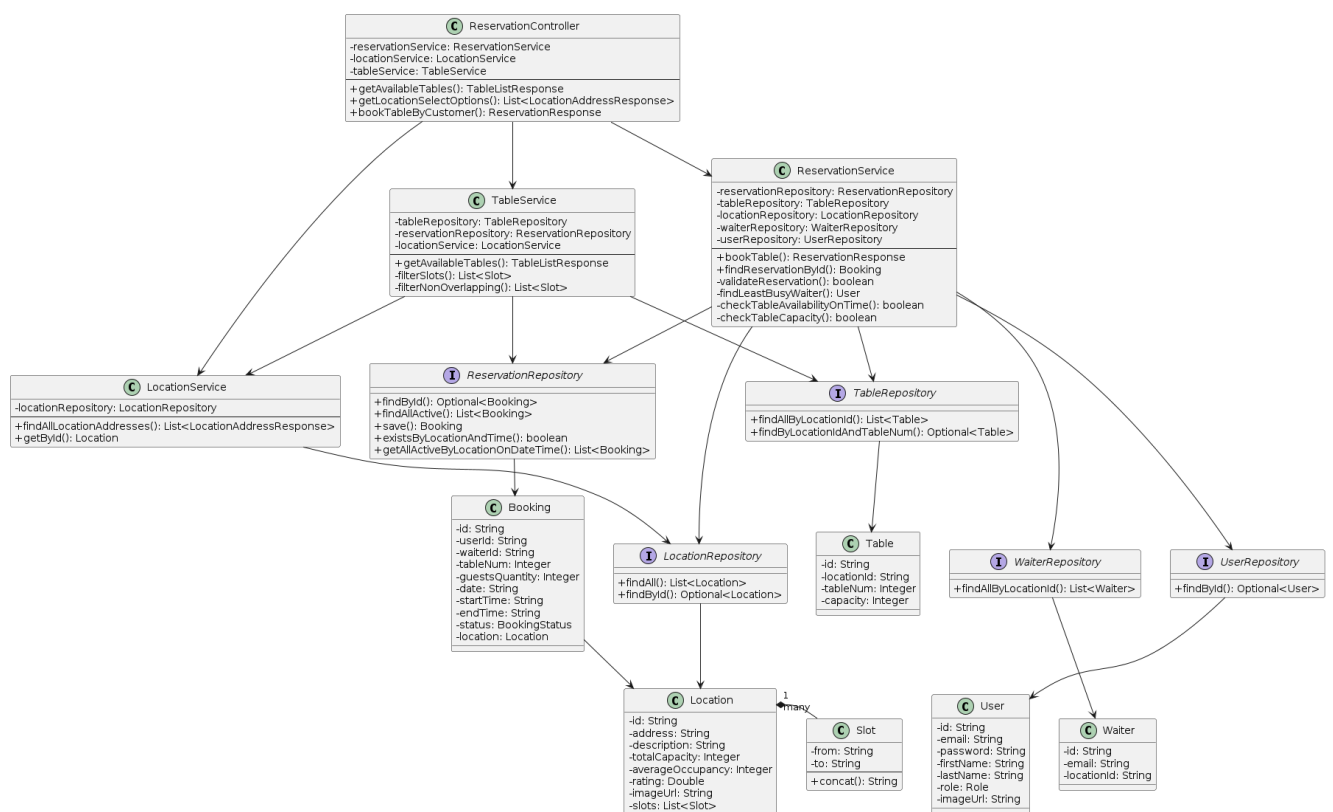


Рисунок 3.10 – Діаграма класів

Шар контролера представлений класом `ReservationController`, який є точкою входу для HTTP-запитів, пов'язаних із бронюванням.

Клас містить залежності від трьох сервісів: `ReservationService`, `LocationService` та `TableService`.

Перелік методів:

- `getAvailableTables()` – повертає список доступних столиків на основі параметрів фільтрації, таких як дата, час, локація та кількість гостей;
- `getLocationSelectOptions()` – повертає список адрес локацій для відображення у формі бронювання;
- `bookTableByCustomer()` – приймає запит на бронювання від клієнта та повертає підтвердження у вигляді `ReservationResponse`.

Шар сервісів містить три класи, кожен з яких інкапсулює відповідну частину бізнес-логіки.

`ReservationService` є центральним класом бізнес-логіки та залежить від п'яти репозиторіїв.

Публічні методи класу:

- `bookTable()` – координує повний процес бронювання: валідацію, збереження та призначення офіціанта, повертаючи `ReservationResponse`;
- `findReservationById()` – здійснює пошук бронювання за ідентифікатором та повертає сутність `Booking`.

Приватні методи класу реалізують логіку валідації та допоміжні операції:

- `validateReservation()` – агрегує всі перевірки запиту перед збереженням бронювання;
- `findLeastBusyWaiter()` – реалізує алгоритм автоматичного призначення офіціанта шляхом підрахунку активних бронювань для кожного офіціанта локації та вибору найменш завантаженого;
- `checkTableAvailabilityOnTime()` – перевіряє відсутність конфліктів із вже існуючими бронюваннями на обраний час;
- `checkTableCapacity()` – перевіряє, чи вміщує столик певну кількість гостей.

`TableService` відповідає за пошук та фільтрацію доступних столиків. Публічний метод `getAvailableTables()` координує процес пошуку, отримуючи активні бронювання та перелік столиків, після чого застосовує фільтрацію.

Приватні методи фільтрації:

- `filterSlots()` – відбирає часові слоти локації, які є актуальними;
- `filterNonOverlapping()` – виключає зі списку слоти, що перетинаються з вже існуючими бронюваннями.

`LocationService` надає методи для отримання локацій ресторану:

- `findAllLocationAddresses()` – повертає скорочений список адрес усіх локацій для відображення у формі вибору;
- `getById()` – повертає повну сутність локації за її ідентифікатором.

Шар репозиторіїв забезпечує взаємодію з базою даних через набір інтерфейсів для керування локаціями, столиками, офіціантами та безпосередньо бронюваннями. Ключовим є `ReservationRepository`, який обробляє збереження даних, пошук активних резервувань та перевірку часових конфліктів. Інші допоміжні репозиторії (`TableRepository`, `LocationRepository`, `WaiterRepository`, `UserRepository`) надають необхідні методи для фільтрації та пошуку сутностей, забезпечуючи повноцінний процес резервування та призначення персоналу.

Шар доменних сутностей включає шість класів, що відображають бізнес-об'єкти системи. `Booking` є головною сутністю та зберігає поля: `id`, `userId`, `waiterId`, `tableNum`, `guestsQuantity`, `date`, `startTime`, `endTime`, `status` та посилання на об'єкт `Location`. `Location` містить поля `id`, `address`, `description`, `totalCapacity`, `averageOccupancy`, `imageUrl`, `rating` та колекцію об'єктів `Slot`. `Slot` є допоміжною сутністю з полями `from` і `to`, що представляють межі часового інтервалу. `Table` зберігає поля `id`, `locationId`, `tableNum` та `capacity`. `User` містить поля `id`, `email`, `password`, `firstName`, `lastName`, `imageUrl` та поле `role`. `Waiter` є розширенням користувача і зберігає прив'язку до локації через поле `locationId`.

3.4 Проєктування структури бази даних

Структура бази даних програмної системи `TasteTales` була спроектована з урахуванням принципів нормалізації до третьої нормальної форми, що забезпечує мінімізацію надмірності даних та збереження цілісності інформації. Архітектура

бази даних оптимізована для ефективного виконання операцій об'єднання таблиць при обробці складних запитів, таких як отримання повної інформації про бронювання з деталями локації, столика, користувача та офіціанта. Модель даних відображає основні бізнес-сутності системи та їхні взаємозв'язки, забезпечуючи гнучкість для майбутнього розширення функціональності системи [14].

ER-діаграма структури бази даних представлена на рис. 3.11.

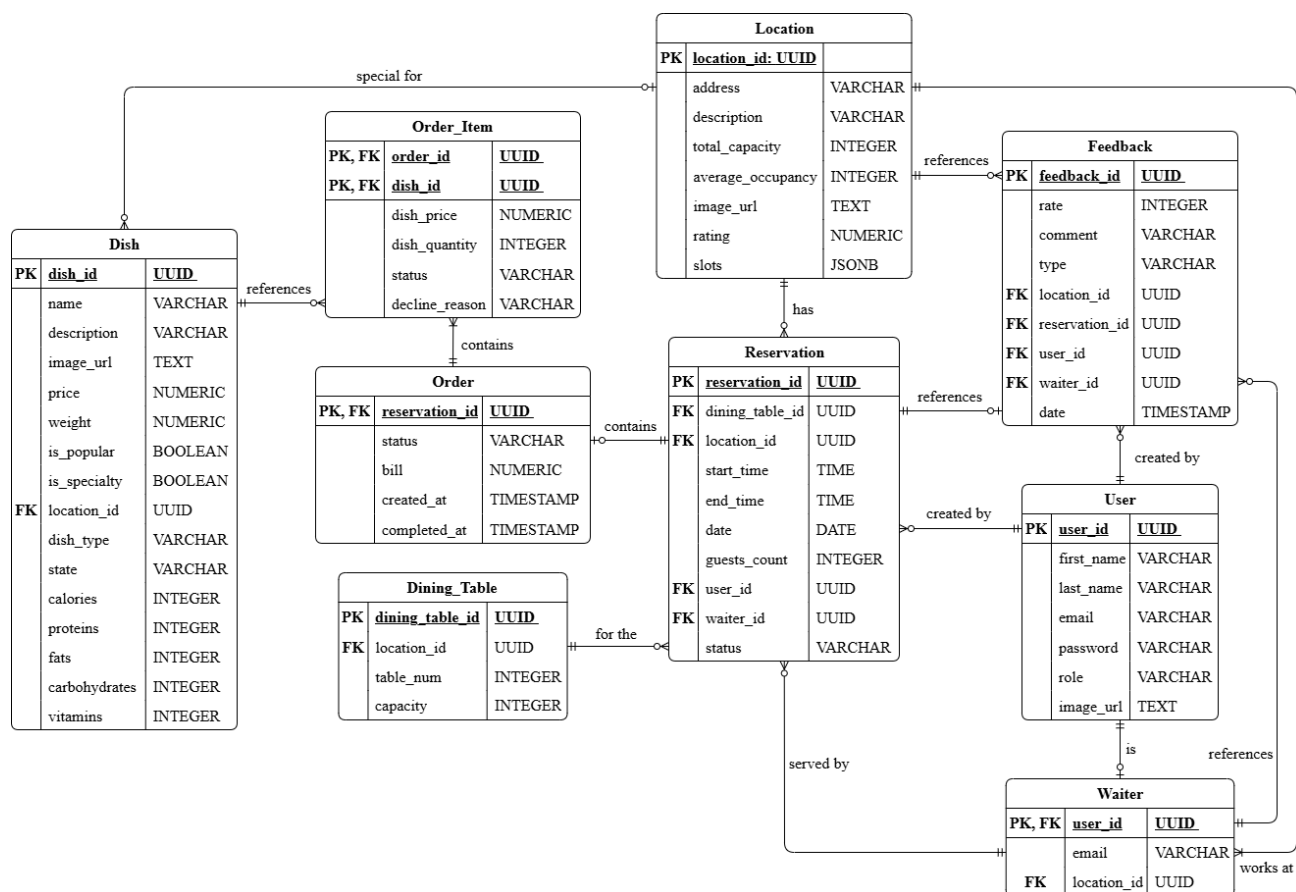


Рисунок 3.11 – ER-діаграма структури бази даних

Таблиця Reservation (табл. 3.1) містить всю інформацію про заброньовані столики, включаючи дату, час початку та завершення, кількість гостей та поточний статус. Таблиця має зв'язки з User (клієнт, що створив бронювання), Dining_Table (заброньований столик), Location (локація бронювання) та Waiter (призначений офіціант). Зв'язок з Order дозволяє прив'язувати попередні замовлення страв до бронювання, а зв'язок з Feedback забезпечує можливість додавання відгуку клієнта щодо даної резервації.

Таблиця 3.1 – Опис стовбців таблиці Reservation

Поле	Тип	Опис
reservation_id	UUID	Унікальний ідентифікатор бронювання
dining_table_id	UUID	Заброньований столик
location_id	UUID	Локація бронювання
start time	TIME	Час початку бронювання
end time	TIME	Час завершення бронювання
date	DATE	Дата бронювання
guest_count	INTEGER	Кількість гостей
user_id	UUID	Клієнт, що створив бронювання
waiter_id	UUID	Призначений офіціант
status	VARCHAR(20)	Статус: reserved, in progress, cancelled тощо

Таблиця Location (табл. 3.2) містить інформацію про локації, включаючи адресу, опис, кількість місць та середній рейтинг на основі відгуків. Поле slots зберігає інформацію про доступні часові слоти для бронювання у форматі JSON. Таблиця має зв'язки з Feedback (відгуки про локацію), Reservation (бронювання в локації), Dish (спеціальні страви) та Waiter (офіціанти, які працюють у локації).

Таблиця 3.2 – Опис стовбців таблиці Location

Поле	Тип	Опис
location_id	UUID	Унікальний ідентифікатор локації
address	VARCHAR(255)	Адреса локації
description	VARCHAR(1000)	Опис атмосфери та особливостей
total_capacity	INTEGER	Загальна кількість місць
average_occupancy	INTEGER	Середня завантаженість у відсотках
image_url	TEXT	URL головного зображення
rating	NUMERIC(3,2)	Середній рейтинг (від 1.00 до 5.00)
slots	JSONB	Доступні часові слоти для бронювання

Таблиця User (табл. 3.3) зберігає інформацію про всіх користувачів системи, включаючи як клієнтів, так і офіціантів. Поле role визначає роль користувача. Кожен користувач має унікальний ідентифікатор та контактні дані для аутентифікації. Таблиця пов'язана з таблицями Reservation (користувачі створюють бронювання), Feedback (користувачі залишають відгуки про відвідування) та Waiter (офіціант пов'язан з відповідним користувачем).

Таблиця 3.3 – Опис стовбців таблиці User

Поле	Тип	Опис
user_id	UUID	Унікальний ідентифікатор користувача
first_name	VARCHAR(100)	Ім'я користувача
last_name	VARCHAR(100)	Прізвище користувача
email	VARCHAR(255)	Електронна пошта для авторизації
password	VARCHAR(255)	Хешований пароль
role	VARCHAR(20)	Роль: client або waiter
image_url	TEXT	URL зображення профілю

Таблиця Waiter (табл. 3.4) є розширенням таблиці User для користувачів з роллю офіціанта та містить інформацію про прив'язку офіціанта до певної локації ресторану. Зв'язок один-до-багатьох з Reservation дозволяє призначати офіціантів для обслуговування бронювань, а зв'язок з Location визначає робочу локацію офіціанта. Зв'язок з Feedback визначає, на роботу якого офіціанта написано відгук.

Таблиця 3.4 – Опис стовбців таблиці Waiter

Поле	Тип	Опис
user_id	UUID	Зовнішній ключ до User (PK)
email	VARCHAR(255)	Email офіціанта
location_id	UUID	Локація, де працює офіціант

Таблиця Dining_Table (табл. 3.5) зберігає інформацію про столики в кожній локації, включаючи їх номер та місткість. Зв'язок багато-до-одного з Reservation забезпечує управління бронюваннями конкретних столиків.

Таблиця 3.5 – Опис стовбців таблиці Dining_Table

Поле	Тип	Опис
dining_table_id	UUID	Унікальний ідентифікатор столика
location_id	UUID	Локація, де знаходиться столик
table_num	INTEGER	Номер столика в межах локації
capacity	INTEGER	Кількість місць за столиком

Таблиця Dish (табл. 3.6) містить повний каталог страв меню ресторану з детальною інформацією про назву, опис, ціну, вагу та харчову цінність кожної

страви. Прапорці `isPopular` та `isSpecialty` дозволяють виділяти популярні та фірмові страви в меню. Зв'язок з `Location` визначає, чи є страва унікальною для конкретної локації, а зв'язок з `OrderItem` забезпечує включення страв до попередніх замовлень.

Таблиця 3.6 – Опис стовбців таблиці `Dish`

Поле	Тип	Опис
<code>dish_id</code>	UUID	Унікальний ідентифікатор страви
<code>name</code>	VARCHAR(200)	Назва страви
<code>description</code>	VARCHAR(1000)	Опис страви та інгредієнтів
<code>image_url</code>	TEXT	URL зображення страви
<code>price</code>	NUMERIC(10,2)	Ціна за порцію
<code>weight</code>	NUMERIC(8,2)	Вага порції в грамах
<code>is_popular</code>	BOOLEAN	Чи є страва популярною
<code>is_specialty</code>	BOOLEAN	Чи є страва унікальною для конкретної локації
<code>location_id</code>	UUID	Локація, де страва унікальна, може бути null
<code>dish_type</code>	VARCHAR(30)	Тип страви: <code>appetizer</code> , <code>main</code> , <code>dessert</code> тощо
<code>state</code>	VARCHAR(20)	Стан: <code>available</code> , <code>on_stop</code>
<code>calories</code>	INTEGER	Калорійність в ккал
<code>proteins</code>	INTEGER	Вміст білків в грамах
<code>fats</code>	INTEGER	Вміст жирів в грамах
<code>carbohydrates</code>	INTEGER	Вміст вуглеводів в грамах
<code>vitamins</code>	VARCHAR(200)	Опис вмісту вітамінів

Таблиця `Order` (табл. 3.7) представляє попереднє замовлення страв, прив'язане до конкретного бронювання. Поля `status` та `bill` зберігають поточний стан замовлення (очікує підтвердження, підтверджено, скасовано) та загальну вартість всіх замовлених страв. Таблиця має зв'язок один-до-одного з `Reservation` (замовлення для конкретного бронювання) та зв'язок один-до-багатьох з `OrderItem` (замовлені страви).

Таблиця 3.7 – Опис стовбців таблиці `Order`

Поле	Тип	Опис
<code>reservation_id</code>	UUID	Зовнішній ключ до <code>Reservation</code> (PK)
<code>status</code>	VARCHAR(20)	Статус: <code>not_submitted</code> , <code>submitted</code> , <code>cancelled</code> тощо
<code>bill</code>	NUMERIC(10,2)	Загальна вартість замовлення
<code>created_at</code>	TIMESTAMP	Дата та час створення
<code>completed_at</code>	TIMESTAMP	Дата та час завершення обробки

Таблиця Order_Item (табл. 3.8) є асоціативною таблицею, що зв'язує замовлення зі стравами та зберігає додаткову інформацію про кожну позицію замовлення. Поля dishPrice та dishQuantity дозволяють зафіксувати ціну страви на момент замовлення та кількість порцій. Поле status визначає стан конкретної позиції (очікує підтвердження, підтверджено офіціантом, відхилено), а declineReason містить причину відхилення страви, якщо офіціант не може її підтвердити. Композитний первинний ключ (orderId, dishId) забезпечує унікальність позицій у межах одного замовлення.

Таблиця 3.8 – Опис стовбців таблиці Order_Item

Поле	Тип	Опис
order_id	UUID	Зовнішній ключ до Reservation (частина РК)
dish_id	UUID	Зовнішній ключ до Dish (частина РК)
dish_price	NUMERIC(10,2)	Ціна страви на момент замовлення
dish_quantity	INTEGER	Кількість порцій
status	VARCHAR(20)	Статус: pending, confirmed, declined
decline_reason	VARCHAR(500)	Причина відхилення офіціантом

Таблиця Feedback (табл. 3.9) зберігає відгуки клієнтів про відвідування ресторану з окремими оцінками для сервісу та кухні. Поля rate (загальна оцінка), comment (текстовий коментар) та type (тип відгуку: service або cuisine) дозволяють детально оцінити різні аспекти роботи ресторану.

Таблиця 3.9 – Опис стовбців таблиці Feedback

Поле	Тип	Опис
feedback_id	UUID	Унікальний ідентифікатор відгуку
rate	INTEGER	Оцінка від 1 до 5
comment	VARCHAR(1000)	Текст відгуку
type	VARCHAR(20)	Тип: service або food
location_id	UUID	Локація, про яку відгук
reservation_id	UUID	Бронювання, після якого відгук
user_id	UUID	Автор відгуку
waiter_id	UUID	Оцінений офіціант
date	TIMESTAMP	Дата та час створення відгуку

Таблиця має зв'язки з Location (локація, про яку залишено відгук), Reservation (бронювання, після якого створено відгук), User (автор відгуку) та Waiter (офіціант, якого оцінено). Поле date зберігає час створення відгуку для сортування та аналізу змін якості обслуговування.

Розроблена структура бази даних забезпечує ефективне зберігання всієї необхідної інформації для функціонування програмної системи TasteTales, підтримує цілісність даних через систему зовнішніх ключів та оптимізована для швидкого виконання запитів завдяки раціональному розподілу даних між таблицями.

Висновки до розділу 3

У третьому розділі було виконано проєктування програмної системи, що охоплює архітектурні, процесні, структурні та класові аспекти системи.

Для системи обрано трирівневу архітектуру, яка забезпечує чіткий розподіл відповідальності між рівнем представлення, бізнес-логіки та даних. Такий підхід гарантує слабе зв'язування компонентів та гнучкість для масштабування системи.

Для восьми сценаріїв використання розроблено діаграми послідовності, що детально описують порядок взаємодії між акторами, контролерами, сервісами та репозиторіями. Діаграми охоплюють основні та альтернативні сценарії виконання.

На основі діаграм послідовності побудовано діаграму класів для ключового сценарію бронювання столика, що відображає чотири шари системи: контролер, сервіси, репозиторії та доменні сутності. Спроектowana структура класів відповідає принципам об'єктно-орієнтованого проєктування та безпосередньо відображає архітектурні рішення, прийняті на початку розділу.

Структуру бази даних спроектовано відповідно до третьої нормальної форми та представлено у вигляді дев'яти взаємопов'язаних таблиць. Використання зовнішніх ключів забезпечує цілісність даних і готовність до масштабування.

4 ПРОГРАМНА РЕАЛІЗАЦІЯ СИСТЕМИ

4.1 Вибір інструментів розробки

Вибір інструментів розробки є важливим етапом реалізації програмної системи, оскільки правильно підібраний технологічний стек безпосередньо впливає на швидкість розробки, якість продукту та можливість його супроводу.

Для реалізації серверної частини системи було обрано фреймворк Spring Boot, який є одним із найпопулярніших рішень для створення корпоративних Java-застосунків [15]. Даний вибір обумовлений необхідністю швидкої розробки надійного REST API з підтримкою складної бізнес-логіки. Цей фреймворк забезпечує автоматичну конфігурацію компонентів, вбудований сервер додатків та широкий набір стартерів для швидкої інтеграції різних технологій.

Для забезпечення безпеки системи застосовується Spring Security – фреймворк аутентифікації та авторизації, який надає можливість реалізації рольової моделі доступу та захисту REST API endpoints. Для аутентифікації користувачів використовується механізм JWT-токенів.

Для роботи з базою даних застосовується Spring Data JPA, що надає абстракцію над об'єктно-реляційним відображенням та генерує реалізацію репозиторіїв. Як реалізація JPA-специфікації використовується Hibernate ORM, який забезпечує ефективне управління життєвим циклом сутностей.

В якості СУБД було обрано PostgreSQL, що обґрунтовується необхідністю забезпечення ACID-властивостей для критичних операцій бронювання та підтримкою JSON-типів для зберігання гнучких структур даних.

Для реалізації клієнтської частини використовується React – бібліотека для побудови користувацьких інтерфейсів з компонентним підходом до розробки [16]. Вибір React обумовлений необхідністю створення динамічного, інтерактивного інтерфейсу з можливістю повторного використання компонентів. Для підвищення надійності коду застосовується TypeScript – типізована надбудова над JavaScript, яка дозволяє виявляти помилки на етапі компіляції.

В якості системи збірки обрано Apache Maven, який забезпечує управління залежностями. Для тестування системи використовуються JUnit 5 та Spring Boot Test, які надають інструменти для написання unit-тестів та інтеграційних тестів.

Обрані інструменти формують збалансований технологічний стек, який поєднує продуктивність, надійність та зручність розробки.

4.2 Ключові елементи програмної реалізації

Програмна реалізація системи TasteTales побудована на основі REST API архітектури з використанням Spring Boot фреймворку. Ключові функціональні можливості системи реалізовані через набір контролерів, які обробляють HTTP-запити та делегують бізнес-логіку відповідним сервісам.

Метод отримання доступних столиків (див. лістинг 4.1) реалізує функціонал пошуку вільних столиків на основі критеріїв користувача. Ендпоінт приймає чотири необов'язкові параметри запиту: дату, час, локацію та кількість гостей. Метод делегує обробку запиту сервісному рівню, який виконує перевірку доступності столиків у базі даних з урахуванням всіх переданих фільтрів, після чого повертає список доступних столиків.

Лістинг 4.1 – Метод для отримання доступних столиків

```
@GetMapping("/tables")
public ResponseEntity<TableListResponse> getAvailableTables(
    @RequestParam(required = false) String date,
    @RequestParam(required = false) String time,
    @RequestParam(required = false) String locationId,
    @RequestParam(required = false) Integer guests) {
    TableListResponse response = tableService.getAvailableTables(date, locationId,
        guests, time);
    return ResponseEntity.ok(response);
}
```

Метод створення бронювання клієнтом (див. лістинг 4.2) реалізує процес резервації столика. Доступ до ендпоінту обмежено лише для користувачів з роллю CUSTOMER через анотацію @PreAuthorize, що запобігає можливості офіціантів створювати бронювання від імені клієнтів. Тіло запиту валідується автоматично

через анотацію `@Valid`, яка перевіряє коректність всіх полів об'єкта, після чого метод витягує ідентифікатор користувача з JWT-токена та передає його разом із даними бронювання до сервісного рівня для створення резервації.

Лістинг 4.2 – Метод для створення бронювання клієнтом

```
@PreAuthorize("hasAuthority('CUSTOMER')")
@PostMapping("/client")
public ResponseEntity<ReservationResponse> bookTableByCustomer(@RequestBody @Valid
ReservationRequest request, @AuthenticationPrincipal Jwt principal) {
    ReservationResponse response = reservationService.bookTable(request,
getClaim(principal, "id"));
    return ResponseEntity.status(201).body(response);
}
```

Метод `findLeastBusyWaiter` (див. лістинг 4.3) реалізує алгоритм автоматичного призначення офіціанта з урахуванням його завантаженості. Спочатку з бази даних отримуються всі активні бронювання для конкретної локації та часового проміжку. Далі формується список офіціантів даної локації та завантажуються їхні облікові записи. Після цього для кожного офіціанта підраховується кількість бронювань, які на нього вже призначені. На основі цих даних система обирає офіціанта з найменшим навантаженням, використовуючи потокову обробку (Stream API).

Такий підхід забезпечує автоматичний баланс навантаження між працівниками та підвищує ефективність розподілу замовлень у системі.

Лістинг 4.3 – Метод автоматичного призначення офіціанта

```
private User findLeastBusyWaiter(Booking booking) {
    List<Booking> activeBooking =
reservationRepository.getAllActiveByLocationIdOnDateTime(booking.getLocation().getId(),
booking.getDate(), booking.getStartTime(), booking.getEndTime());
    List<String> waiterEmails =
waiterRepository.findAllByLocationId(booking.getLocation().getId())
        .stream().map(Waiter::getEmail).toList();
    List<User> waiterUserList = userRepository.findAllByEmailIn(waiterEmails);

    return waiterUserList.stream()
        .collect(Collectors.toMap(key -> key,
            value -> activeBooking.stream()
                .filter(b -> b.getWaiterId().equals(value.getId()))
                .count()))
        .entrySet().stream()
        .min(Map.Entry.comparingByValue())
        .map(Map.Entry::getKey)
}
```

```
.orElseThrow(() -> new EntityNotFoundException("No active waiters for  
location '%s' found".formatted(booking.getLocation().getId())));  
}
```

Представлені фрагменти коду демонструють ключові архітектурні рішення системи: використання Spring Security для захисту ендпоінтів, JWT-токенів для аутентифікації користувачів, автоматичної валідації вхідних даних та чіткого розділення відповідальності між контролерами та сервісним рівнем. Такий підхід забезпечує безпеку, надійність та легкість супроводу коду.

Висновки до розділу 4

У четвертому розділі було описано програмну реалізацію системи TasteTales, включаючи вибір технологічного стеку, розробку користувацького інтерфейсу та реалізацію ключових функціональних компонентів системи.

Обрано збалансований технологічний стек, що включає Spring Boot для серверної частини, React з TypeScript для клієнтської частини, PostgreSQL як СУБД та Hibernate ORM для роботи з базою даних. Використання Spring Security та JWT-токенів забезпечує надійний захист системи.

У процесі роботи було розроблено обидві складові програмної системи – як серверну, так і клієнтську частини. Їх успішна інтеграція між собою дозволила реалізувати повноцінну взаємодію вебінтерфейсу з бекендом та базою даних.

Пояснено ключові REST API ендпоінти для управління бронюваннями з підтримкою рольової моделі доступу. Методи контролерів демонструють правильне використання Spring Security для захисту ресурсів, автоматичної валідації вхідних даних та чіткого розділення відповідальності між рівнями архітектури. Диференційована логіка обробки запитів для різних типів користувачів забезпечує гнучкість системи та дотримання принципів безпеки.

Результати програмної реалізації підтверджують правильність архітектурних рішень та вибору технологій, що дозволило створити надійну, безпечну та зручну у використанні систему для автоматизації процесів ресторанного бізнесу

5 ТЕСТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ

5.1 Принципи вибору тестових прикладів

Тестування програмної системи TasteTales проводилося з метою перевірки відповідності реалізованого функціоналу встановленим вимогам та виявлення помилок. Для забезпечення повноти тестування було обрано комплексний підхід, що включає функціональне та нефункціональне тестування.

Принципи вибору тестових прикладів базувалися на наступних критеріях:

- для кожного варіанту використання, описаного у специфікації вимог, було розроблено відповідні тест-кейси;
- кожна функція перевіряється як з коректними вхідними даними, так і з некоректними значеннями параметрів для забезпечення надійності системи;
- особлива увага приділяється тестуванню складної бізнес-логіки системи з метою забезпечення коректності її роботи та відповідності вимогам;
- для різних виняткових випадків тестується коректність повернення помилки та відповідного HTTP-статусу;
- тестуються механізми пагінації для списків, фільтрації та сортування меню за різними критеріями, оновлення залежних даних (наприклад, середніх оцінок локацій, загальної вартості замовлення).

5.2 Функціональне тестування

Функціональне тестування охоплює перевірку всіх основних модулів програмної системи: автентифікацію, бронювання, попереднє замовлення, відгуки та перегляд контенту.

Кожен тест-кейс включає унікальний ідентифікатор, назву сценарію, конкретні вхідні дані, очікуваний результат та фактичний результат виконання.

5.2.1 Тестування модуля автентифікації. Модуль автентифікації є критичним для безпеки програмної системи. Тестування охоплює процеси реєстрації та авторизації з використанням різних комбінацій вхідних даних. Основна увага приділяється обробці помилок: спробам реєстрації з існуючим email, використанню некоректних паролів та захисту від несанкціонованого доступу.

У табл. 5.1 наведено результати тестування даного модуля. Усі 5 тест-кейсів виконано успішно, що підтверджує коректність процесів реєстрації та авторизації.

Таблиця 5.1 – Тест-кейси модуля автентифікації

ID	Сценарій	Вхідні дані	Очікуваний результат	Фактичний результат
ТС-01	Реєстрація нового користувача-клієнта	Email: test@example.com Пароль: Pass123! Ім'я: Іван Петренко	Обліковий запис створено Роль: CUSTOMER Статус: 201 Created	Пройдено
ТС-02	Реєстрація з існуючим email	Email: existing@example.com Ім'я: Іван Петренко	Помилка: «Email already in use» Статус: 409 Conflict	Пройдено
ТС-03	Реєстрація офіціанта	Email: waiter@restaurant.com (є в WaiterRepository) Ім'я: Іван Попов	Обліковий запис створено Роль: WAITER Статус: 201 Created	Пройдено
ТС-04	Авторизація з коректними даними	Email: test@example.com Пароль: Pass123!	JWT токен отримано Перенаправлення до системи	Пройдено
ТС-05	Авторизація з неправильним паролем	Email: test@example.com Пароль: WrongPass	Помилка: «Incorrect email or password» Статус: 401 Unauthorized	Пройдено

5.2.2 Тестування модуля бронювання. Модуль бронювання є складним компонентом, що включає перевірку столиків, фільтрацію за параметрами запиту, обробку часових слотів та створення бронювань з автоматичним призначенням офіціантів.

Тестування охоплює весь цикл бронювання: від перегляду столиків на вказану дату та час до створення бронювання.

У табл. 5.2 представлено результати тестування модуля бронювання. Усі тест-кейси виконано успішно. Система коректно обробляє складну логіку перевірки доступності столиків, фільтрації, валідації та призначення офіціантів.

Таблиця 5.2 – Тест-кейси модуля бронювання

ID	Сценарій	Вхідні дані	Очікуваний результат	Фактичний результат
ТС-06	Перегляд доступних столиків	Локація: Downtown Дата: 2026-06-01 Час: 19:00 Гості: 4	Список вільних столиків відображено Фільтрація столиків за місткістю ≥ 4	Пройдено
ТС-07	Створення бронювання	Локація: Downtown Дата: 2026-06-01 Час: 19:00-20:30 Гості: 4 Столик: #5	Бронювання створено Офіціант призначений Статус: 201 Created	Пройдено
ТС-08	Бронювання на минулу дату	Дата: 2026-04-01	Помилка валідації Статус: 400 Bad Request	Пройдено
ТС-09	Бронювання при відсутності столиків	Усі столики зайняті на обрану дату та локацію	Повідомлення: «No available tables» Пропозиція обрати інший час	Пройдено

5.2.3 Тестування модуля попереднього замовлення. Цей модуль забезпечує можливість клієнтам замовляти страви заздалегідь до відвідування. Тестування охоплює основні операції: додавання та видалення страв із кошика, зміну кількості, підтвердження замовлення клієнтом і його обробку офіціантом.

Особлива увага приділяється перевірці бізнес-правил: статусу страв (стоп-лист), відповідність спеціальних страв локації бронювання, часовим обмеженням редагування та розмежуванню прав доступу між клієнтами й офіціантами.

У табл. 5.3 наведено результати тестування функціоналу попереднього замовлення з боку клієнта та офіціанта. Усі тест-кейси виконано успішно. Система забезпечує коректне розмежування доступу, коректно обробляє додавання та видалення страв, валідує бізнес-правила щодо доступності страв та часових

обмежень, оновлює загальну вартість замовлення, належним чином оновлює статуси позицій та загальний статус всього попереднього замовлення.

Таблиця 5.3 – Тест-кейси модуля попереднього замовлення (клієнт)

ID	Сценарій	Вхідні дані	Очікуваний результат	Фактичний результат
1	2	3	4	5
ТС-10	Додавання страви до кошика	reservationId: #res123 dishId: #dish1	OrderItem створено Кількість: 1 Статус замовлення: NOT_SUBMITTED Статус: 201 Created	Пройдено
ТС-11	Додавання страви зі стоп-листа	dishId: #dish33, де dishStatus = ON_STOP	Помилка: «Dish is on stop» Статус: 400 Bad Request	Пройдено
ТС-12	Додавання спеціальної страви з іншої локації	dishId: #dish44, де specialty = true з локації В Бронювання в локації А	Помилка: «Specialty dish from another location» Статус: 400 Bad Request	Пройдено
ТС-13	Видалення страви з кошика	reservationId: #res123 dishId: #dish1	OrderItem видалено Загальна сума оновлена	Пройдено
ТС-14	Додавання вже доданої страви	dishId: #dish1 (вже є в замовленні)	Помилка: «This dish is already in your cart» Статус: 409 Conflict	Пройдено
ТС-15	Спроба додати страву менше ніж за 30 хв до бронювання	reservationId: #res123 dishId: #dish2 Час до бронювання: 20 хв	Помилка: «Can't modify a reservation's order less than 30 min before the start» Статус: 400 Bad Request	Пройдено
ТС-16	Підтвердження попереднього замовлення	reservationId: #res123, в якому додано 3 страви	Статус замовлення: SUBMITTED Повідомлення офіціанту надіслано	Пройдено
ТС-17	Перегляд деталей попереднього замовлення	reservationId: #res123	Список страв з кількістю порцій	Пройдено

Кінець таблиці 5.3

1	2	3	4	5
ТС-18	Підтвердження страви в замовленні	orderId: #order123 dishId: #dish1 status: CONFIRMED	OrderItem статус: CONFIRMED Статус: 200 OK	Пройдено
ТС-19	Відхилення страви з вказанням причини	orderId: #order123 dishId: #dish2 status: REJECTED comment: «Out of ingredients»	OrderItem статус: REJECTED Причина збережена Статус: 200 OK	Пройдено
ТС-20	Обробка всіх позицій замовлення	Всього три страви: дві підтверджено, одна відхилена	Загальний статус: PARTIALLY_CONFIRMED Email клієнту з деталями надіслано	Пройдено
ТС-21	Підтвердження всіх страв	Усі страви підтверджено	Загальний статус: CONFIRMED Email клієнту з деталями надіслано	Пройдено

5.2.4 Тестування модуля відгуків. Модуль відгуків дозволяє клієнтам залишати оцінки та коментарі після відвідування ресторану. Тестування включає перевірку можливості публікації двох типів відгуків (про сервіс та кухню), перевірки умов можливості залишення відгуку (бронювання має бути завершеним, для відгуку про кухню необхідна наявність замовлення). Також перевіряється коректність збереження оцінок та коментарів та оновлення середніх оцінок локацій.

У табл. 5.4 наведено результати тестування модуля відгуків. Усі тест-кейси виконано успішно. Система коректно розрізняє два типи відгуків, валідує можливість їх публікації залежно від стану бронювання та наявності замовлення та належним чином оновлює середні оцінки локацій.

Таблиця 5.4 – Тест-кейси модуля відгуків

ID	Сценарій	Вхідні дані	Очікуваний результат	Фактичний результат
ТС-22	Перегляд історії бронювань для залишення відгуку	userId: #user123	Список бронювань Кнопки для залишення відгуку (для завершених бронювань)	Пройдено
ТС-23	Залишення відгуку про сервіс	reservationId: #res123 (завершене) type: SERVICE rating: 5 comment: «Excellent service»	Відгук збережено Тип: SERVICE_QUALITY Середня оцінка локації оновлена Статус: 201 Created	Пройдено
ТС-24	Залишення відгуку про кухню	reservationId: #res123 (завершене та Order існує) type: CUISINE rating: 4 comment: «Delicious food»	Відгук збережено Тип: CUISINE_EXPERIENCE Середня оцінка локації оновлена Статус: 201 Created	Пройдено
ТС-25	Залишення обох типів відгуків	SERVICE + CUISINE	Обидва відгуки збережено Оцінки оновлено	Пройдено
ТС-26	Відгук про кухню без замовлення	ReservationId: #res456 (Order відсутній) type: CUISINE	Помилка: «No pre-order was found» Статус: 400 Bad Request	Пройдено
ТС-27	Спроба залишити відгук про незавершене бронювання	reservationId: #res789 (не завершене)	Помилка: «Reservation is not completed» Статус: 400 Bad Request	Пройдено

5.2.5 Тестування модуля перегляду контенту. Модуль перегляду контенту забезпечує доступ до інформації про меню, страви та локації для всіх користувачів, включаючи неавторизованих гостей. Тестування включає перевірку відображення списків з підтримкою пагінації, фільтрації та сортування, коректність отримання детальної інформації про окремі елементи, обробку запитів неіснуючих ресурсів (табл. 5.5).

Таблиця 5.5 – Тест-кейси модуля перегляду контенту

ID	Сценарій	Вхідні дані	Очікуваний результат	Фактичний результат
ТС-28	Перегляд меню без фільтрів	page: 0 size: 8	Список страв (8) Статус: 200 OK	Пройдено
ТС-29	Фільтрація меню за типом страви	dishType: DESSERT page: 0, size: 8	Лише десерти Кількість елементів <= 8 Статус: 200 OK	Пройдено
ТС-30	Сортування страв за ціною	sort: price_asc page: 0, size: 8	Страви відсортовані за зростанням ціни (8) Статус: 200 OK	Пройдено
ТС-31	Пагінація меню (друга сторінка)	page: 1 size: 8	Список страв (9 – 16) Статус: 200 OK	Пройдено
ТС-32	Перегляд деталей страви	dishId: #dish1	Повна інформація dish1 Статус: 200 OK	Пройдено
ТС-33	Перегляд неіснуючої страви	dishId: #non_existent	Статус: 404 Not Found	Пройдено
ТС-34	Перегляд списку всіх локацій	-	Список локацій з адресами, фото Статус: 200 OK	Пройдено
ТС-35	Перегляд деталей локації	locationId: #loc1	Повна інформація: назва, адреса, опис, місткість, фото Статус: 200 OK	Пройдено
ТС-36	Перегляд спеціальних страв локації	locationId: #loc1 limit: 5	До 5 спеціальних страв даної локації specialty=true Статус: 200 OK	Пройдено
ТС-37	Перегляд відгуків локації з фільтром	locationId: #loc1 Type: SERVICE page: 0, size: 4	4 відгуків про сервіс Відгуки лише про сервіс Статус: 200 OK	Пройдено

У табл. 5.5 представлено результати тестування модуля перегляду контенту. Усі тест-кейси виконано успішно. Система забезпечує швидке та коректне

відображення інформації про меню, страви та локації, належним чином підтримує пагінацію, фільтрацію та сортування.

Отже, усі 37 розроблених тест-кейсів виконано успішно, що свідчить про високу якість реалізації функціональних вимог. Покриття основних сценаріїв використання становить 100%, що забезпечує впевненість у коректності роботи системи. Покриття коду бізнес-логіки автоматизованими тестами становить 91%, що є високим показником для індивідуального проєкту та підтверджує ретельність підходу до тестування критичних компонентів програмної системи.

5.3 Нефункціональне тестування

Після завершення функціонального тестування проведено перевірку нефункціональних характеристик системи, що впливають на якість роботи програмної системи в умовах реальної експлуатації.

5.3.1 Тестування надійності. Тестування надійності перевіряє стабільність роботи системи при різних умовах експлуатації та коректність обробки помилкових ситуацій. Результати тестування надійності наведено у табл. 5.6.

Таблиця 5.6 – Результати тестування надійності

Параметр	Опис тестування	Нормативне значення	Фактичний результат	Статус
Успішність обробки запитів	Відсоток запитів, оброблених без помилок, протягом 5 днів тестування	$\geq 97\%$	98,5%	Пройдено
Обробка некоректних вхідних даних	Тестування з невалідними email, датами, порожніми обов'язковими полями	Коректні повідомлення про помилки без падіння системи	Усі помилки оброблені, повернуті повідомлення, статуси 400	Пройдено
Логування основних операцій	Перевірка наявності логів для основних запитів до сервера	Операції логуються з деталями	100% основних операцій логуються	Пройдено

У табл. 5.6 наведено результати тестування надійності системи. Усі параметри відповідають встановленим критеріям. Система демонструє високу доступність, правильно обробляє помилкові дані, зберігає стабільність та забезпечує високий рівень логування критичних операцій.

5.3.2 Тестування ефективності. Тестування ефективності проводилося з метою перевірки швидкодії системи при виконанні основних операцій та роботі множини користувачів одночасно. Результати тестування продуктивності наведено у табл. 5.7.

Таблиця 5.7 – Результати тестування ефективності

Параметр	Опис тестування	Нормативне значення	Фактичний результат	Статус
Час відгуку API для перевірки доступності столиків	GET /tables з параметрами фільтрації	≤ 3 с	0,72 с	Пройдено
Час відгуку API для створення бронювання	POST /client з валідацією, призначенням офіціанта, збереженням у БД	≤ 3 с	0,98 с	Пройдено
Підтримка одночасної роботи користувачів	Симуляція 50 одночасних користувачів, виконання різних операцій за 10 хв	Стабільна робота без зниження продуктивності	Середній час відгуку: 0,91 с, помилок: 0, деградації продуктивності: 0%	Пройдено

У табл. 5.7 наведено результати тестування ефективності системи. Усі параметри відповідають встановленим нормативним значенням. Система демонструє швидкий час відгуку API, ефективно обробляє одночасні запити від 50 користувачів без деградації продуктивності, забезпечує швидке завантаження сторінок.

5.3.3 Тестування безпеки. Тестування безпеки спрямоване на перевірку захисту даних користувачів, контролю доступу до функціоналу та стійкості до типових атак.

У табл. 5.8 представлено результати тестування безпеки. Усі перевірки виконано успішно. Система забезпечує надійний захист даних користувачів через безпечне зберігання паролів та ефективну рольову модель доступу. Система обмежує доступ до функцій відповідно до прав користувачів і запобігає несанкціонованим діям.

Таблиця 5.8 – Результати тестування безпеки

Параметр	Опис тестування	Очікуваний результат	Фактичний результат	Статус
Хешування паролів	Перевірка зберігання паролів у базі даних	Паролі захешовані з використанням BCrypt	Паролі зберігаються у хешованому вигляді, сіль генерується автоматично	Пройдено
Рольова модель доступу – клієнт	Спроба клієнта отримати доступ до функцій офіціанта (обробка замовлень)	Відмова доступу, статус 403 Forbidden	Доступ заборонено, помилка: «Access denied»	Пройдено
Рольова модель доступу – офіціант	Спроба офіціанта отримати доступ до чужих бронювань	Відмова доступу, статус 403 Forbidden	Система блокує доступ до бронювань інших офіціантів	Пройдено
Обмеження доступу для гостей	Спроба неавторизованого користувача створити бронювання	Відмова доступу, статус 401 Unauthorized	Доступ заборонено, система перенаправляє на сторінку входу	Пройдено
Доступ гостей до публічних функцій	Спроба доступу до локацій, меню, відгуків без авторизації	Доступ дозволено	Гості можуть переглядати локації, меню та відгуки	Пройдено

5.3.4 Тестування зручності використання. Тестування зручності інтерфейсу проводилося з метою оцінки швидкості навчання користувачів та ефективності виконання типових операцій. У тестуванні брали участь 10 респондентів (6 потенційних клієнтів, 4 офіціанти) без попереднього досвіду роботи з системою. Результати тестування зручності наведено у табл. 5.9.

Таблиця 5.9 – Результати тестування зручності використання

Параметр	Опис тестування	Нормативне значення	Фактичний результат	Статус
Час створення бронювання новим користувачем	Виміряно повний цикл від реєстрації до підтвердження бронювання	≤ 3 хв	Середній час: 2 хв 10 с; Мін: 1 хв 30 с; Макс: 2 хв 45 с	Пройдено
Час створення попереднього замовлення	Додавання 5 страв до кошика та підтвердження	≤ 3 хв	Середній час: 2 хв 05 с	Пройдено
Зрозумілість повідомлень про помилки	Оцінка респондентами якості повідомлень	Середня оцінка ≥ 4 (шкала 1–5)	Середня оцінка: 4,8 (зрозумілі та містять рекомендації)	Пройдено
Інтуїтивність інтерфейсу	Користувачі, які виконали операції без інструкцій	$\geq 75\%$	87,5% (7 з 8 респондентів)	Пройдено
Кількість натискань для переходу до бронювання	Після входу клієнт має почати бронювання не більше ніж за 2 кліки	≤ 2 кліки	1 клік (кнопка створення бронювання є в навігаційній панелі)	Пройдено

У табл. 5.9 представлено результати тестування зручності використання. Усі параметри відповідають встановленим критеріям. Користувачі швидко опановують роботу з системою, основні операції виконуються за мінімальний час, інтерфейс адаптується під різні типи пристроїв, повідомлення про помилки є зрозумілими та інформативними.

5.3.5 Тестування масштабованості. Було проведено тестування масштабованості системи для оцінки можливості розширення функціоналу та збільшення навантаження. Результати наведено у табл. 5.10.

Таблиця 5.10 – Результати тестування масштабованості

Параметр	Опис тестування	Нормативне значення	Фактичний результат	Статус
Незалежне масштабування компонентів	Тестування навантаження окремо на модуль бронювання та модуль замовлень	Можливість масштабування компонентів без впливу один на одного	Модулі працюють незалежно, не впливаючи один на одного	Виконано
Робота з ростом обсягу даних	Тестування БД при збільшенні записів	Час відповіді має бути ≤ 2 с	Час відповіді становить 0,99–1,5	Виконано

У табл. 5.10 наведено результати тестування масштабованості. Отримані результати свідчать про здатність системи ефективно працювати при зростанні навантаження та обсягу даних.

Висновки до розділу 5

В п'ятому розділі було проведене комплексне тестування програмної системи TasteTales, яке підтвердило високу якість реалізованого програмного засобу. Функціональне тестування охопило 37 тест-кейсів, що перевіряють всі основні варіанти використання системи, включаючи автентифікацію, бронювання, попереднє замовлення, відгуки та перегляд контенту. Усі тест-кейси виконано успішно (100% пройдено), а загальне покриття коду тестами склало 91%, що свідчить про коректність реалізації вимог та обробку помилкових ситуацій.

Нефункціональне тестування продемонструвало відповідність системи встановленим критеріям ефективності, надійності, безпеки, зручності використання та масштабованості.

Результати тестування підтверджують готовність програмної системи TasteTales до промислової експлуатації та її здатність забезпечувати надійне та безпечне обслуговування користувачів ресторанного бізнесу.

6 ЕКСПЛУАТАЦІЯ ПРОГРАМНОЇ СИСТЕМИ

6.1 Контрольний приклад застосування програмної системи

У цьому розділі наведено контрольний приклад роботи програмної системи TasteTales, що демонструє виконання основних функцій системи з боку різних типів користувачів. Приклад ілюструє послідовність дій при взаємодії з програмною системою через вебінтерфейс.

6.1.1 Робота з системою з боку гостя та клієнта. На рис. 6.1 та 6.2 зображено головну сторінку застосунку. Гість потрапляє на головну сторінку, де відображається інформація про мережу ресторанів TasteTales, фотографії задоволених гостей (Guest Moments), популярні страви (Most Popular Dishes) та доступні локації з їх характеристиками.

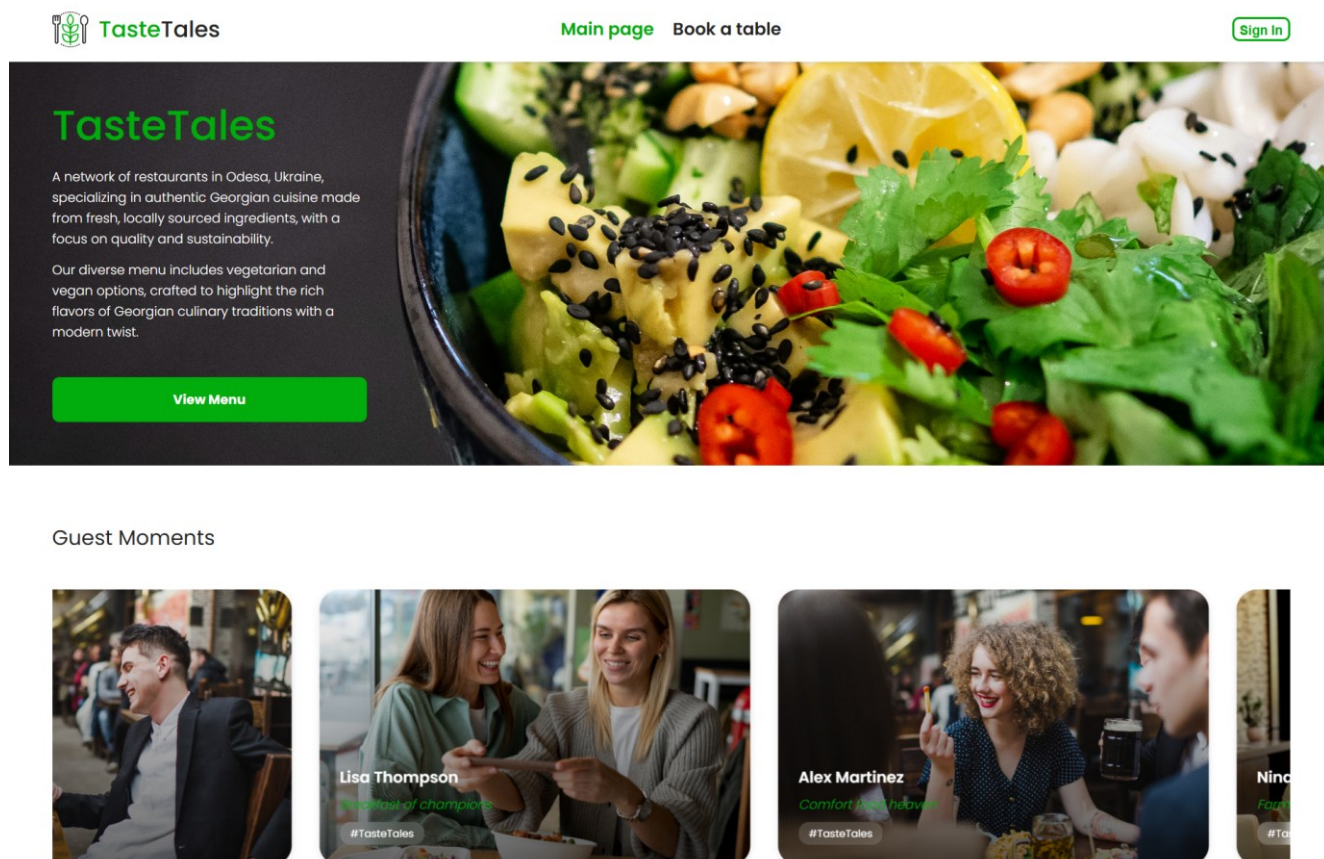
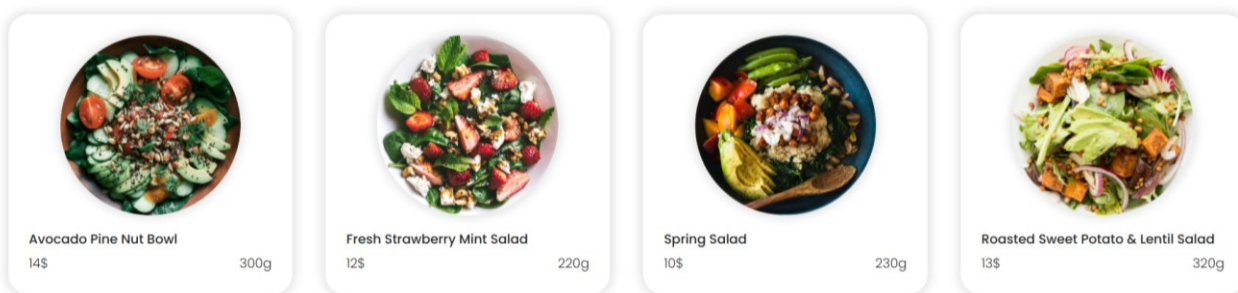


Рисунок 6.1 – Головна сторінка застосунку

Most Popular Dishes



Locations



Рисунок 6.2 – Продовження головної сторінки застосунку

На рис. 6.3 показано форму реєстрації з обробкою помилки валідації. Гість заповнює форму реєстрації, вводячи ім'я, прізвище, email та пароль, але система виявляє помилку – паролі не співпадають, та відображає відповідне повідомлення про помилку для виправлення. Після цього гість виправляє помилку та натискає кнопку «Create an account».

LET'S GET YOU STARTED

Create an account

First Name Last Name
e.g. Johnson e.g. Doe

Email
e.g. username@domain.com

Password
Password must be 8-16 latin characters with at least one uppercase, one lowercase, one number and one special character

Confirm Password
Confirm password must match new password

[Create an Account](#)

Already have an account? [Login](#) instead.

[Go back to main page](#)



Рисунок 6.3 – Форма реєстрації з обробкою помилки

На рис. 6.4 представлено форму авторизації користувача, яка відкривається після успішної реєстрації. Тут він вводить свій email та пароль для входу в систему.

WELCOME BACK
Sign in to your account

Email
gr.katrin.05@gmail.com
log: user@domain.com

Password

Enter your account password

Log in

Don't have an account? [Create an account](#) instead.
[Go back to main page](#)

TasteTales



Рисунок 6.4 – Форма авторизації

На рис. 6.5 показано сторінку деталей конкретної локації. Клієнт переглядає інформацію про локацію з її описом, спеціальними стравами (Speciality Dishes) та відгуками клієнтів (Customer reviews), які можна фільтрувати за типом (Service або Cuisine experience) та сортувати за оцінкою або датою.

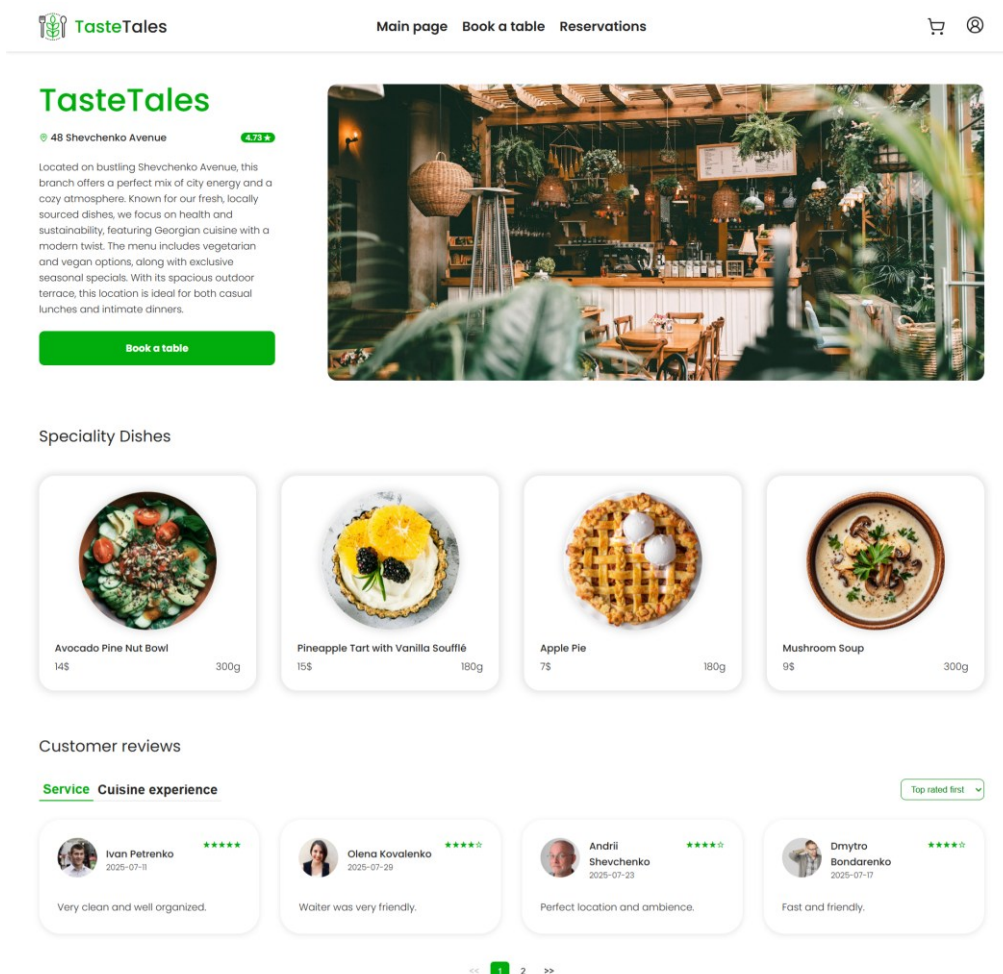


Рисунок 6.5 – Деталі локації зі спеціальними стравами та відгуками

На рис. 6.6 показано сторінку меню з усіма стравами та фільтрами. Клієнт переглядає повний список страв з фотографіями, назвами, цінами та вагою, з можливістю сортування за популярністю та пагінацією для перегляду всіх позицій.

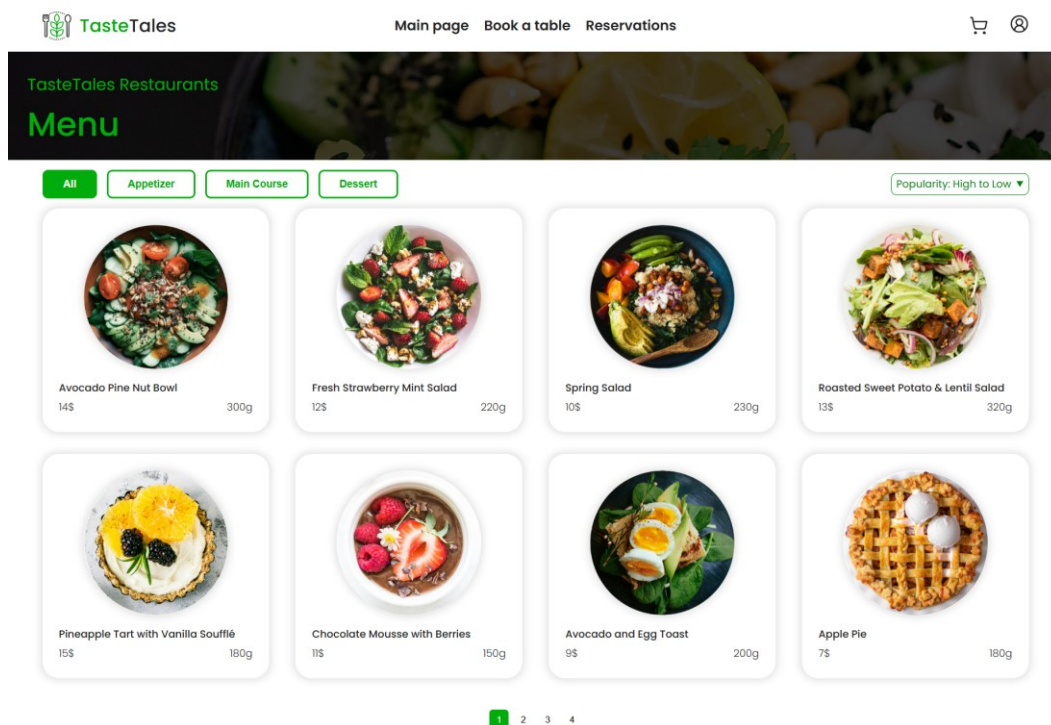


Рисунок 6.6 – Сторінка меню ресторану

На рис. 6.7 показано модальне вікно з детальною інформацією про страву. Клієнт відкриває картку страви та бачить повний опис, кбжв, вітаміни, ціну та вагу.

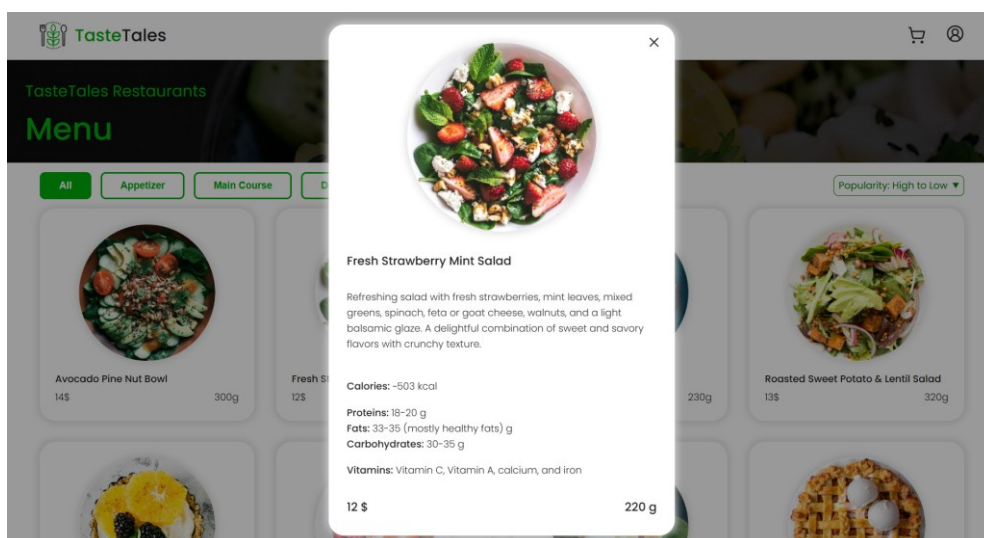


Рисунок 6.7 – Детальна інформація про страву

На рис. 6.8 зображено сторінку бронювання з результатами пошуку за датою та кількістю гостей. Клієнт обирає дату та кількість гостей, після чого система відображає доступні столики у різних локаціях з переліком вільних часових слотів. Також клієнт може застосувати додаткову фільтрацію за локацією та часом

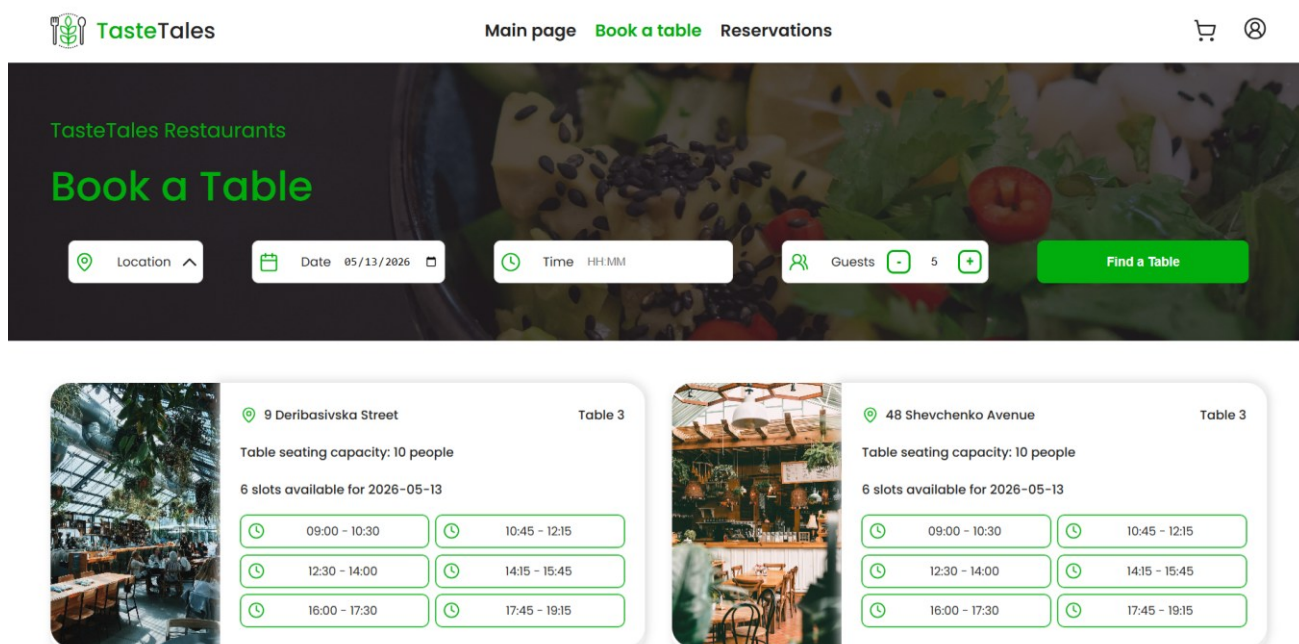


Рисунок 6.8 – Пошук доступних столиків за датою та кількістю гостей

На рис. 6.9 показано модальне вікно створення бронювання. Клієнт обирає конкретний столик і часовий слот, після чого система відображає форму для підтвердження з деталями бронювання – адресою, столиком, датою, кількістю гостей та обраним часом. Після цього він натискає кнопку підтвердження.

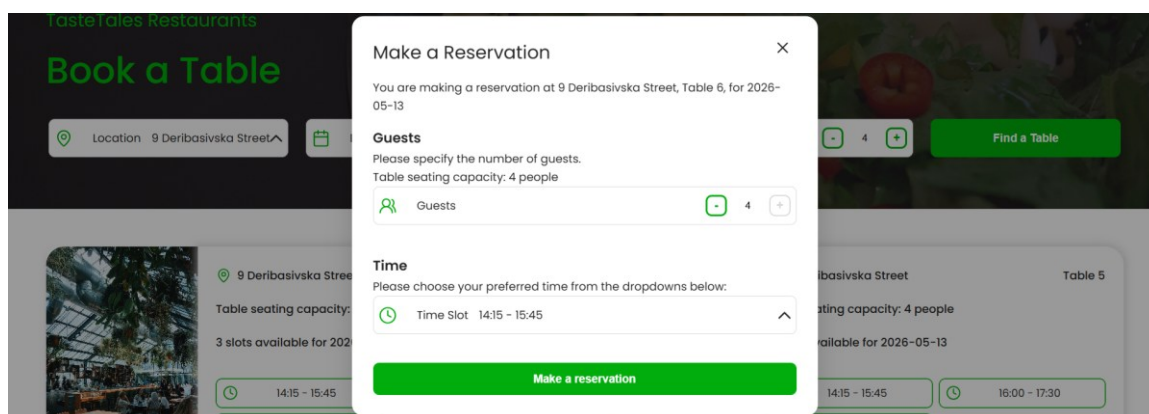


Рисунок 6.9 – Форма створення бронювання

На рис. 6.10 зображено сторінку «Reservations» з переліком усіх бронювань клієнта. Клієнт бачить як завершені, так і майбутні бронювання. Для активних бронювань доступні кнопки скасування, редагування та створення попереднього замовлення, а для завершених – кнопка залишення відгуку.

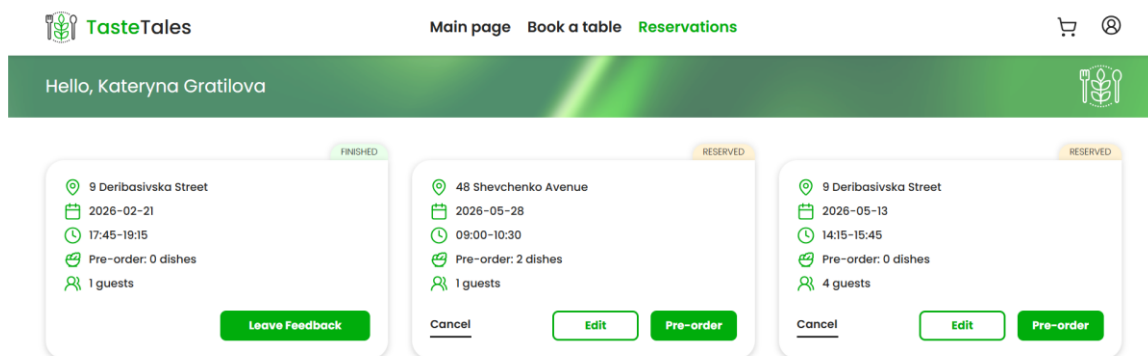


Рисунок 6.10 – Список бронювань клієнта

На рис. 6.11 зображено сторінку меню в режимі створення попереднього замовлення. Клієнт переходить до меню через кнопку «Pre-order» у бронюванні, після чого система відображає верхній банер з деталями бронювання, а доступні страви мають кнопку «Pre-order», тоді як недоступні позначені міткою «On Stop».

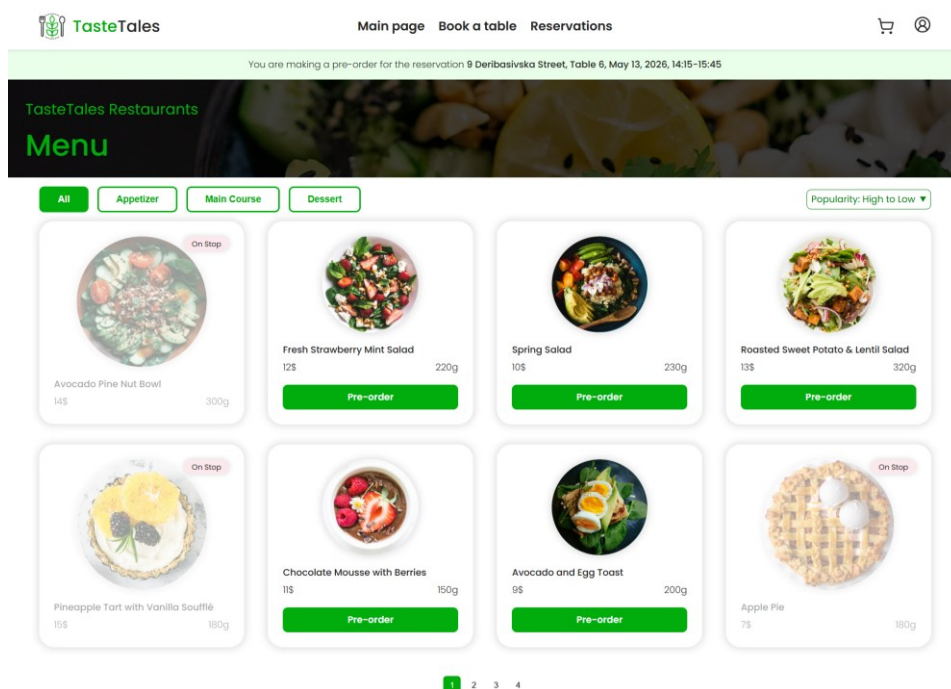


Рисунок 6.11 – Меню в режимі створення попереднього замовлення

На рис. 6.12 показано модальне вікно деталей страви з кнопкою попереднього замовлення. Клієнт відкриває детальну картку страви і може одразу додати її до попереднього замовлення, натиснувши кнопку «Pre-order».

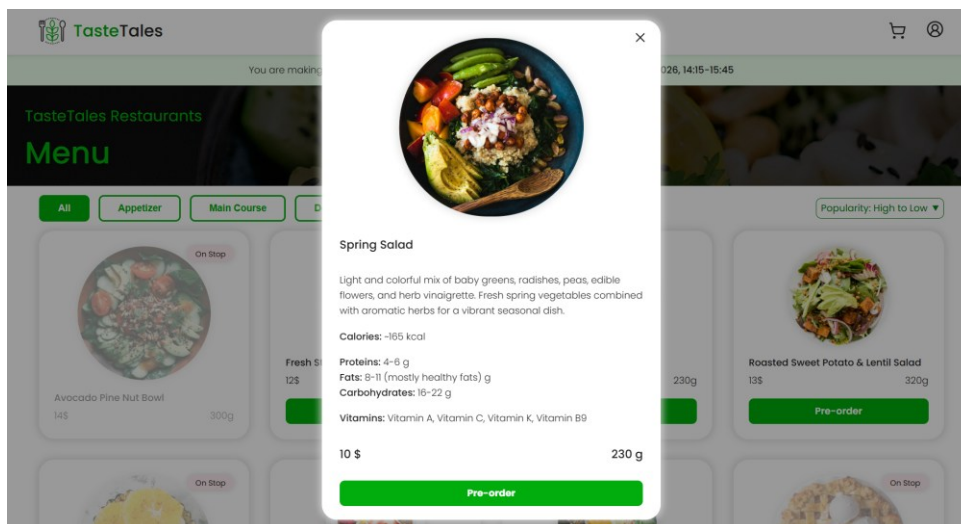


Рисунок 6.12 – Додавання страви до попереднього замовлення з картки страви

На рис. 6.13 показано кошик клієнта, який відображає поточне замовлення зі стравами, їх кількістю, вартістю та загальною сумою. Тут користувач може налаштувати кількість страв або видалити непотрібні з кошику. Для підтвердження попереднього замовлення клієнт натискає кнопку «Submit Pre-order».

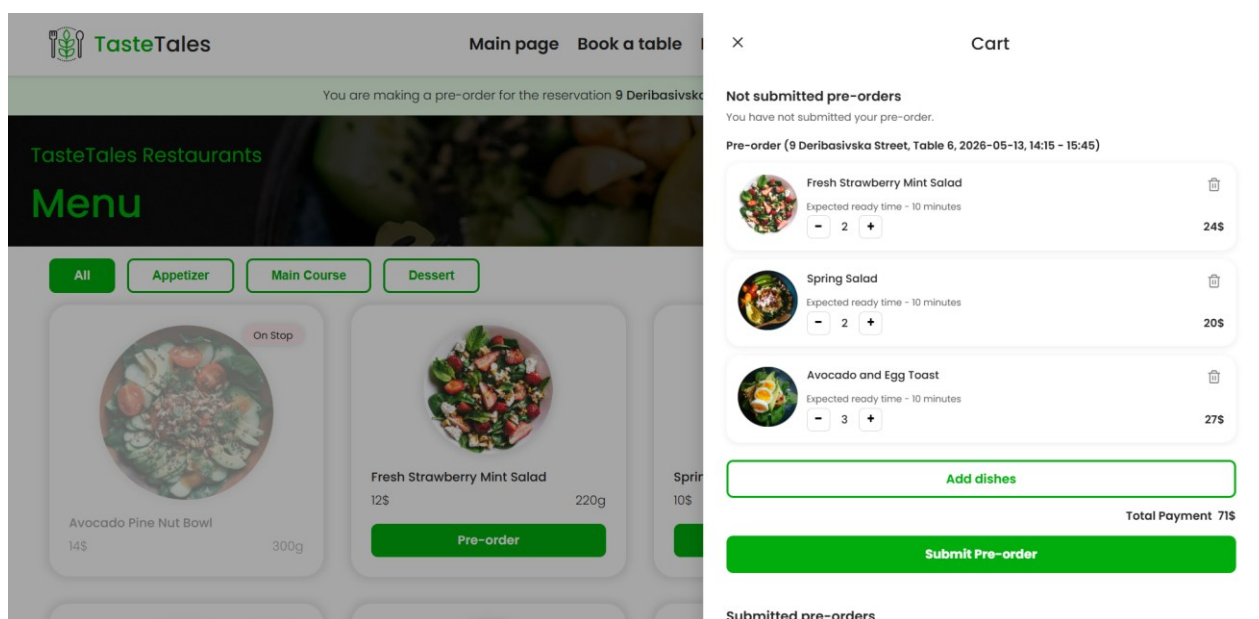


Рисунок 6.13 – Кошик з доданими стравами попереднього замовлення

На рис. 6.14 зображено кошик із двома підтвердженими попередніми замовленнями. Клієнт бачить обидва замовлення у секції «Submitted pre-orders» з переліком страв, їх кількістю, вартістю та кнопками редагування або скасування КОЖНОГО з НИХ.

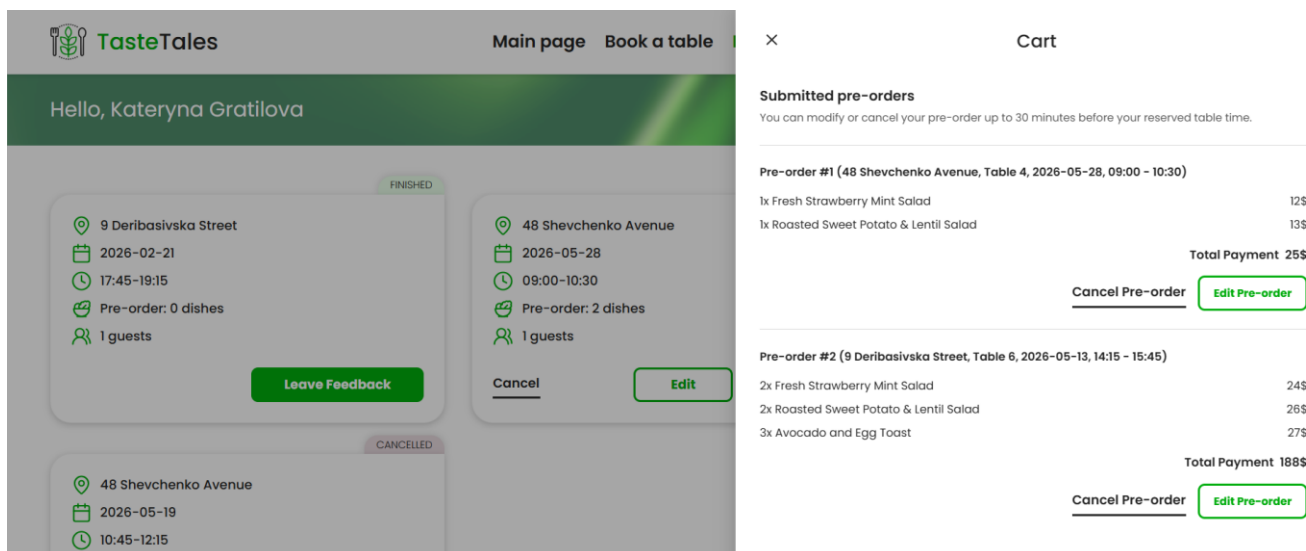


Рисунок 6.14 – Список підтверджених попередніх замовлень у кошику

На рис. 6.15 зображено модальне вікно для залишення відгуку. Через вкладки «Service» та «Culinary Experience» користувач може оцінити роботу офіціанта і якість страв за п'ятизірковою шкалою, додати текстові коментарі та надіслати заповнену форму.

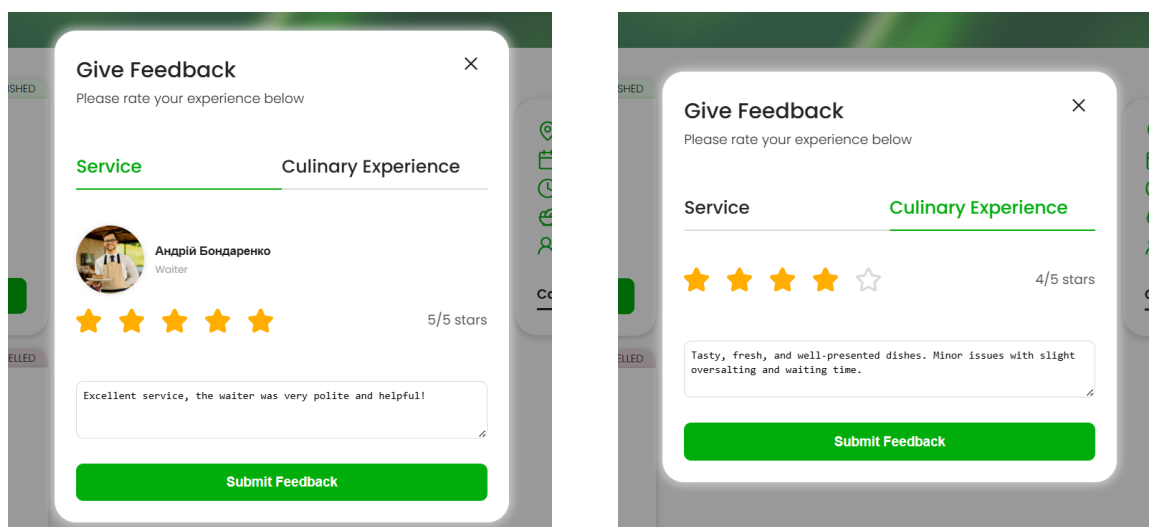


Рисунок 6.15 – Форма залишення відгуку

6.1.2 Робота з системою з боку офіціанта. На рис. 6.16 зображено сторінку, яка відкривається після авторизації офіціанта. Він бачить список із призначених йому бронювань, які містять інформацію про локацію, дату, час та кількість гостей і замовлених страв. Також для кожного майбутнього бронювання доступні кнопки «Edit» та «Create Pre-order». Також доступна фільтрація бронювань офіціанта за датою та часовим слотом.

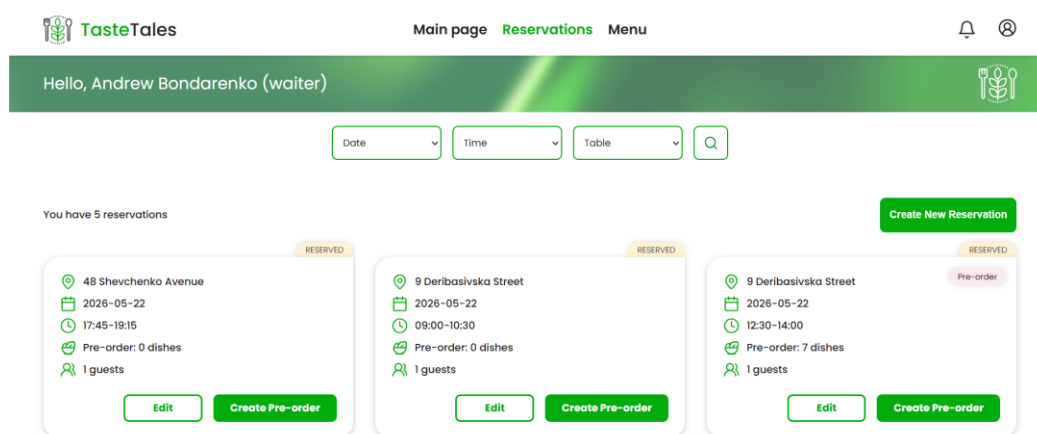


Рисунок 6.16 – Список призначених бронювань офіціанта

На рис. 6.17 показано модальне вікно «New Reservation» для створення нового бронювання офіціантом. Офіціант заповнює форму – обирає локацію, тип гостя, кількість гостей, дату, часовий проміжок та столик, після чого натискає «Create Reservation».

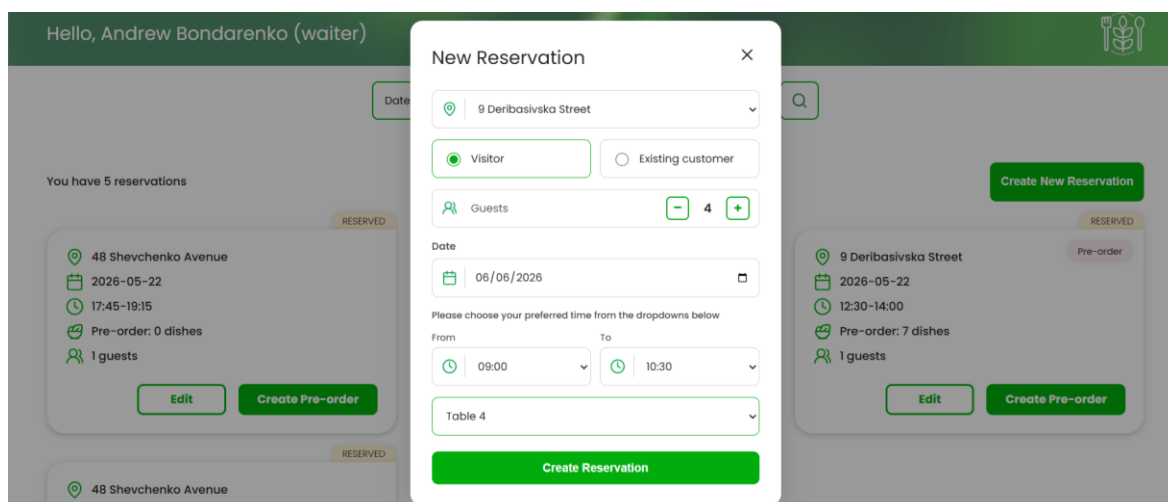


Рисунок 6.17 – Форма створення нового бронювання офіціантом

На рис. 6.18 зображено сторінку обробки попереднього замовлення клієнта. Офіціант бачить три замовлені страви із зазначенням їх кількості – і для кожної має кнопки «Cancel» та «Confirm».

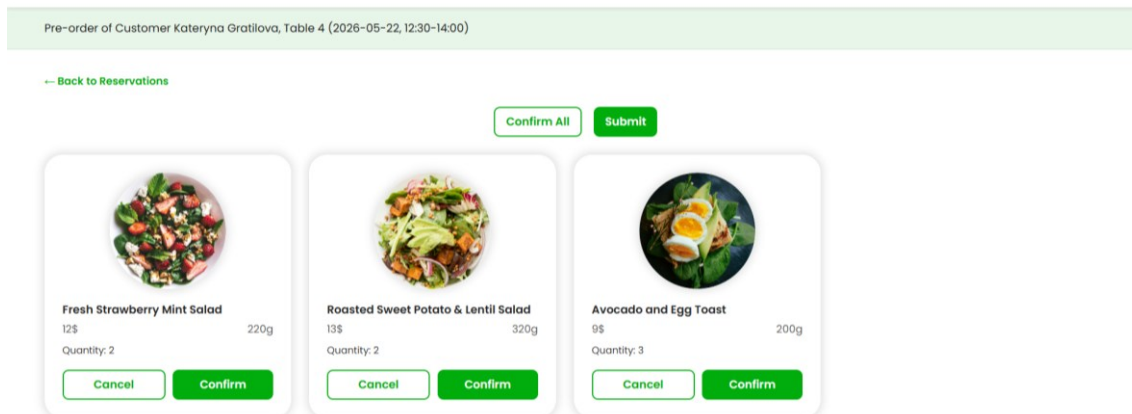


Рисунок 6.18 – Перегляд страв попереднього замовлення клієнта

На рис. 6.19 показано модальне вікно для відхилення страви. Офіціант вводить причину відхилення у поле та підтверджує дію кнопкою «Submit».

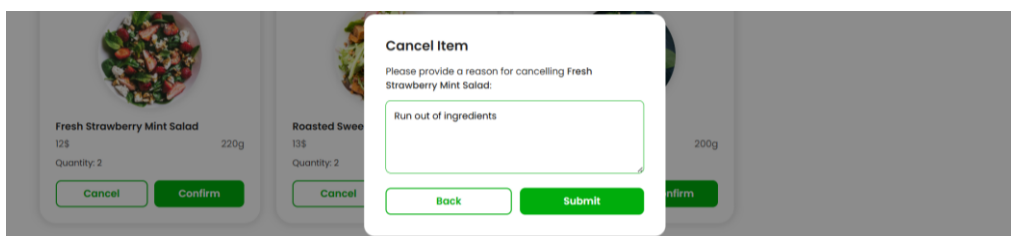


Рисунок 6.19 – Введення причини відхилення страви офіціантом

На рис. 6.20 зображено результат обробки позицій попереднього замовлення. Одна страву отримала статус DECLINED, а інші – статус CONFIRMED.

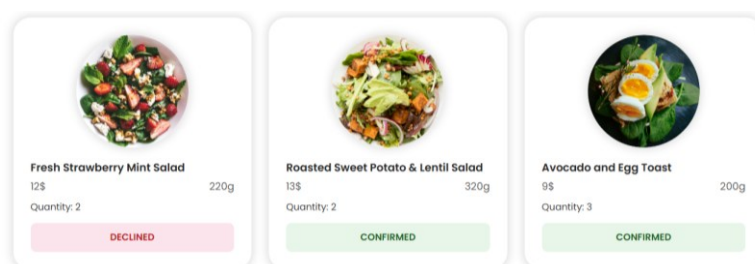


Рисунок 6.20 – Результат обробки попереднього замовлення офіціантом

Контрольний приклад демонструє повний цикл взаємодії з програмною системою з боку обох типів користувачів – клієнта та офіціанта. Система правильно обробляє всі ключові сценарії, забезпечуючи зручний та інтуїтивний інтерфейс.

6.2 Експлуатаційні випробування програмної системи

Метою експлуатаційних випробувань програмної системи TasteTales є підтвердження досягнення поставленої мети кваліфікаційної роботи – скорочення часу оформлення бронювання клієнтом до 3 хвилин порівняно з традиційним способом бронювання через телефонний дзвінок.

Випробування проводилися у два етапи. Перший – перевірка функціональних можливостей: тестування основних сценаріїв відповідно до специфікації вимог. Другий – вимірювання часових характеристик: фіксація часу виконання ключових операцій та їх порівняння з традиційним способом.

Для проведення випробувань було сформовано тестову групу з 10 респондентів: 6 потенційних клієнтів ресторану віком від 25 років до 45 років та 4 офіціанти віком від 22 років до 35 років – усі без попереднього досвіду роботи з системою. Кожен учасник отримував стандартизований сценарій дій без попередніх пояснень щодо роботи системи, що дозволило об'єктивно оцінити інтуїтивність інтерфейсу. Для порівняння зафіксовано час виконання аналогічних операцій традиційним способом на основі реальних взаємодій у ресторані.

У табл. 6.1 наведено порівняння часу виконання клієнтських операцій із використанням традиційного та розробленого способів. Вимірювання проводилися серед 6 клієнтів-респондентів. Традиційний спосіб включає час очікування відповіді оператора, усне перерахування варіантів та повторне уточнення деталей.

Отримані результати з табл. 6.1 пояснюються особливостями виконання операцій у традиційному способі. Перший контакт із клієнтом передбачає очікування відповіді оператора. Перегляд меню здійснюється через усну комунікацію, де клієнт змушений уточнювати деталі, тоді як у системі TasteTales доступна фільтрація та детальна інформація про страви.

Таблиця 6.1 – Порівняння часу виконання операцій клієнтами

Операція	Традиційний спосіб	TasteTales		
		мін. час	серед. час	макс. час
Реєстрація / перший контакт	3 хв 20 с	1 хв 15 с	1 хв 42 с	2 хв 08 с
Перегляд меню	5 хв 20 с	1 хв 20 с	1 хв 45 с	2 хв 28 с
Перегляд інформації про локації	3 хв 30 с	1 хв 05 с	1 хв 35 с	2 хв 12 с
Повний цикл бронювання	6 хв 30 с	1 хв 30 с	2 хв 10 с	2 хв 45 с
Попереднє замовлення 5 страв	5 хв 30 с	1 хв 45 с	2 хв 05 с	3 хв 15 с
Залишення відгуку	-	1 хв 05 с	1 хв 18 с	1 хв 35 с

Аналогічно, отримання інформації про локації в традиційному способі відбувається через пояснення оператора, тоді як у TasteTales вся інформація доступна одразу. Процес бронювання включає як етап очікування, так і усне узгодження деталей, що збільшує час. Попереднє замовлення страв додатково потребує уточнення можливості їх приготування, що також подовжує процес.

Результати порівняння часу виконання операцій офіціантами між традиційним та розробленим способом наведено у табл. 6.2.

Таблиця 6.2 – Порівняння часу виконання операцій офіціантами

Операція	Традиційний спосіб	TasteTales		
		мін. час	серед. час	макс. час
Перегляд списку бронювань	2 хв 00 с	12 с	18 с	24 с
Підтвердження однієї страви	-	4 с	7 с	12 с
Відхилення страви з причиною	-	15 с	22 с	28 с

З табл. 6.2 видно, що при традиційному способі роботи офіціанта витрати часу зростають через ручне виконання операцій. Перегляд списку бронювань потребує пошуку в паперовому журналі або уточнення в адміністратора. Підтвердження або відхилення страв у традиційному підході не потребується. У TasteTales всі операції автоматизовані: список бронювань доступний одразу в інтерфейсі, а перевірка страв виконується швидко із можливістю додати причину.

На рис. 6.21 наведено гістограму порівняння середнього часу виконання ключових операцій.



Рисунок 6.21 – Порівняння середнього часу виконання основних операцій

З рис. 6.21 видно, що найбільше скорочення часу досягнуто для операції бронювання столика – з 6 хв 30 с до 2 хв 10 с, тобто утричі.

Висновки до розділу 6

В шостому розділі було продемонстровано повний цикл взаємодії з системою з боку двох типів користувачів. З боку клієнта показано послідовність дій від реєстрації та авторизації до перегляду меню і локацій, бронювання столика, створення та підтвердження попереднього замовлення, скасування замовлення та залишення відгуку. З боку офіціанта продемонстровано перегляд і фільтрацію бронювань, створення нового бронювання, обробку попереднього замовлення клієнта з підтвердженням або відхиленням окремих страв із зазначенням причини.

Після цього були проведені експлуатаційні випробування за участю 10 респондентів. Результати підтвердили досягнення поставленої мети роботи: середній час оформлення бронювання становить 2 хв 10 с, що на 4 хв 20 с менше ніж при традиційному способі. Отримані результати свідчать про те, що розроблена програмна система TasteTales ефективно вирішує завдання автоматизації процесів ресторанного бізнесу.

ЗАГАЛЬНІ ВИСНОВКИ

У результаті виконання кваліфікаційної роботи розроблено програмну систему для автоматизації процесів ресторанного бізнесу. Створена програмна система забезпечує зручне бронювання столиків, управління попередніми замовленнями, координацію роботи персоналу та організацію зворотного зв'язку.

Аналіз предметної області підтвердив неефективність традиційних методів роботи ресторанів. Водночас порівняльний аналіз наявних систем бронювання засвідчив, що жодна з них не пропонує повного набору функціональних можливостей для комплексного вирішення виявлених проблем.

На основі проведеного аналізу сформульовано вимоги до системи: функціональні, що описують логіку взаємодії користувачів із системою, і нефункціональні, які охоплюють ключові показники якості програмної системи.

Виконано комплексне планування розробки системи, що включає об'єктивну оцінку трудомісткості проєкту та складання детального календарного графіка. Додатково проаналізовано потенційні проєктні ризики та сформовано стратегії реагування для мінімізації їхнього впливу на успішність створення продукту.

Для програмної системи обрано трирівневу архітектуру, що забезпечує чітке розділення відповідальності між рівнями представлення, бізнес-логіки та даних. Розроблено діаграми послідовності для сценаріїв використання, спроектовано структуру бази даних та побудовано діаграму класів. Програмну реалізацію виконано з використанням сучасних технологій: Spring Boot для серверної частини, React з TypeScript для клієнтської частини, PostgreSQL як СУБД.

Комплексне тестування підтвердило високу якість розробленого програмного засобу. Функціональні тести успішно охопили всі основні сценарії використання системи, а нефункціональні – продемонстрували її відповідність встановленим критеріям якості.

Експлуатаційні випробування за участю десяти респондентів підтвердили досягнення головної мети роботи: середній час оформлення бронювання клієнтом скорочено до трьох хвилин, що втричі менше порівняно з традиційними методами.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Гратілова К. О. Програмна система підтримки бізнес-процесів ресторану. Сучасні інформаційні технології – 2026 (Modern Information Technology – 2026) : матер. Шістнадцятої Міжнар. наук. конф. студ. та молодих вчених, 14–15 травня 2026 р., Одеса. С. 232–234.
2. How Digitalization Improves Efficiency and Profitability in Restaurants. URL: <https://sundayapp.com/how-digitalization-improves-efficiency-and-profitability-in-restaurants/> (дата звернення: 04.02.2026).
3. The Importance of Nutritional Information on Menus. URL: <https://www.ckitchen.com/blog/2025/3/the-importance-of-nutritional.html> (дата звернення: 04.02.2026).
4. OpenTable – online restaurant-reservation service. URL: <https://www.opentable.de> (дата звернення: 05.02.2026).
5. Resy – online restaurant-reservation service. URL: <https://resy.com> (дата звернення: 05.02.2026).
6. TableAgent – restaurant reservation system . URL: <https://tableagent.com> (дата звернення: 05.02.2026).
7. Cockburn A. Writing Effective Use Cases. Upper Saddle River : Addison-Wesley, 2001. 204 p.
8. Non-functional Requirements and its Importance. URL: <https://techcavass.com/blogs/non-functional-requirements-in-software-engineering> (дата звернення: 06.02.2026).
9. Pattiradjawane V. E., Wattimena E. M. C., Waas D. V. Improving Accuracy of Software Development Effort Estimation Using Use Case Points and Fuzzy Logic. Journal of Embedded System Security and Intelligent Systems. 2025. Vol. 6, No. 3. P. 363–378.
10. Wysocki R. K. Effective Project Management: Traditional, Agile, Extreme, Hybrid. 8th ed. Indianapolis : Wiley, 2019. 656 p.

11. Fishbone Diagram. URL: <https://purplegriffon.com/blog/fishbone-diagram-ishikawa> (дата звернення: 07.02.2026).
12. Bass L., Clements P. C., Kazman R. Software Architecture in Practice. 4th ed. Boston : Addison-Wesley Professional, 2021. 464 p.
13. Valacich J., George J. Modern Systems Analysis and Design. 9th ed. London : Pearson, 2020. 528 p.
14. Obe R., Hsu. L. PostgreSQL: Up and Running. 3rd ed. Sebastopol : O'Reilly Media, 2017. 314 p.
15. Heckler M. Spring Boot: Up and Running. Sebastopol : O'Reilly Media, 2021. 325 p.
16. Rippon C. React and TypeScript: Build high-quality React web apps using the power of TypeScript. 2nd ed. Birmingham : Packt Publishing, 2023. 474 p.

ДОДАТОК А

Лістинг програми

```

package com.tastetales.controller;

import static com.tastetales.utils.JwtUtil.getClaim;

import com.tastetales.dto.TableListResponse;
import com.tastetales.dto.request.ReservationRequest;
import com.tastetales.dto.request.UpdateReservationRequestDto;
import com.tastetales.dto.request.WaiterReservationRequest;
import com.tastetales.dto.response.*;
import com.tastetales.entity.User;
import com.tastetales.service.FeedbackService;

import com.tastetales.service.TableService;

import com.tastetales.service.LocationService;
import com.tastetales.service.OrderService;
import com.tastetales.service.ReservationService;
import com.tastetales.utils.JwtUtil;
import com.tastetales.validation.Time;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/reservations")
@RequiredArgsConstructor
@Slf4j
@Validated
public class ReservationController {
    private final ReservationService reservationService;
    private final LocationService locationService;
    private final TableService tableService;
    private final FeedbackService feedbackService;
    private final OrderService orderService;

    @GetMapping("/tables")
    public ResponseEntity<TableListResponse> getAvailableTables(
        @RequestParam(required = false) String date,
        @RequestParam(required = false) String time,
        @RequestParam(required = false) String locationId,
        @RequestParam(required = false) Integer guests) {

        TableListResponse response = tableService.getAvailableTables(date,
locationId, guests, time);
        return ResponseEntity.ok(response);
    }

    @PreAuthorize("hasAnyAuthority('CUSTOMER', 'WAITER')")
    @GetMapping
    public ResponseEntity<?> getReservations(

```

```

    @AuthenticationPrincipal Jwt principal,
    @RequestParam(required = false) String date,
    @RequestParam(required = false) @Time String fromTime,
    @RequestParam(required = false) Integer tableNumber) {

    String userId = principal.getClaimAsString("id");
    String role = principal.getClaimAsString("scope");

    if ("WAITER".equals(role)) {
        log.info("Fetching reservations for waiterId={}", userId);
        List<WaiterReservationResponse> reservations =
reservationService.getWaiterReservations(userId, date, fromTime, tableNumber);
        return ResponseEntity.ok(reservations);
    } else {
        log.info("Fetching reservations for customerId={}", userId);
        List<CustomerReservationResponse> reservations =
reservationService.getCustomerReservations(userId);
        return ResponseEntity.ok(reservations);
    }
}

    @PreAuthorize("hasAuthority('CUSTOMER')")
    @PostMapping("/client")
    public ResponseEntity<ReservationResponse> bookTableByCustomer(@RequestBody
@Valid ReservationRequest request,

    @AuthenticationPrincipal Jwt principal) {
        ReservationResponse response = reservationService.bookTable(request,
getClaim(principal, "id"));
        return ResponseEntity.status(201).body(response);
    }

    @PreAuthorize("hasAuthority('WAITER')")
    @PostMapping("/waiter")
    public ResponseEntity<ReservationResponse> bookTableByWaiter(@RequestBody
@Valid WaiterReservationRequest request,

    @AuthenticationPrincipal Jwt principal) {
        ReservationResponse response =
reservationService.bookTableByWaiter(request, getClaim(principal, "id"));
        return ResponseEntity.status(201).body(response);
    }

    @GetMapping("/locations")
    public ResponseEntity<List<LocationAddressResponse>>
getLocationSelectOptions() {
        List<LocationAddressResponse> addresses =
locationService.findAllLocationAddresses();
        return ResponseEntity.ok(addresses);
    }

    @PreAuthorize("hasAuthority('CUSTOMER')")
    @GetMapping("/{id}/feedbacks")
    public ResponseEntity<ReservationFeedbackResponse>
getReservationFeedbacks(@PathVariable String id,

    @AuthenticationPrincipal Jwt principal) {
        ReservationFeedbackResponse feedbacks =
feedbackService.findByReservationId(id, principal.getClaim("id"));
        return ResponseEntity.ok(feedbacks);
    }
}

```

```

@PreAuthorize("hasAuthority('WAITER')")
@GetMapping("/{id}/receipt")
public ReceiptResponse getReceipt(@PathVariable String id,
                                  @AuthenticationPrincipal Jwt principal) {
    final var user = JwtUtil.getUserFromJwt(principal);
    return reservationService.getReceipt(id, user);
}

@PreAuthorize("hasAnyAuthority({'CUSTOMER', 'WAITER'})")
@DeleteMapping("/{id}")
public ResponseEntity<MessageResponse> cancelReservation(@PathVariable String
id,

@AuthenticationPrincipal Jwt principal) {
    User user = JwtUtil.getUserFromJwt(principal);
    reservationService.cancelReservation(id, user);
    return ResponseEntity.ok().body(new MessageResponse("Reservation '%s'
cancelled".formatted(id)));
}

@PreAuthorize("hasAuthority('CUSTOMER')")
@PostMapping("/{reservationId}/preorder/{dishId}")
public ResponseEntity<MessageResponse> orderDish(@PathVariable String
reservationId,
                                                  @PathVariable String dishId,
                                                  @AuthenticationPrincipal Jwt
principal) {
    MessageResponse response =
orderService.addDishToReservationOrder(reservationId, dishId,
principal.getClaim("id"));
    return ResponseEntity.status(201).body(response);
}

@PreAuthorize("hasAnyAuthority('WAITER')")
@PatchMapping("/{id}/complete")
public ResponseEntity<MessageResponse> completeReservation(@PathVariable
String id,

@AuthenticationPrincipal Jwt principal) {
    var user = JwtUtil.getUserFromJwt(principal);
    return ResponseEntity.ok(reservationService.complete(id, user));
}

@PreAuthorize("hasAnyAuthority('CUSTOMER', 'WAITER')")
@PatchMapping("/{id}")
public ResponseEntity<MessageResponse> updateReservation(@PathVariable String
id,
                                                         @Valid @RequestBody
UpdateReservationRequestDto requestDto,

@AuthenticationPrincipal Jwt principal) {
    var user = JwtUtil.getUserFromJwt(principal);
    reservationService.updateReservation(id, requestDto, user);
    return ResponseEntity.ok(new MessageResponse("Reservation successfully
updated"));
}
}

package com.tastetales.controller;

import com.tastetales.dto.request.CreateOrderRequest;
import com.tastetales.dto.request.OrderItemChangeStatusReq;
import com.tastetales.dto.request.UpdateOrderRequest;

```



```

import com.tastetales.dto.response.MessageResponse;
import com.tastetales.dto.response.OrderItemResponse;
import com.tastetales.entity.User;
import com.tastetales.service.OrderItemService;
import com.tastetales.service.OrderService;
import com.tastetales.utils.JwtUtil;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/orders")
@RequiredArgsConstructor
public class OrderController {
    private final OrderService orderService;
    private final OrderItemService orderItemService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    @PreAuthorize("hasAuthority('WAITER')")
    public MessageResponse createOrder(@RequestBody @Valid CreateOrderRequest
createOrderRequest, @AuthenticationPrincipal
    Jwt jwt) {
        var user = JwtUtil.getUserFromJwt(jwt);
        return orderService.createOrder(createOrderRequest, user);
    }

    @GetMapping("/{orderId}/items")
    @PreAuthorize("hasAnyAuthority('WAITER', 'CUSTOMER')")
    public List<OrderItemResponse> getAllByOrderId(@PathVariable String orderId,
                                                    @AuthenticationPrincipal Jwt
jwt) {
        final var user = JwtUtil.getUserFromJwt(jwt);
        return orderItemService.getAllByOrderId(orderId, user);
    }

    @PutMapping("/{orderId}/item/change-status")
    @PreAuthorize("hasAnyAuthority('WAITER')")
    public void changeStatus(@PathVariable String orderId,
                             @RequestBody OrderItemChangeStatusReq
orderItemChangeStatusReq,
                             @AuthenticationPrincipal Jwt jwt) {
        final var user = JwtUtil.getUserFromJwt(jwt);
        orderItemService.changeStatus(user, orderId, orderItemChangeStatusReq);
    }

    @PreAuthorize("hasAuthority('CUSTOMER')")
    @DeleteMapping("/{orderId}/preorder/{dishId}")
    public ResponseEntity<MessageResponse> deleteDishFromPreorder(@PathVariable
String orderId,
                                                                    @PathVariable
String dishId,
                                                                    @AuthenticationPrincipal Jwt principal) {
        MessageResponse response =
orderService.deleteDishFromReservationOrder(orderId, dishId,

```

```

principal.getClaim("id"));
    return ResponseEntity.ok(response);
}

    @PreAuthorize("hasAuthority('CUSTOMER')")
    @DeleteMapping("/{orderId}")
    public ResponseEntity<MessageResponse> cancelPreorder(@PathVariable String
orderId,
                                                                    @AuthenticationPrincipal
Jwt principal) {
        MessageResponse response = orderService.deletePreorder(orderId,
principal.getClaim("id"));
        return ResponseEntity.ok(response);
    }
}

package com.tastetales.controller;

import com.tastetales.dto.request.UpdatePreorderRequest;
import com.tastetales.dto.response.CartResponse;
import com.tastetales.service.OrderService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/cart")
@RequiredArgsConstructor
@PreAuthorize("hasAuthority('CUSTOMER')")
public class CartController {

    private final OrderService orderService;

    @GetMapping
    public ResponseEntity<CartResponse> getCart(@AuthenticationPrincipal Jwt
principal) {
        return ResponseEntity.ok(orderService.getCart(principal.getClaim("id")));
    }

    @PutMapping("/preorder/submit")
    public ResponseEntity<CartResponse> submitPreorder(@Valid @RequestBody
UpdatePreorderRequest request,
                                                                    @AuthenticationPrincipal
Jwt principal) {
        return ResponseEntity.ok(orderService.submitPreorder(request,
principal.getClaim("id")));
    }

    @PreAuthorize("hasAuthority('CUSTOMER')")
    @PutMapping("/preorder/draft")
    public ResponseEntity<Void> updateDraftPreorder(@Valid @RequestBody
UpdatePreorderRequest request,
                                                                    @AuthenticationPrincipal Jwt
principal) {
        orderService.updateDraftPreorder(request, principal.getClaim("id"));
        return ResponseEntity.status(204).build();
    }
}

```

```

package com.tastetales.controller;

import com.tastetales.dto.request.FeedbackRequest;
import com.tastetales.service.FeedbackService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;
import com.tastetales.dto.response.MessageResponse;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/feedbacks")
@RequiredArgsConstructor
public class FeedbackController {

    private final FeedbackService feedbackService;

    @PostMapping
    @PreAuthorize("hasAuthority('CUSTOMER')")
    public ResponseEntity<MessageResponse> create(@RequestBody @Valid
FeedbackRequest request, @AuthenticationPrincipal Jwt principal) {
        var resultMessage = feedbackService.create(request,
principal.getClaim("id"));
        return ResponseEntity.status(201).body(resultMessage);
    }

    @PutMapping
    @PreAuthorize("hasAuthority('CUSTOMER')")
    public ResponseEntity<MessageResponse> update(@RequestBody @Valid
FeedbackRequest request, @AuthenticationPrincipal Jwt principal) {
        var resultMessage = feedbackService.update(request,
principal.getClaim("id"));
        return ResponseEntity.ok(resultMessage);
    }
}

package com.tastetales.controller;

import com.tastetales.dto.DishesResponseDto;
import com.tastetales.dto.filter.TableOnDateFilterDto;
import com.tastetales.dto.response.*;
import com.tastetales.entity.enums.FeedbackType;
import com.tastetales.exception.UnknownFeedbackType;
import com.tastetales.service.*;
import jakarta.validation.Valid;
import jakarta.validation.constraints.Positive;
import java.util.List;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/locations")
@Validated
@RequiredArgsConstructor

```

```

@Slf4j
public class LocationController {
    private final DishesService dishesService;
    private final LocationService locationService;
    private final FeedbackService feedbackService;
    private final TableService tableService;

    @GetMapping
    public ResponseEntity<List<LocationDetailsResponse>> getAllLocations() {
        List<LocationDetailsResponse> locations =
locationService.findAllLocations();
        return ResponseEntity.ok(locations);
    }

    @GetMapping("/{id}/feedbacks")
    public ResponseEntity<PageResponse<FeedbackResponse>> getFeedbacksByLocation(
        @PathVariable("id") String locationId,
        @RequestParam("type") String type,
        @RequestParam(value = "page", defaultValue = "0") int page,
        @RequestParam(value = "size", defaultValue = "20") int size,
        @RequestParam(value = "sort", defaultValue = "rate,asc") String sort
    ) throws UnknownFeedbackType {
        FeedbackType feedbackType = FeedbackType.fromValue(type);
        String[] sortParts = sort.split(",");
        String sortBy = sortParts.length > 0 ? sortParts[0] : "rate";
        boolean ascending = sortParts.length < 2 ||
!"desc".equalsIgnoreCase(sortParts[1]);
        page = validateIntOrDefault(page, 0);
        size = validateIntOrDefault(size, 20);

        PageResponse<FeedbackResponse> response =
feedbackService.findFeedbacksByLocationIdAndType(
            locationId, feedbackType, sortBy, ascending, page, size
        );
        log.info("Successfully retrieved feedback page: {} items",
response.getNumberOfElements());

        return ResponseEntity.ok(response);
    }

    @GetMapping("/{locationId}/specialty-dishes")
    public DishesListResponseDto getSpecialtyDishes(@PathVariable String
locationId,
                                                    @RequestParam(defaultValue =
"10") int limit) {
        List<DishesResponseDto> dishes =
dishesService.getSpecialtyDishes(locationId, limit);
        log.info(dishes.toString());
        return new DishesListResponseDto(dishes);
    }

    @GetMapping("/{id}/tables/{tableNumber}")
    public ResponseEntity<TableSlotsResponseDto> getAvailableSlotsForTableOnDate(
        @PathVariable String id,
        @PathVariable @Positive(message = "Table number must be positive") Integer
tableNumber,
        @Valid TableOnDateFilterDto filterDto) {
        return ResponseEntity.ok(tableService.getAvailableSlotsOnDate(id,
tableNumber, filterDto));
    }

    private int validateIntOrDefault(int value, int defaultValue) {
        return value >= 0 ? value : defaultValue;
    }
}

```

```

    }
}

package com.tastetales.controller;

import com.tastetales.dto.MenuItem;
import com.tastetales.dto.response.*;
import com.tastetales.entity.enums.*;
import com.tastetales.service.MenuService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequiredArgsConstructor
@RequestMapping("/main/menu")
@Slf4j
public class MenuController {

    private final MenuService menuService;

    @GetMapping
    public ResponseEntity<PageResponse<MenuItem>> getMenu(
        @RequestParam(required = false) DishType dishType,
        @RequestParam(required = false) SortField sort,
        @PageableDefault(size = 3) Pageable pageable
    ) {
        return ResponseEntity.ok(menuService.getMenu(dishType, sort, pageable));
    }

    @GetMapping("/{id}")
    public ResponseEntity<DishExtendedResponse> getDishById(@PathVariable String
id) {
        DishExtendedResponse response = menuService.getDishById(id);
        return ResponseEntity.ok(response);
    }
}

package com.tastetales.controller;

import com.tastetales.dto.*;
import com.tastetales.dto.request.PasswordChangeRequestDto;
import com.tastetales.dto.response.PasswordResetResponseDto;
import com.tastetales.service.AuthService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/auth")
@Validated
@RequiredArgsConstructor
public class AuthController {

    private final AuthService authService;

```

```

    @PostMapping("/sign-up")
    public SignUpResponse signUp(@Valid @RequestBody SignUpRequest signUpRequest)
    {
        return authService.signUp(signUpRequest);
    }

    @PostMapping("/sign-in")
    public SignInResponse signIn(@Valid @RequestBody SignInRequest signInRequest)
    {
        return authService.signIn(signInRequest);
    }

    @PreAuthorize("hasAnyAuthority('CUSTOMER', 'WAITER')")
    @PutMapping("/change-password")
    public PasswordResetResponseDto resetPassword(@AuthenticationPrincipal Jwt
principal,
                                                @Valid @RequestBody
PasswordChangeRequestDto passwordChangeRequestDto) {
        return authService.resetPassword(principal.getClaim("id"),
passwordChangeRequestDto);
    }
}

package com.tastetales.service.impl;

import static com.tastetales.constant.AppConstant.ANONYMOUS_USER_ID;
import static com.tastetales.entity.enums.BookingStatus.*;
import static com.tastetales.entity.enums.Role.CUSTOMER;

import com.tastetales.constant.AppConstant;
import com.tastetales.dto.UserContacts;
import com.tastetales.dto.request.*;
import com.tastetales.dto.response.*;
import com.tastetales.entity.*;
import com.tastetales.entity.enums.*;
import com.tastetales.exception.*;
import com.tastetales.mapper.*;
import com.tastetales.repository.*;
import com.tastetales.service.*;

import com.tastetales.service.EmailService;

import java.time.*;
import java.util.*;
import java.util.stream.Collectors;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@RequiredArgsConstructor
@Slf4j
public class ReservationServiceImpl implements ReservationService {
    private final ReservationRepository reservationRepository;
    private final TableRepository tableRepository;
    private final LocationRepository locationRepository;
    private final WaiterRepository waiterRepository;
    private final FeedbackRepository feedbackRepository;
    private final UserRepository userRepository;
    private final OrderRepository orderRepository;
    private final OrderItemRepository orderItemRepository;

```

```

private final ReservationMapper reservationMapper;
private final UserMapper userMapper;
private final EmailService emailService;

@Override
public Booking findReservationById(String id) {
    return reservationRepository.findById(id)
        .orElseThrow(() -> new EntityNotFoundException("Reservation with '%s'
id wasn't found!".formatted(id)));
}

@Override
public ReservationResponse bookTable(ReservationRequest request, String
userId) {
    log.info("in bookTable(): {}", request);

    Location location = getLocationOrThrow(request.locationId());
    Table table = getTableOrThrow(request.locationId(),
request.tableNumber());
    validateReservation(request, table, location);
    Booking booking = reservationMapper.toBooking(request,
UUID.randomUUID().toString(), userId, location);
    booking.setWaiterId(findLeastBusyWaiter(booking).getId());
    reservationRepository.save(booking);
    userRepository.findById(userId).ifPresent(user ->
emailService.sendBookingConfirmation(user, booking));
    return reservationMapper.toReservationResponse(booking);
}

@Override
public ReservationResponse bookTableByWaiter(WaiterReservationRequest request,
String waiterId) {
    log.info("Attempting to book a table by waiter '{}' with request: {}",
waiterId, request);

    User user = userRepository.findById(waiterId).orElseThrow(() -> new
EntityNotFoundException("User with id '%s' not found".formatted(waiterId)));
    Waiter waiter = waiterRepository.findByEmail(user.getEmail())
        .orElseThrow(() -> new EntityNotFoundException("Waiter with email '%s'
not found".formatted(user.getEmail())));

    String userInfo;
    String userId;
    switch (request.clientType()) {
        case VISITOR -> {
            if (request.isCustomerEmailExistsWithVisitorRole()) {
                throw new ReservationException("Customer email is not allowed
for VISITOR role");
            }
            userInfo = "VISITOR";
            userId = ANONYMOUS_USER_ID;
            log.debug("Client is a VISITOR.");
        }
        case CUSTOMER -> {
            if ((request.customerEmail() == null) ||
(request.customerEmail().isBlank())) {
                throw new ReservationException("Customer email is required for
CUSTOMER role");
            }
            User customer =
userRepository.findByEmail(request.customerEmail())
                .orElseThrow(() -> new EntityNotFoundException(
                    "Customer with email '%s' not

```

```

        found".formatted(request.customerEmail())));
        userInfo = "CUSTOMER " + customer.getFirstName() + " " +
customer.getLastName();
        userId = customer.getId();
        log.debug("Client is a CUSTOMER with id: {}", userId);
    }
    default -> throw new ReservationException("Invalid client type:
%s".formatted(request.clientType()));
    }

    Table table = getTableOrThrow(request.locationId(),
request.tableNumber());
    Location location = getLocationOrThrow(request.locationId());

    String waiterEmail = waiter.getEmail();
    String waiterLocation = waiter.getLocationId();

    if (!waiterLocation.equals(location.getId()))
        throw new AuthorizationException(
            "Waiter '%s' is not allowed to manage reservations at location
'%s'"
                .formatted(waiterEmail, location.getId()));
    validateReservation(request, table, location);

    Booking booking = reservationMapper.toBooking(request, userId, location);
    booking.setWaiterId(waiterId);
    reservationRepository.save(booking);
    log.info("Reservation with id '{}' was successfully created by waiter
'{}'", booking.getId(), waiterId);

    if (!ANONYMOUS_USER_ID.equals(userId)) {
        userRepository.findById(userId).ifPresent(u ->
emailService.sendBookingConfirmation(u, booking));
    }

    return reservationMapper.toReservationResponse(booking,
location.getAddress(), userInfo);
}

@Override
@Transactional
public void cancelReservation(String id, User user) {
    log.info("in cancelReservation(), reservationId: {}, userId: {}", id,
user.getId());

    Booking reservation = getReservationWithStatusOrThrow(id, RESERVED);
    checkAuthorization(user, reservation);
    reservation.setStatus(CANCELLED);
    cancelOrderIfPresent(reservation.getId());
    orderItemRepository.deleteAllByOrderId(reservation.getId());
    reservationRepository.save(reservation);
}

@Override
public ReceiptResponse getReceipt(String reservationId, User user) {
    checkPermission(reservationId, user);

    return reservationRepository.findById(reservationId)
        .map(reservationEntity -> {

            if (!reservationEntity.getStatus().equals(BookingStatus.FINISHED))
                throw new ReservationException("Reservation with id '%s' was
not finished".formatted(reservationId));
        });
}

```



```

        final UserContacts waiter =
userRepository.findById(reservationEntity.getWaiterId())
        .map(userMapper::toUserContacts)
        .orElseThrow(() -> new EntityNotFoundException("User '%s' not
found".formatted(reservationId)));

        final UserContacts customer =
userRepository.findById(reservationEntity.getUserId())
        .map(userMapper::toUserContacts)
        .orElseThrow(() -> new EntityNotFoundException("User '%s' not
found".formatted(reservationId)));

        final Order preOrder = orderRepository.findById(reservationId)
        .orElseThrow(() -> new EntityNotFoundException("Order '%s' not
found".formatted(reservationId)));

        return reservationMapper.toReceiptResponse(reservationEntity,
waiter, preOrder, customer);
    })
    .orElseThrow(() -> new EntityNotFoundException("Reservation with id
'%s' not found".formatted(reservationId)));
}

@Override
public List<WaiterReservationResponse> getWaiterReservations(String waiterId,
String date, String fromTime, Integer tableNumber) {
    log.info("in getWaiterReservations(), waiterId: {}", waiterId);

    User waiter =
        userRepository
            .findById(waiterId)
            .orElseThrow(
                () ->
                    new EntityNotFoundException(
                        "Waiter with id '%s' not found".formatted(waiterId)));
    log.debug("Found waiter: {} {}", waiter.getFirstName(),
waiter.getLastName());

    List<Booking> filteredReservations =
        reservationRepository.findByWaiterAndFilters(waiterId, date, fromTime,
tableNumber);
    log.debug("Found {} reservations for waiterId: {}",
filteredReservations.size(), waiterId);
    if (filteredReservations.isEmpty()) {
        return Collections.emptyList();
    }

    Map<String, User> userMap = fetchAndMapUsers(filteredReservations);
    Map<String, Order> ordersMap = fetchAndMapOrders(filteredReservations);

    String waiterName = String.format("Waiter %s %s", waiter.getFirstName(),
waiter.getLastName());

    List<WaiterReservationResponse> responses =
        filteredReservations.stream()
            .map(
                reservation ->
                    buildWaiterReservationResponse(reservation,
reservation.getLocation().getAddress(), userMap, ordersMap, waiterName))
            .filter(Objects::nonNull)
            .collect(Collectors.toList());
    log.info("Successfully fetched {} reservations for waiterId: {}",

```

```

responses.size(), waiterId);
    return responses;
}

@Override
public List<CustomerReservationResponse> getCustomerReservations(String
customerId) {
    log.info("Fetching reservations for customerId={}", customerId);
    List<Booking> reservations =
reservationRepository.findAllByUserId(customerId);
    log.debug("Retrieved {} reservations for customerId={}",
reservations.size(), customerId);

    if (reservations.isEmpty()) {
        log.info("No reservations found for customerId={}", customerId);
        return Collections.emptyList();
    }

    return reservations.stream()
        .map(reservation -> {
            Location location = reservation.getLocation();
            if (location == null || location.getAddress() == null) {
                log.warn("Location not found for bookingId={} with
locationId={}",
                    reservation.getId(),
                    location != null ? location.getId() : null);
                return null;
            }

            var waiter = userRepository.findById(reservation.getWaiterId())
                .orElseThrow(() -> new EntityNotFoundException(
                    "Waiter with id '%s' not
found".formatted(reservation.getWaiterId())));

            int dishesCount = orderRepository.findById(reservation.getId())
                .map(order -> {
                    if (order.getOrderItems() == null) return 0;
                    return order.getOrderItems().stream()
                        .mapToInt(item -> item.getDishQuantity() != null ?
item.getDishQuantity() : 0)
                        .sum();
                })
                .orElse(0);

            CustomerReservationResponse response =
reservationMapper.toCustomerReservationResponse(
                reservation,
                location.getAddress(),
                waiter,
                dishesCount
            );

            boolean isFeedbackExists =
!feedbackRepository.findAllByReservationId(reservation.getId()).isEmpty();
            response.setFeedbackExists(isFeedbackExists);

            return response;
        })
        .filter(Objects::nonNull)
        .toList();
}

```

```

@Override
@Transactional
public MessageResponse complete(String id, User user) {
    log.info("in complete(), reservationId: {}, user id: {}", id,
user.getId());

    Booking reservation = findReservationById(id);
    checkWaiterRights(reservation, user);
    checkReservationToComplete(reservation);
    Order order = getOrderOrThrow(reservation.getId());
    reservation.complete();
    order.complete(OffsetDateTime.now());
    reservationRepository.save(reservation);
    orderRepository.save(order);
    return new MessageResponse("Reservation with id '%s'
completed".formatted(id));
}

@Override
public void updateReservation(String id, UpdateReservationRequestDto request,
User user) {
    log.info("in updateReservation(), reservationId: {}, updateRequest: {},
user id: {}", id, request, user.getId());

    Booking reservation = getReservationWithStatusOrThrow(id, RESERVED);
    checkAuthorization(user, reservation);
    Table table = getTableOrThrow(reservation.getLocation().getId(),
        user.getRole() == CUSTOMER ? reservation.getTableNum() :
request.tableNumber());
    checkSlots(reservation.getLocation(), new Slot(request.timeFrom(),
request.timeTo()));
    checkDateTimeValidity(LocalDate.parse(reservation.getDate()),
request.timeFrom());
    checkTableCapacity(table.getCapacity(), request.guests());
    checkTableAvailabilityToUpdate(reservation, table.getTableNum(),
reservation.getDate(),
    request.timeFrom(), request.timeTo());

    reservationMapper.updateFromUpdateReservationRequestDto(reservation,
request);
    reservationRepository.save(reservation);
}

private WaiterReservationResponse buildWaiterReservationResponse(Booking
reservation, String locationAddress, Map<String, User> userMap, Map<String, Order>
ordersMap, String waiterName) {
    log.debug("Building response for reservationId: {}", reservation.getId());

    String customerName = determineCustomerName(reservation, userMap,
waiterName);
    Order order = ordersMap.get(reservation.getId());

    return reservationMapper.toWaiterReservationsResponse(
        locationAddress, waiterName, customerName, reservation, order);
}

private String determineCustomerName(Booking reservation, Map<String, User>
userMap, String waiterName) {
    log.debug("Determining customer name for reservationId: {}",
reservation.getId());
    if (ANONYMOUS_USER_ID.equals(reservation.getUserId())) {
        log.debug("Reservation is for an anonymous user.");
        return String.format("%s (%s)", waiterName, ANONYMOUS_USER_ID);
    }
}

```

```

    }

    User user = userMap.get(reservation.getUserId());
    if (user == null) {
        log.warn(
            "Data integrity issue: User with ID {} not found for reservation
{{}. Using default.",
            reservation.getUserId(),
            reservation.getId());
        return "Unknown User";
    }
    log.debug("Customer found: {} {}", user.getFirstName(),
user.getLastName());
    return String.format("Customer %s %s", user.getFirstName(),
user.getLastName());
}

private Map<String, User> fetchAndMapUsers(List<Booking> reservations) {
    log.debug("Fetching and mapping users for {} reservations.",
reservations.size());
    Set<String> userIds =
        reservations.stream()
            .map(Booking::getUserId)
            .filter(id -> !id.equals(ANONYMOUS_USER_ID))
            .collect(Collectors.toSet());
    log.debug("Found {} unique user IDs to fetch.", userIds.size());
    List<User> users = userRepository.findAllByIdIn(userIds);
    log.debug("Fetched {} users.", users.size());
    return users.stream().collect(Collectors.toMap(User::getId, user ->
user));
}

private Map<String, Order> fetchAndMapOrders(List<Booking> reservations) {
    log.debug("Fetching and mapping orders for {} reservations.",
reservations.size());
    Set<String> reservationIds =
        reservations.stream().map(Booking::getId).collect(Collectors.toSet());
    log.debug("Found {} unique booking IDs to fetch orders for.",
reservationIds.size());
    List<Order> orders =
        orderRepository.findAllByReservationIdIn(reservationIds);
    log.debug("Fetched {} orders.", orders.size());
    return orders.stream().collect(Collectors.toMap(Order::getReservationId,
booking -> booking));
}

private void checkAuthorization(User user, Booking reservation) {
    if (user.getRole() == CUSTOMER) {
        checkUserRights(reservation, user);
    } else {
        checkWaiterRights(reservation, user);
    }
}

private Booking getReservationWithStatusOrThrow(String id, BookingStatus
status) {
    return reservationRepository.findByIdAndStatus(id, status)
        .orElseThrow(() -> new EntityNotFoundException("Reservation '%s' not
found"
            .formatted(id)));
}

private Location getLocationOrThrow(String locationId) {

```

```

        return locationRepository.findById(locationId)
            .orElseThrow(() -> new EntityNotFoundException("Location '%s' not
found".formatted(locationId)));
    }

    private Table getTableOrThrow(String locationId, Integer tableNumber) {
        return tableRepository.findByLocationIdAndTableNum(locationId,
tableNumber)
            .orElseThrow(() -> new EntityNotFoundException("Table '%s' for
location '%s' not found"
                .formatted(tableNumber, locationId)));
    }

    private void validateReservation(BaseReservationRequest request, Table table,
Location location) {
        checkTableCapacity(table.getCapacity(), request.guestsNumber());
        checkDateTimeValidity(request.date(), request.timeFrom());
        checkSlots(location, new Slot(request.timeFrom(), request.timeTo()));
        checkTableAvailabilityOnTime(request.locationId(), request.tableNumber(),
request.date().toString(),
            request.timeFrom(), request.timeTo());
    }

    private void checkTableAvailabilityOnTime(String locationId, Integer
tableNumber, String date, String timeFrom, String timeTo) {
        if
(reservationRepository.existsByLocationIdAndDateAndTableNumAndStatusInAndStartTime
AndEndTime(locationId, date, tableNumber,
            List.of(RESERVED, IN_PROGRESS), timeFrom, timeTo)) {
            throw new ReservationException("Reservation slot is not available");
        }
    }

    private void checkSlots(Location location, Slot slot) {
        if (location.getSlots().stream().noneMatch(s -> s.equals(slot))) {
            throw new ReservationException("Invalid Slot for location " +
location.getId());
        }
    }

    private void checkTableCapacity(Integer tableCapacity, Integer guests) {
        if (tableCapacity < guests) {
            throw new ReservationException("Table capacity '%s' exceeded for
guests number '%s'"
                .formatted(tableCapacity, guests));
        }
    }

    private User findLeastBusyWaiter(Booking booking) {
        List<Booking> activeBooking =
reservationRepository.getAllActiveByLocationIdOnDateTime(booking.getLocation().get
Id(),
            booking.getDate(), booking.getStartTime(), booking.getEndTime());
        List<String> waiterEmails =
waiterRepository.findAllByLocationId(booking.getLocation().getId())
            .stream().map(Waiter::getEmail).toList();
        List<User> waiterUserList = userRepository.findAllByEmailIn(waiterEmails);

        return waiterUserList.stream()
            .collect(Collectors.toMap(key -> key,
                value -> activeBooking.stream()
                    .filter(b -> b.getWaiterId().equals(value.getId()))
                    .count()))
    }

```

```

        .entrySet().stream()
        .min(Map.Entry.comparingByValue())
        .map(Map.Entry::getKey)
        .orElseThrow(() -> new EntityNotFoundException("No active waiters for
location '%s' found"
            .formatted(booking.getLocation().getId())));
    }

    private void checkUserRights(Booking reservation, User user) {
        checkOwner(reservation, user);
        checkTimeToDeleteOrModify(reservation);
    }

    private void checkOwner(Booking reservation, User user) {
        if (!reservation.getUserId().equals(user.getId())) {
            throw new AuthorizationException("User '%s' is not owner of
reservation '%s'"
                .formatted(user.getId(), reservation.getId()));
        }
    }

    private void checkTimeToDeleteOrModify(Booking reservation) {
        if (!reservation.canBeDeletedOrModified()) {
            throw new ReservationException("Can't cancel/modify a reservation '%s'
less than 30 min before the start."
                .formatted(reservation.getId()));
        }
    }

    private void checkWaiterRights(Booking reservation, User user) {
        if (!reservation.getWaiterId().equals(user.getId())) {
            throw new AuthorizationException("Waiter '%s' is not authorized to
perform action on reservation '%s'"
                .formatted(user.getId(), reservation.getId()));
        }
    }

    private void checkDateTimeValidity(LocalDate date, String startTime) {
        var startDateTime = LocalDateTime.of(date, LocalTime.parse(startTime));
        if (startDateTime.isBefore(LocalDateTime.now(AppConstant.ZONE_ID))) {
            throw new ReservationException("Time for reservation cannot be in the
past");
        }
    }

    private void checkReservationToComplete(Booking reservation) {
        if (!reservation.isActive()) {
            throw new ReservationException("Reservation '%s' is already cancelled
or finished"
                .formatted(reservation.getId()));
        }
        if (!reservation.isStarted()) {
            throw new ReservationException("Reservation '%s' is not
started".formatted(reservation.getId()));
        }
    }

    private Order getOrderOrThrow(String orderId) {
        return orderRepository.findById(orderId)
            .orElseThrow(() -> new EntityNotFoundException("No order found for
orderId '%s'"
                .formatted(orderId)));
    }

```

```

        private void cancelOrderIfPresent(String reservationId) {
            orderRepository.findAndSetStatusByReservationId(reservationId,
                OrderStatus.CANCELLED);
        }

        private void checkTableAvailabilityToUpdate(Booking reservation, Integer
            tableNumber, String date, String timeFrom, String timeTo) {
            if (!reservation.getTableNum().equals(tableNumber)
                || !isSameSlot(reservation.getStartTime(), timeFrom,
                    reservation.getEndTime(), timeTo)) {
                checkTableAvailabilityOnTime(reservation.getLocation().getId(),
                    tableNumber, date, timeFrom, timeTo);
            }
        }

        private boolean isSameSlot(String firstFrom, String secondFrom, String
            firstTo, String secondTo) {
            return firstFrom.equals(secondFrom) && firstTo.equals(secondTo);
        }
    }

package com.tastetales.service.impl;

import com.tastetales.constant.AppConstant;
import com.tastetales.dto.TableListResponse;
import com.tastetales.dto.filter.*;
import com.tastetales.dto.response.TableSlotsResponseDto;
import com.tastetales.entity.*;
import com.tastetales.exception.EntityNotFoundException;
import com.tastetales.mapper.TableMapper;
import com.tastetales.repository.*;
import com.tastetales.service.*;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.*;
import java.util.stream.Collectors;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;

@Slf4j
@Service
@RequiredArgsConstructor
public class TableServiceImpl implements TableService {
    private final TableRepository tableRepository;
    private final LocationService locationService;
    private final ReservationRepository reservationRepository;
    private final TableMapper tableMapper;

    @Override
    public TableListResponse getAvailableTables(String date, String locationId,
        Integer guests, String time) {
        Map<String, String> params = new HashMap<>();
        params.put("locationId", locationId != null ? locationId : "");
        params.put("date", date);
        params.put("time", time);
        params.put("guests", guests != null ? String.valueOf(guests) : "");
    }

```

```

TableFilterDto filterDto = TableFilterDto.of(params);
log.debug("Parsed filterDto: {}", filterDto);
List<TableResponseDto> result = new ArrayList<>();
List<Location> locations = (locationId == null || locationId.isBlank())
    ? locationService.getLocationsForTableCheck()
    : List.of(locationService.getById(locationId));

List<Booking> activeBookings = reservationRepository.findAllActive();
log.debug("Locations: {}", locations);
for (Location location : locations) {
    List<Slot> filteredSlots = filterSlots(location.getSlots(),
filterDto.getFromDateTime());
    List<Table> tables =
tableRepository.findAllByLocationId(location.getId());
    List<Booking> locationBookings = activeBookings.stream()
        .filter(b -> location.getId().equals(b.getLocation().getId()))
        .filter(b -> filterDto.getDate().equals(b.getDate()))
        .toList();
    for (Table table : tables) {
        Integer guestCount = filterDto.getGuests();
        if (guestCount == null || guestCount <= 0 || table.getCapacity()
>= guestCount) {
            List<Booking> bookings = locationBookings.stream()
                .filter(b -> Objects.equals(table.getTableNum(),
b.getTableNum()))
                .toList();
            List<Slot> availableSlots =
filterNonOverlapping(filteredSlots, bookings);
            if (!availableSlots.isEmpty()) {
                TableResponseDto dto =
tableMapper.toTableResponseDto(location, table, availableSlots);
                result.add(dto);
            }
        }
    }
}
return new TableListResponse(result);
}

@Override
public TableSlotsResponseDto getAvailableSlotsOnDate(String id, Integer
tableNumber, TableOnDateFilterDto filterDto) {
    Location location = locationService.getById(id);
    List<Slot> slots = getUpcomingSlots(location, filterDto.date());
    Table table = getTableOrThrow(id, tableNumber);
    List<Slot> busySlotsForTableOnDate =
getBusySlotsForTableOnDate(tableNumber, filterDto, location);
    List<String> availableSlots = filterAvailable(slots,
busySlotsForTableOnDate);
    return new TableSlotsResponseDto(table.getCapacity(), availableSlots);
}

private List<Slot> filterSlots(List<Slot> slots, LocalDateTime fromDateTime) {
    System.out.println(slots);
    return slots.stream()
        .map(s -> new Slot(s.getFrom(), s.getTo()))
        .filter(slot -> {
            LocalTime slotStart = LocalTime.parse(slot.getFrom());
            LocalDateTime slotStartDateTime =
LocalDateTime.of(fromDateTime.toLocalDate(), slotStart);
            return !slotStartDateTime.isBefore(fromDateTime);
        })
        .toList();
}

```



```

    private List<Slot> filterNonOverlapping(List<Slot> slots, List<Booking>
bookings) {
        List<Slot> bookingSlots = bookings.stream()
            .map(b -> new Slot(b.getStartTime(), b.getEndTime()))
            .toList();
        return slots.stream()
            .filter(slot -> bookingSlots.stream().noneMatch(booked ->
overlaps(slot, booked)))
            .toList();
    }

    private boolean overlaps(Slot a, Slot b) {
        LocalTime aStart = LocalTime.parse(a.getFrom());
        LocalTime aEnd = LocalTime.parse(a.getTo());
        LocalTime bStart = LocalTime.parse(b.getFrom());
        LocalTime bEnd = LocalTime.parse(b.getTo());
        return aStart.isBefore(bEnd) && bStart.isBefore(aEnd);
    }

    private List<Slot> getUpcomingSlots(Location location, String dateFromFilter)
{
        LocalDate date = LocalDate.parse(dateFromFilter);
        if (date.isAfter(LocalDate.now(AppConstant.ZONE_ID))) {
            return location.getSlots();
        }
        LocalTime now = LocalTime.now(AppConstant.ZONE_ID);
        return location.getSlots().stream()
            .filter(slot -> {
                try {
                    LocalTime fromTime = LocalTime.parse(slot.getFrom());
                    return fromTime.isAfter(now);
                } catch (Exception e) {
                    return false;
                }
            })
            .collect(Collectors.toList());
    }

    private List<String> filterAvailable(List<Slot> tableSlots, List<Slot>
busySlotsForTableOnDate) {
        return tableSlots.stream()
            .filter(slot -> !busySlotsForTableOnDate.contains(slot))
            .map(Slot::concat)
            .toList();
    }

    private List<Slot> getBusySlotsForTableOnDate(Integer tableNumber,
TableOnDateFilterDto filterDto, Location location) {
        return reservationRepository
            .findAllByLocationIdAndDateAndTableNum(location.getId(),
filterDto.date(), tableNumber)
            .stream()
            .map(b -> new Slot(b.getStartTime(), b.getEndTime()))
            .toList();
    }

    private Table getTableOrThrow(String id, Integer tableNumber) {
        return tableRepository.findByLocationIdAndTableNum(id, tableNumber)
            .orElseThrow(() -> new EntityNotFoundException("Table '%s' not
found for location '%s'".formatted(tableNumber, id)));
    }
}

```